An Investigation of Five Implementations of GBoost and XGBoost Algoritms On A Safety Problem

January 13, 2021

Gradient boosting (GBoost) is a powerful ensemble machine learning algorithm. It generates a prediction model that evolves weak prediction models (i.e decision trees). The weak prediction models added one at a time to the prediction model and fit to correct the prediction errors made by prior models. This is a type of ensemble machine learning model referred to as boosting. Models are fit using any arbitrary differentiable loss function and gradient descent optimization algorithm. This gives the technique its name, "gradient boosting," as the loss gradient is minimized as the model is fit, much like a neural network. This approach makes the GBoost one of the most powerful techniques for building predictive models for classification or regression problems.

Extreme Gradient Boosting (XGBoost) is an efficient implementation of the gradient boosting algorithm. It is built on the principles of gradient boosting framework and designed to "push the extreme of the computation limits of machines to provide a scalable, portable and accurate library." Its efficient performance makes the algorithm one of the main algorithms used in winning solutions to machine learning competitions, like those on Kaggle.

This investigation applies the GBoost and the XGBoost algorithms for a safety problem. The GBoost and the XGBoost algorithms are applied to find important features that are critical in safety prediction thought both classification and regression approaches. Thus, the investigation is sepeared into two sections for the classification and the regression problem. In each section, five mainstream implementations of the GBoost and the XGBoost are applied to build prediction model with feature importances. The five implemetation are:

- GBoost Algorithm
  - Scikit-Learn Gradient Boosting Approach
    - GradientBoostingClassifier
    - GradientBoostingRegressor
  - Scikit-Learn Histogram-based Gradient Boosting Approach
    - HistGradientBoostingClassifier
    - HistGradientBoostingRegressor
- XGBoost Algorithm
  - XGBoost
    - XGBClassifier
    - XGBRFRegressor
  - LightGBM
    - LGBMClassifier
    - LGBMRegressor
  - Catboost

    - CatBoostClassifier
    - CatBoostRegressor
  - H2O

To provide an investigation as exhaustive as possible, H2O is also mentioned here as another major XGBoost implementation. As its high-performance on GPU, H2O worthes a try in the future, although its install requirements are beyond the reach of this investigation.

The implementations of the XGBoost Algorithm also introduce a hyperparameter-tuning process for parameter optimization. The implementations of XGBoost, LightGBM and Catboost have different hyperparameters. Thus, various subsets of these hyperparameters are selected for the tuning. As the project may face a complex dataset, an elaboratedhyperparameter-tuning process will be necessary for the future.

The first section applies the GBoost and the XGBoost algorithms for classification problem. After training default models and optimised models, various feature importance are generated. The rankings of feature importance for the classification problem are summarised in the following table.

[1]:
```
# Put the "Feature_importance_ranking_classifiers.csv" file in the "result"␣
↪directory. Or run this cell after this ipynb file reaches the summary of first␣
↪section.
#import pandas as pd

#read_rank_classifiers = pd.read_csv('result/
↪Feature_importance_ranking_classifiers.csv', index_col="Features")
#read_rank_classifiers
```

[2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize = (12, 6))

# hide axes
fig.patch.set_visible(False)
ax.axis('off')
#ax.axis('tight')
read_rank_classifiers = pd.read_csv('result/
↪Feature_importance_ranking_classifiers.csv')
ax.table(cellText=read_rank_classifiers.values, colLabels=read_rank_classifiers.
↪columns, loc='center')
#fig.tight_layout()
plt.show()
```

Features	GradientBoosting	XGBoost (Optimised)	CatBoost (Optimised)
TrainingCount5Years	1	1.0	12
NM_Rolling12NMCount	2	nan	1
NM_MonthlyCount	3	2.0	8
MaxActualDays	4	nan	13
MaxReqDays	5	nan	4
Rolling12DelayedActionCnt	6	4.0	5
DelayedPerEmp	7	nan	2
AvgReqDays	8	7.0	11
AvgActualDays	9	6.0	9
MaxDelayedDays	10	3.0	6
MinReqDays	11	nan	7
AvgDelayedDays	12	5.0	10
AuditCount5Years	13	nan	14
MinActualDays	14	nan	3
MinDelayedDays	15	nan	15

The second section also applies the GBoost and the XGBoost algorithms for regression problem. After training default models and optimised models, various feature importance are generated. The rankings of feature importance for the regression problem are summarised in the following table.

[3]:
```
# Put the "Feature_importance_ranking_regressor.csv" file in the "result"␣
↪directory. Or run this cell after this ipynb file reaches the summary of␣
↪second section.
#read_rank_regressor = pd.read_csv('result/Feature_importance_ranking_regressor.
↪csv', index_col="Features")
#read_rank_regressor
```

[4]:
```
fig, ax = plt.subplots(figsize = (12, 6))

# hide axes
fig.patch.set_visible(False)
ax.axis('off')
#ax.axis('tight')
read_rank_classifiers = pd.read_csv('result/Feature_importance_ranking_regressor.
↪csv')
ax.table(cellText=read_rank_classifiers.values, colLabels=read_rank_classifiers.
↪columns, loc='center')
#fig.tight_layout()
plt.show()
```

Features	GradientBoosting	XGBoost (Optimised)	CatBoost (Optimised)
Rolling12DelayedActionCnt	1	1.0	1
DelayedPerEmp	2	nan	2
AvgReqDays	3	3.0	9
NM_Rolling12NMCount	4	5.0	6
AvgActualDays	5	6.0	5
AvgDelayedDays	6	2.0	8
MinActualDays	7	nan	7
MinReqDays	8	nan	11
MaxDelayedDays	9	nan	3
NM_MonthlyCount	10	nan	12
MaxReqDays	11	nan	4
AuditCount5Years	12	nan	15
TrainingCount5Years	13	4.0	13
MaxActualDays	14	nan	10
MinDelayedDays	15	nan	14

1 Library Preparation

This investigation requires these libraries: sklearn, xgboost, lightbgm, and catboost.

The current version of sklearn is 0.23.2.

[193]:
```
import sklearn
print(sklearn.__version__)
```

```
0.23.2
```

The current version of xgboost is 1.3.1.

[195]:
```
import xgboost
print(xgboost.__version__)
```

```
1.3.1
```

Run this line to install the xgboost library.

[ ]:
```
#! pip install xgboost
```

The current version of lightgbm is 3.1.1.

[196]:
```
import lightgbm
print(lightgbm.__version__)
```

```
3.1.1
```

Run this line to install the lightgbm library.

[ ]:
```
#!conda install -c conda-forge lightgbm
```

The current version of lightgbm is 0.24.4.

[197]:
```
import catboost
print(catboost.__version__)
```

```
0.24.4
```

Run this line to install the catboost library.

[ ]:
```
# conda install -c conda-forge catboost
```

2 Section 1: Classification

The first section applies the GBoost and the XGBoost algorithms for classification problem.

2.1 Data Preprocessing

2.1.1 Import Libraries

[3]:
```
import numpy as np
import pandas as pd
from time import time
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from scipy import stats
```

2.1.2 Import Dataset

[4]:
```
#import dataset
data = pd.read_csv('dataset/impact.csv')
# split the traing and testing datasets
X = data.drop('TRIFR',1)
y = data['TRIFR']
```

2.1.3 Transforms 'Y' value into labels for classificaiton

Transforms 'TRIFR' value into label for the classification problem. Set the threshold at the 35% distribution of the 'TRIFR' value.

[5]:
```
threshold = np.sort(y)[int(np.ceil(len(y)* 0.35))]
threshold
```

[5]: 4.787116911

Label "0" if the value is lower or equal to this threshold, otherwise label "1".

[6]:
```
y= np.where(y<=threshold, 0, 1)
```

2.1.4 Skew "X" values

For normally distributed data, the skewness of the data should be about 0. A skewness value > 0 means that there is more weight in the left tail of the distribution, and vice versa for a skewness value < 0. The log method can be applied to decrease the skewness of the data, which should be positive values. In this case, columns of `AvgDelayedDays` and `MinDelayedDays` are selected to correct their skewness. The skewness value of `AvgDelayedDays` decreases from 2.19824437 to 1.57089682. Yet the skewness value of `MinDelayedDays` remains the same (3.47439614) because it is a high bias data. Only one instance has a value of 2 in this feature/attribute, while the rest have another value of 1. This extreme case shows the effect of bias data on its skewness value.

transform features with skewness > 1 or skewness < -1

[7]:
```
import copy
X_skewed = copy.deepcopy(X)
for col in range(X_skewed.shape[1]):
    if stats.mstats.skew(X_skewed.iloc[:, col]).data >1 or stats.mstats.
 ↪skew(X_skewed.iloc[:, col]).data < -1 :
        if len(X_skewed[X_skewed.iloc[:, col]>0]) < X_skewed.shape[0]:
            # to shew, each value in the column should be positive number
            pass
        else:
            X_skewed.iloc[:, col] = np.log(X_skewed.iloc[:, col])
    else:
        pass
```

[8]:
```
stats.mstats.skew(X).data
```

[8]:
```
array([-0.35461034, -0.32561725,  2.19824437,  3.47439614,  0.05099458,
        0.03762476,  0.16789008, -0.92026225, -0.57649107, -0.407614  ,
       -0.60945159,  0.77786451, -0.64815153,  0.46823692,  1.79700391])
```

[9]:
```
stats.mstats.skew(X_skewed).data
```

[9]:
```
array([-0.35461034, -0.32561725,  1.57089682,  3.47439614,  0.05099458,
        0.03762476,  0.16789008, -0.92026225, -0.57649107, -0.407614  ,
       -0.60945159,  0.77786451, -0.64815153,  0.46823692,  1.79700391])
```

[10]:
```
X
```

[10]:
```
    Rolling12DelayedActionCnt  DelayedPerEmp  AvgDelayedDays  MinDelayedDays  \
0                         132           0.61              65               1
1                          92           0.42              78               1
2                          90           0.41              81               1
```

3	89	0.41	82	1
4	132	0.61	65	1
5	131	0.60	65	1
6	134	0.62	64	1
7	134	0.62	62	1
8	117	0.54	59	1
9	118	0.54	91	1
10	92	0.42	79	1
11	81	0.38	139	2
12	63	0.30	64	1
13	51	0.24	64	1
14	50	0.23	55	1

	MaxDelayedDays	AvgReqDays	MinReqDays	MaxReqDays	AvgActualDays \
0	699	146	-699	619	212
1	528	78	-528	577	156
2	528	66	-528	577	148
3	528	58	-528	371	141
4	699	146	-699	619	211
5	699	153	-699	619	218
6	699	157	-699	619	222
7	699	161	-699	619	223
8	457	170	-204	577	230
9	528	106	-528	577	198
10	528	85	-528	577	164
11	952	67	-952	688	206
12	272	52	-216	345	116
13	272	25	-216	304	90
14	272	31	-216	304	87

	MinActualDays	MaxActualDays	NM_MonthlyCount	NM_Rolling12NMCount \
0	-52	708	2	12
1	0	581	1	31
2	0	581	3	33
3	0	428	4	35
4	-52	708	1	20
5	-52	708	2	22
6	-52	708	2	24
7	-52	708	4	27
8	-52	708	2	27
9	0	581	6	31
10	0	581	5	35
11	0	713	2	29
12	-1	487	6	38
13	-1	385	2	40
14	-1	385	2	38

```
    TrainingCount5Years  AuditCount5Years
0                     0                 0
1                     5                 9
2                     3                 0
3                     0                 0
4                     0                 0
5                     3                11
6                     1                 0
7                     3                 0
8                     2                 4
9                     3                12
10                    2                29
11                    0                 0
12                    2                11
13                    0                 0
14                    0                 0
```

[11]:
```
X_skewed
```

[11]:
```
    Rolling12DelayedActionCnt  DelayedPerEmp  AvgDelayedDays  MinDelayedDays  \
0                         132           0.61        4.174387        0.000000
1                          92           0.42        4.356709        0.000000
2                          90           0.41        4.394449        0.000000
3                          89           0.41        4.406719        0.000000
4                         132           0.61        4.174387        0.000000
5                         131           0.60        4.174387        0.000000
6                         134           0.62        4.158883        0.000000
7                         134           0.62        4.127134        0.000000
8                         117           0.54        4.077537        0.000000
9                         118           0.54        4.510860        0.000000
10                         92           0.42        4.369448        0.000000
11                         81           0.38        4.934474        0.693147
12                         63           0.30        4.158883        0.000000
13                         51           0.24        4.158883        0.000000
14                         50           0.23        4.007333        0.000000

    MaxDelayedDays  AvgReqDays  MinReqDays  MaxReqDays  AvgActualDays  \
0              699         146        -699         619            212
1              528          78        -528         577            156
2              528          66        -528         577            148
3              528          58        -528         371            141
4              699         146        -699         619            211
5              699         153        -699         619            218
6              699         157        -699         619            222
7              699         161        -699         619            223
8              457         170        -204         577            230
9              528         106        -528         577            198
```

```
10              528          85        -528         577             164
11              952          67        -952         688             206
12              272          52        -216         345             116
13              272          25        -216         304              90
14              272          31        -216         304              87

    MinActualDays  MaxActualDays  NM_MonthlyCount  NM_Rolling12NMCount  \
0             -52            708                2                   12
1               0            581                1                   31
2               0            581                3                   33
3               0            428                4                   35
4             -52            708                1                   20
5             -52            708                2                   22
6             -52            708                2                   24
7             -52            708                4                   27
8             -52            708                2                   27
9               0            581                6                   31
10              0            581                5                   35
11              0            713                2                   29
12             -1            487                6                   38
13             -1            385                2                   40
14             -1            385                2                   38

    TrainingCount5Years  AuditCount5Years
0                     0                 0
1                     5                 9
2                     3                 0
3                     0                 0
4                     0                 0
5                     3                11
6                     1                 0
7                     3                 0
8                     2                 4
9                     3                12
10                    2                29
11                    0                 0
12                    2                11
13                    0                 0
14                    0                 0
```

[12]:
```
X = X_skewed
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=0)
```

[13]:
```
#keep record for comparison
accuracy = {}
speed = {}
```

2.2 GBoost: Scikit-Learn Gradient Boosting Approach

This part applies GradientBoostingClassifier from the Scikit-Learn package for the classification problem.

2.2.1 Import Libraries

[14]:
```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RepeatedStratifiedKFold
```

2.2.2 Import Model

[15]:
```
model = GradientBoostingClassifier()
```

2.2.3 Cross-validation

Cross-validation is conduected here to test the average performance of the default model on the dataset. The average of score and time consumption in the iterations of cross-validation are record for a comparison with other implementations.

[16]:
```
start = time()
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=50, random_state=0)
score = cross_val_score(model, X, y, scoring="accuracy", cv=cv, n_jobs=-1)

speed["GradientBoosting"] = np.round(time() - start, 3)
accuracy["GradientBoosting"] = np.nanmean(score).round(3)

print(
    f"Mean Accuracy: {accuracy['GradientBoosting']}\nStd: {np.std(score):.
 ↪3f}\nRun time: {speed['GradientBoosting']}s"
)
```

```
Mean Accuracy: 0.633
Std: 0.257
Run time: 7.093s
```

It argues that this dataset is extremely small, so n_splits is set to 5 and n_repeats is 50 for a robust result for a model with default parameters. The score values of these 50 repeats and their high variance (see below) can support this argument. Some reaches 100% accuracy while others just reach 33%. To decrease this variance, a dataset with a decent amount of the instance will be helpful. In addition, optimization of the model's parameters will also lead to a better, more consistent results of cross-validation.

[17]:
```
score
```

```
[17]: array([0.33333333, 0.66666667, 1.        , 0.33333333, 0.66666667,
             0.66666667, 0.33333333, 0.66666667, 0.66666667, 1.        ,
             0.66666667, 0.33333333, 1.        , 0.33333333, 0.66666667,
             0.33333333, 1.        , 1.        , 0.33333333, 0.33333333,
             0.66666667, 0.66666667, 0.66666667, 0.66666667, 0.33333333,
             0.33333333, 0.66666667, 1.        , 1.        , 0.66666667,
             0.33333333, 0.66666667, 1.        , 0.66666667, 1.        ,
             0.66666667, 0.        , 1.        , 0.33333333, 0.33333333,
             0.66666667, 1.        , 0.66666667, 0.33333333, 0.33333333,
             0.66666667, 0.66666667, 1.        , 0.66666667, 0.33333333,
             1.        , 1.        , 0.66666667, 0.66666667, 0.33333333,
             0.33333333, 0.66666667, 0.66666667, 0.66666667, 0.66666667,
             1.        , 0.33333333, 1.        , 0.33333333, 0.66666667,
             0.66666667, 1.        , 1.        , 0.        , 0.33333333,
             0.66666667, 0.66666667, 0.33333333, 0.66666667, 0.66666667,
             0.33333333, 0.66666667, 1.        , 0.33333333, 1.        ,
             0.66666667, 0.66666667, 0.66666667, 0.33333333, 0.33333333,
             0.66666667, 0.33333333, 1.        , 0.66666667, 1.        ,
             1.        , 1.        , 0.33333333, 0.66666667, 0.33333333,
             1.        , 0.33333333, 0.66666667, 0.66666667, 0.66666667,
             0.33333333, 0.66666667, 0.33333333, 0.66666667, 0.33333333,
             0.66666667, 0.66666667, 0.33333333, 0.66666667, 0.66666667,
             1.        , 0.66666667, 0.66666667, 0.66666667, 1.        ,
             1.        , 0.66666667, 0.66666667, 0.66666667, 0.66666667,
             0.66666667, 0.66666667, 0.33333333, 0.66666667, 0.66666667,
             0.66666667, 0.33333333, 1.        , 0.66666667, 0.66666667,
             0.66666667, 1.        , 1.        , 0.66666667, 0.        ,
             0.66666667, 1.        , 0.66666667, 0.66666667, 0.66666667,
             0.33333333, 1.        , 0.66666667, 0.        , 0.66666667,
             0.33333333, 0.66666667, 1.        , 0.66666667, 0.66666667,
             0.        , 1.        , 0.        , 1.        , 0.66666667,
             0.66666667, 0.66666667, 0.66666667, 0.66666667, 0.33333333,
             0.33333333, 1.        , 1.        , 0.66666667, 0.33333333,
             1.        , 0.66666667, 0.33333333, 0.33333333, 0.33333333,
             0.33333333, 0.66666667, 1.        , 0.66666667, 0.66666667,
             0.33333333, 0.66666667, 0.66666667, 0.66666667, 0.66666667,
             0.66666667, 1.        , 0.        , 0.66666667, 0.33333333,
             0.33333333, 1.        , 0.33333333, 0.66666667, 0.66666667,
             0.66666667, 0.66666667, 0.66666667, 0.66666667, 0.66666667,
             0.66666667, 1.        , 1.        , 0.33333333, 0.66666667,
             0.66666667, 1.        , 0.33333333, 0.66666667, 0.33333333,
             0.66666667, 0.33333333, 1.        , 0.33333333, 0.66666667,
             1.        , 1.        , 1.        , 0.66666667, 0.66666667,
             0.66666667, 0.33333333, 1.        , 0.66666667, 0.33333333,
             0.33333333, 0.66666667, 1.        , 0.66666667, 0.33333333,
             1.        , 0.33333333, 0.66666667, 0.33333333, 0.33333333,
             1.        , 0.66666667, 0.66666667, 0.66666667, 0.33333333,
```

```
       0.66666667, 0.66666667, 0.66666667, 0.66666667, 1.        ,
       0.33333333, 0.66666667, 0.33333333, 0.33333333, 1.        ,
       0.66666667, 1.        , 0.33333333, 0.66666667, 0.33333333])
```

2.2.4 Fit The Model

[18]:
```
# fit the model on the whole dataset
model.fit(X, y)
```

[18]:
```
GradientBoostingClassifier()
```

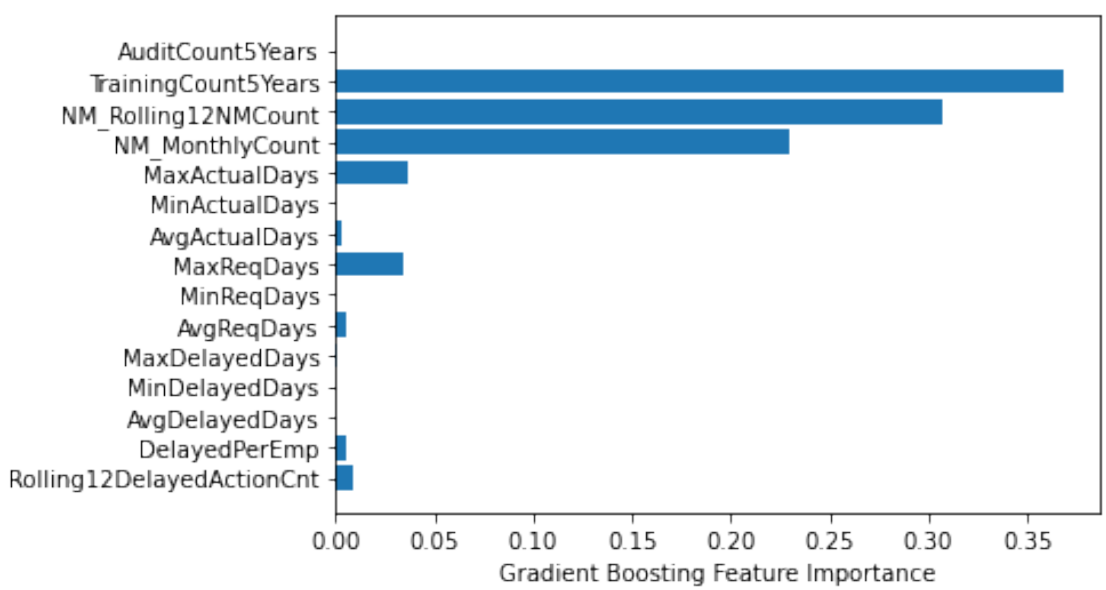
2.2.5 Plot Feature Importance

[19]:
```
print("The impurity-based feature importances :\n", model.feature_importances_)
```

```
The impurity-based feature importances :
 [8.86975041e-03 5.65095295e-03 3.56238638e-23 0.00000000e+00
 3.55637255e-04 4.99261638e-03 2.73380170e-21 3.44585264e-02
 3.09767989e-03 0.00000000e+00 3.69549574e-02 2.29591837e-01
 3.07405594e-01 3.68622449e-01 0.00000000e+00]
```

[20]:
```
plt.barh(X.columns, model.feature_importances_)
plt.xlabel("Gradient Boosting Feature Importance")
```

[20]:
```
Text(0.5, 0, 'Gradient Boosting Feature Importance')
```

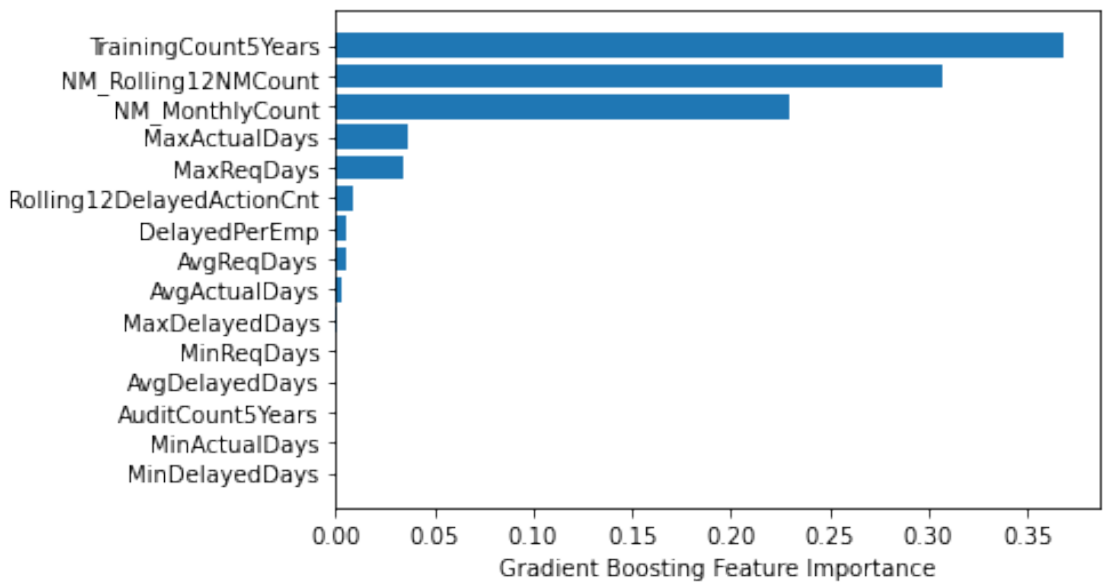


[21]:
```
sorted_idx = model.feature_importances_.argsort()
sorted_idx
```

[21]:
```
array([ 3,  9, 14,  2,  6,  4,  8,  5,  1,  0,  7, 10, 11, 12, 13],
      dtype=int64)
```

[22]:
```
sorted_idx = model.feature_importances_.argsort()
plt.barh(X.columns[sorted_idx], model.feature_importances_[sorted_idx])
plt.xlabel("Gradient Boosting Feature Importance")
```

[22]:
```
Text(0.5, 0, 'Gradient Boosting Feature Importance')
```



[23]:
```
rank = pd.DataFrame( (np.arange(1,len(model.
↪feature_importances_[sorted_idx])+1,1)),index=X.columns[sorted_idx][::-1],␣
↪columns=["GradientBoosting"])
print("Feature Importance Ranking:")
rank
```

```
Feature Importance Ranking:
```

[23]:

	GradientBoosting
TrainingCount5Years	1
NM_Rolling12NMCount	2
NM_MonthlyCount	3
MaxActualDays	4
MaxReqDays	5
Rolling12DelayedActionCnt	6
DelayedPerEmp	7

```
AvgReqDays                                  8
AvgActualDays                               9
MaxDelayedDays                             10
MinReqDays                                 11
AvgDelayedDays                             12
AuditCount5Years                           13
MinActualDays                              14
MinDelayedDays                             15
```

[ ]:

```

```

[ ]:

```

```

[ ]:

```

```

2.3 Scikit-Learn Histogram-based Gradient Boosting Approach

This part implements a histogram-based gradient boosting Approach for the classification problem. The HistGradientBoostingClassifier also comes from the Scikit-Learn package.

2.3.1 Import Libraries

[24]:

```
from sklearn.experimental import enable_hist_gradient_boosting
from sklearn.ensemble import HistGradientBoostingClassifier
```

2.3.2 Import Model

[25]:

```
model = HistGradientBoostingClassifier()
```

2.3.3 Cross-validation

[26]:

```
start = time()
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=50, random_state=0)
score = cross_val_score(model, X, y, scoring="accuracy", cv=cv, n_jobs=-1)

speed["HistGradientBoosting"] = np.round(time() - start, 3)
accuracy["HistGradientBoosting"] = np.nanmean(score).round(3)

print(
    f"Mean Accuracy: {accuracy['HistGradientBoosting']}\nStd: {np.std(score):.
 ↪3f}\nRun time: {speed['HistGradientBoosting']}s"
)
```

```
Mean Accuracy: 0.333
Std: 0.000
Run time: 2.144s
```

[27]:
```
score
```

[27]:
```
array([0.33333333, 0.33333333, 0.33333333, 0.33333333, 0.33333333,
       
```

```
       0.33333333, 0.33333333, 0.33333333, 0.33333333, 0.33333333,
       0.33333333, 0.33333333, 0.33333333, 0.33333333, 0.33333333,
       0.33333333, 0.33333333, 0.33333333, 0.33333333, 0.33333333,
       0.33333333, 0.33333333, 0.33333333, 0.33333333, 0.33333333,
       0.33333333, 0.33333333, 0.33333333, 0.33333333, 0.33333333])
```

[ ]:

```
```

2.3.4 fit the model

[28]:

```
# fit the model on the whole dataset
model.fit(X, y)
```

[28]:

```
HistGradientBoostingClassifier()
```

2.3.5 Feature Importance

As an experimental approach, feature importance is not provided by this approach (see API and attribute of the model below), although such requests can be found in the community (https://github.com/scikit-learn/scikit-learn/issues/16064).

[29]:

```
dir(model)
```

[29]:

```
['_VALID_LOSSES',
 '__abstractmethods__',
 '__class__',
 '__delattr__',
 '__dict__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__le__',
 '__lt__',
 '__module__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
```

```
'__setattr__',
'__setstate__',
'__sizeof__',
'__slots__',
'__str__',
'__subclasshook__',
'__weakref__',
'_abc_impl',
'_baseline_prediction',
'_bin_data',
'_check_early_stopping_loss',
'_check_early_stopping_scorer',
'_check_n_features',
'_clear_state',
'_compute_partial_dependence_recursion',
'_encode_y',
'_estimator_type',
'_get_loss',
'_get_param_names',
'_get_small_trainset',
'_get_tags',
'_is_fitted',
'_more_tags',
'_predictors',
'_print_iteration_stats',
'_random_seed',
'_raw_predict',
'_repr_html_',
'_repr_html_inner',
'_repr_mimebundle_',
'_should_stop',
'_use_validation_data',
'_validate_data',
'_validate_parameters',
'bin_mapper_',
'classes_',
'decision_function',
'do_early_stopping_',
'early_stopping',
'fit',
'get_params',
'l2_regularization',
'learning_rate',
'loss',
'loss_',
'max_bins',
'max_depth',
```

```
 'max_iter',
 'max_leaf_nodes',
 'min_samples_leaf',
 'monotonic_cst',
 'n_features_',
 'n_features_in_',
 'n_iter_',
 'n_iter_no_change',
 'n_trees_per_iteration_',
 'predict',
 'predict_proba',
 'random_state',
 'score',
 'scorer_',
 'scoring',
 'set_params',
 'tol',
 'train_score_',
 'validation_fraction',
 'validation_score_',
 'verbose',
 'warm_start']
```

[ ]:

```
```

[ ]:

```
```

2.3.6 Performance Comparison

[30]:

```
print("Accuracy:")
{k: v for k, v in sorted(accuracy.items(), key=lambda i: i[1], reverse=True)}
```

```
Accuracy:
```

[30]:

```
{'GradientBoosting': 0.633, 'HistGradientBoosting': 0.333}
```

[31]:

```
print("Speed:")
{k: v for k, v in sorted(speed.items(), key=lambda i: i[1], reverse=False)}
```

```
Speed:
```

[31]:

```
{'HistGradientBoosting': 2.144, 'GradientBoosting': 7.093}
```

"Instead of finding the split points on the sorted feature values, histogram-based algorithm buckets continuous feature values into discrete bins and uses these bins to construct feature histograms during training. (https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html)"

As a result, this histogram-based approach runs much faster than its original approach. The time consumption of the histogram-based approach is much less than the previous one.

However, its accuracy is only 33%, which is inferior to the original approach (65.8% accuracy). The reason for the inferior performance roots on the consequence of its training process and the limited dataset. Estimators that construct the histogram only access subsets of the completed dataset. When the dataset has a decent amount of instance and its subsets have similar data distribution, estimators learns from the distribution of the dataset. In contrast, subsets fail to represent the distribution of the dataset when the dataset only have a few instances. In this case, these subsets affect the estimators, causing ithe nferior performance.

[ ]:

```

```

2.4 XGBoost

XGBoost is an advanced implementation of gradient boosting algorithm. This section applies XGBClassifier from XGBoost to learn the prediction model.

2.4.1 Import Libraries

[32]:

```
#! pip install xgboost
from xgboost import XGBClassifier, plot_importance
```

2.4.2 import model

[33]:

```
model = XGBClassifier()
#model = XGBClassifier(learning_rate=0.02, n_estimators=600, objective='binary:
↪logistic', silent=True, nthread=1)
```

2.4.3 cross-validation

[34]:

```
start = time()
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=50, random_state=0)
score = cross_val_score(model, X, y, scoring="accuracy", cv=cv, n_jobs=-1)

speed["XGB"] = np.round(time() - start, 3)
accuracy["XGB"] = np.nanmean(score).round(3)

print(
    f"Mean Accuracy: {accuracy['XGB']}\nStd: {np.std(score):.3f}\nRun time:␣
↪{speed['XGB']}s"
)
```

```
Mean Accuracy: 0.605
Std: 0.244
Run time: 4.189s
```

[35]:
```
#score
```

2.4.4 Performance Comparison

The performances of XGBoost are similar to the GradientBoosting's.

[36]:
```
print("Accuracy:")
{k: v for k, v in sorted(accuracy.items(), key=lambda i: i[1], reverse=True)}
```

```
Accuracy:
```

[36]:
```
{'GradientBoosting': 0.633, 'XGB': 0.605, 'HistGradientBoosting': 0.333}
```

[37]:
```
print("Speed:")
{k: v for k, v in sorted(speed.items(), key=lambda i: i[1], reverse=False)}
```

```
Speed:
```

[37]:
```
{'HistGradientBoosting': 2.144, 'XGB': 4.189, 'GradientBoosting': 7.093}
```

[ ]:
```

```

2.4.5 Fit The Model (Before Parametere Optimization)

[38]:
```
model.fit(X, y, eval_metric='auc')
```

```
C:\ProgramData\Anaconda3\lib\site-packages\xgboost\sklearn.py:892: UserWarning:
The use of label encoder in XGBClassifier is deprecated and will be removed in a
future release. To remove this warning, do the following: 1) Pass option
use_label_encoder=False when constructing XGBClassifier object; and 2) Encode
your labels (y) as integers starting with 0, i.e. 0, 1, 2, ..., [num_class - 1].
  warnings.warn(label_encoder_deprecation_msg, UserWarning)
```

[38]:
```
XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
              colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
              importance_type='gain', interaction_constraints='',
              learning_rate=0.300000012, max_delta_step=0, max_depth=6,
              min_child_weight=1, missing=nan, monotone_constraints='()',
              n_estimators=100, n_jobs=4, num_parallel_tree=1, random_state=0,
              reg_alpha=0, reg_lambda=1, scale_pos_weight=1, subsample=1,
              tree_method='exact', validate_parameters=1, verbosity=None)
```

2.4.6 Plot Feature Importance (Before Parametere Optimization)

Feature importance is calculated by 3 approaches: weight, gain, and cover.

- gain is the average gain of splits which use the feature
- weight is the number of times a feature appears in a tree

- cover is the average coverage of splits which use the feature where coverage is defined as the number of samples affected by the split

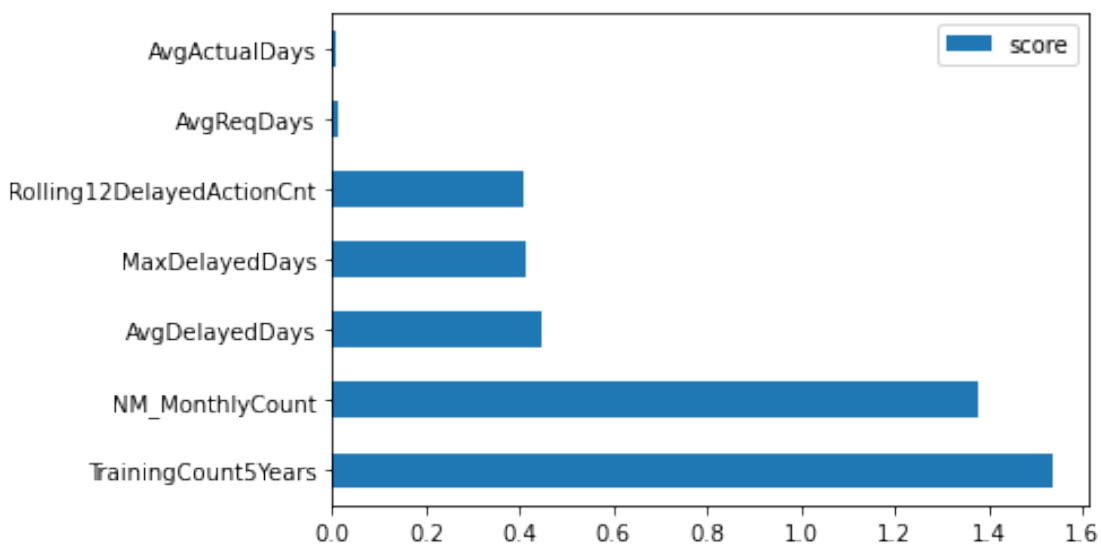
This report record the "gain" approach of feature importance for comparison.

[39]:
```
model.get_booster().get_score(importance_type="gain")
```

[39]:
```
{'TrainingCount5Years': 1.5378739814,
 'AvgDelayedDays': 0.4474608838819285,
 'NM_MonthlyCount': 1.3757188630000001,
 'MaxDelayedDays': 0.41414853640000004,
 'Rolling12DelayedActionCnt': 0.4069084077999999,
 'AvgReqDays': 0.011747607267424997,
 'AvgActualDays': 0.009767331086206898}
```

[40]:
```
feature_important = model.get_booster().get_score(importance_type='gain')
keys = list(feature_important.keys())
values = list(feature_important.values())

data = pd.DataFrame(data=values, index=keys, columns=["score"]).sort_values(by =␣
 ↪"score", ascending=False)
data.plot(kind='barh')
```

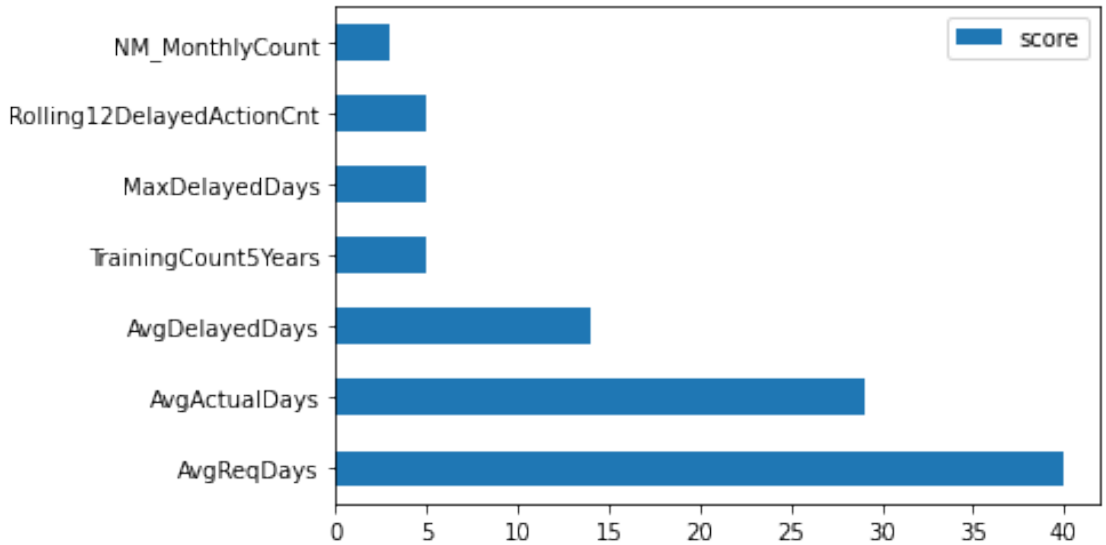
[40]:
```
<AxesSubplot:>
```



[41]:
```
feature_important = model.get_booster().get_score(importance_type='weight')
keys = list(feature_important.keys())
values = list(feature_important.values())
```

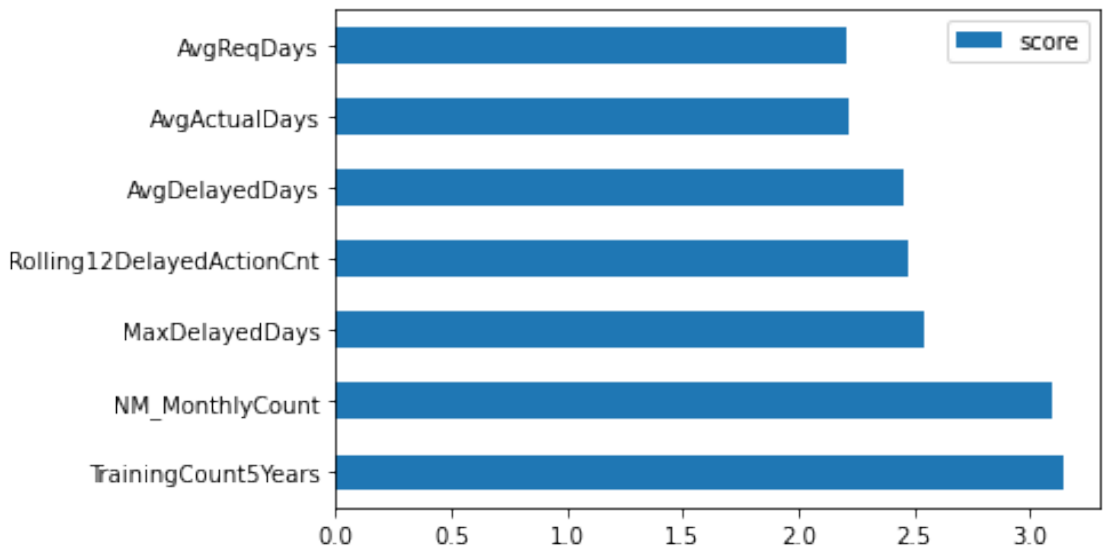
```
data = pd.DataFrame(data=values, index=keys, columns=["score"]).sort_values(by =␣
↪"score", ascending=False)
data.plot(kind='barh')
```

[41]: <AxesSubplot:>



[42]:
```
feature_important = model.get_booster().get_score(importance_type='cover')
keys = list(feature_important.keys())
values = list(feature_important.values())

data = pd.DataFrame(data=values, index=keys, columns=["score"]).sort_values(by =␣
↪"score", ascending=False)
data.plot(kind='barh')
```

[42]: <AxesSubplot:>



[ ]:

```

```

[43]:

```
feature_important_ = pd.DataFrame.from_dict(feature_important, orient='index',␣
↪columns=['feature_importance'])
feature_important_ = feature_important_.sort_values(by='feature_importance',␣
↪ascending=False)
rank_serial = range(1, len(feature_important_) +1)
#list(rank_serial)

feature_important_['XGBoost (Default)']= list(rank_serial) #rank
ranking_xbg_b = feature_important_.drop("feature_importance", axis=1)
ranking_xbg_b
```

[43]:

	XGBoost (Default)
TrainingCount5Years	1
NM_MonthlyCount	2
MaxDelayedDays	3
Rolling12DelayedActionCnt	4
AvgDelayedDays	5
AvgActualDays	6
AvgReqDays	7

[ ]:

```

```

[44]:

```
model.get_params()
```

[44]:

```
{'objective': 'binary:logistic',
 'use_label_encoder': True,
```

```
 'base_score': 0.5,
 'booster': 'gbtree',
 'colsample_bylevel': 1,
 'colsample_bynode': 1,
 'colsample_bytree': 1,
 'gamma': 0,
 'gpu_id': -1,
 'importance_type': 'gain',
 'interaction_constraints': '',
 'learning_rate': 0.300000012,
 'max_delta_step': 0,
 'max_depth': 6,
 'min_child_weight': 1,
 'missing': nan,
 'monotone_constraints': '()',
 'n_estimators': 100,
 'n_jobs': 4,
 'num_parallel_tree': 1,
 'random_state': 0,
 'reg_alpha': 0,
 'reg_lambda': 1,
 'scale_pos_weight': 1,
 'subsample': 1,
 'tree_method': 'exact',
 'validate_parameters': 1,
 'verbosity': None}
```

[ ]:

```
```

[ ]:

```
```

Hyperparameter Optimization The XGBoost model requires hyperparameter tuning to improve and fully leverage its advantages over other algorithms.

https://www.analyticsvidhya.com/blog/2016/03/complete-guide-parameter-tuning-xgboost-with-codes-python/

[45]:

```
# A parameter grid for XGBoost
params = {
        'min_child_weight': [1, 2, 3],
        'gamma': [0, 0.5, 1],
        'subsample': [0.6, 0.8, 1.0],
        'colsample_bytree': [0.6, 0.8, 1.0],
        'max_depth': [5, 6, 7]
        }
# fit model no training data
```

```
#xgb = XGBClassifier(learning_rate=0.02, n_estimators=600, objective='binary:
↪logistic', silent=True, nthread=1)
xgb = XGBClassifier()
folds = 3
param_comb = 5

cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=50, random_state=0)
#score = cross_val_score(model, X, y, scoring="accuracy", cv=cv, n_jobs=-1)
```

Randomized search is applied for tuning hyperparameters. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html

[46]:
```
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV
random_search = RandomizedSearchCV(xgb,␣
↪param_distributions=params,n_iter=param_comb, scoring='roc_auc', n_jobs=-1,␣
↪cv=cv, verbose=0, random_state=0 )
```

[47]:
```
random_search.fit(X, y, eval_metric='auc')
```

```
C:\ProgramData\Anaconda3\lib\site-packages\xgboost\sklearn.py:892: UserWarning:
The use of label encoder in XGBClassifier is deprecated and will be removed in a
future release. To remove this warning, do the following: 1) Pass option
use_label_encoder=False when constructing XGBClassifier object; and 2) Encode
your labels (y) as integers starting with 0, i.e. 0, 1, 2, ..., [num_class - 1].
  warnings.warn(label_encoder_deprecation_msg, UserWarning)
```

[47]:
```
RandomizedSearchCV(cv=RepeatedStratifiedKFold(n_repeats=50, n_splits=5,
random_state=0),
                   estimator=XGBClassifier(base_score=None, booster=None,
                                           colsample_bylevel=None,
                                           colsample_bynode=None,
                                           colsample_bytree=None, gamma=None,
                                           gpu_id=None, importance_type='gain',
                                           interaction_constraints=None,
                                           learning_rate=None,
                                           max_delta_step=None, max_depth=None,
                                           min_child_weigh...
                                           num_parallel_tree=None,
                                           random_state=None, reg_alpha=None,
                                           reg_lambda=None,
                                           scale_pos_weight=None,
                                           subsample=None, tree_method=None,
                                           validate_parameters=None,
                                           verbosity=None),
                   n_iter=5, n_jobs=-1,
                   param_distributions={'colsample_bytree': [0.6, 0.8, 1.0],
                                        'gamma': [0, 0.5, 1],
```

```
                                                 'max_depth': [5, 6, 7],
                                                 'min_child_weight': [1, 2, 3],
                                                 'subsample': [0.6, 0.8, 1.0]},
                        random_state=0, scoring='roc_auc')
```

[48]:
```
print('\n Best hyperparameters:')
print(random_search.best_params_)
print('\n Best estimator:')
print(random_search.best_estimator_)
```

```
 Best hyperparameters:
{'subsample': 0.8, 'min_child_weight': 1, 'max_depth': 6, 'gamma': 0.5,
'colsample_bytree': 0.6}

 Best estimator:
XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
              colsample_bynode=1, colsample_bytree=0.6, gamma=0.5, gpu_id=-1,
              importance_type='gain', interaction_constraints='',
              learning_rate=0.300000012, max_delta_step=0, max_depth=6,
              min_child_weight=1, missing=nan, monotone_constraints='()',
              n_estimators=100, n_jobs=4, num_parallel_tree=1, random_state=0,
              reg_alpha=0, reg_lambda=1, scale_pos_weight=1, subsample=0.8,
              tree_method='exact', validate_parameters=1, verbosity=None)
```

[49]:
```
#dir(random_search.best_estimator_)
```

[50]:
```
#check importance_type
random_search.best_estimator_.importance_type
```

[50]:
```
'gain'
```

[51]:
```
random_search.best_estimator_.feature_importances_
```

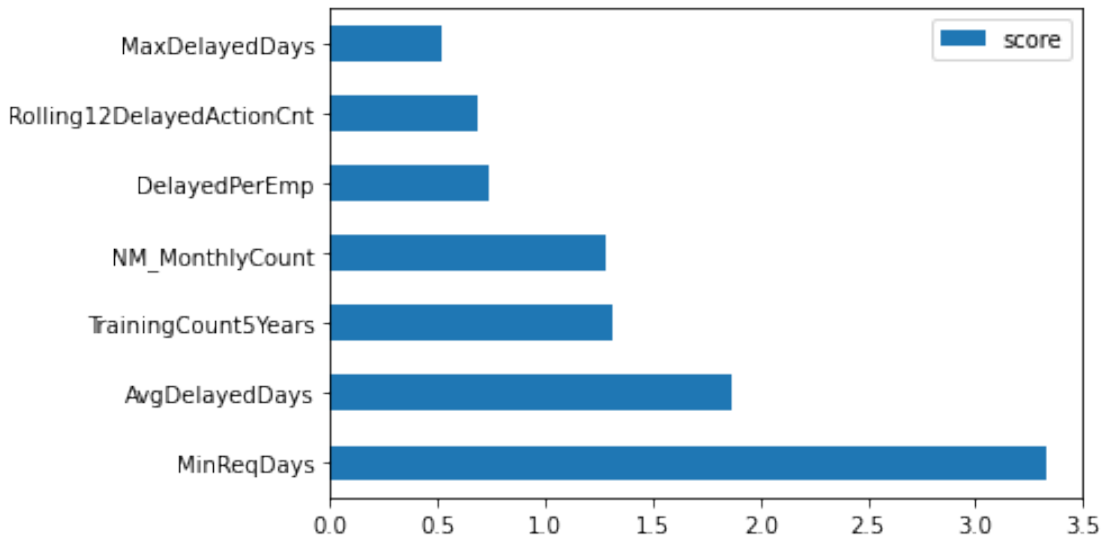
[51]:
```
array([0.07012209, 0.07542349, 0.19199921, 0.        , 0.05338017,
       0.        , 0.34302276, 0.        , 0.        , 0.        ,
       0.        , 0.13138352, 0.        , 0.13466877, 0.        ],
      dtype=float32)
```

[52]:
```
print("Feature Importance: \n ", random_search.best_estimator_.get_booster().
 ↪get_score(importance_type='gain'))
```

```
Feature Importance:
  {'MinReqDays': 3.33333349, 'Rolling12DelayedActionCnt': 0.6814134319999999,
'NM_MonthlyCount': 1.276723115, 'AvgDelayedDays': 1.8657578633333334,
'TrainingCount5Years': 1.3086475585, 'MaxDelayedDays': 0.518723309,
'DelayedPerEmp': 0.732929885}
```

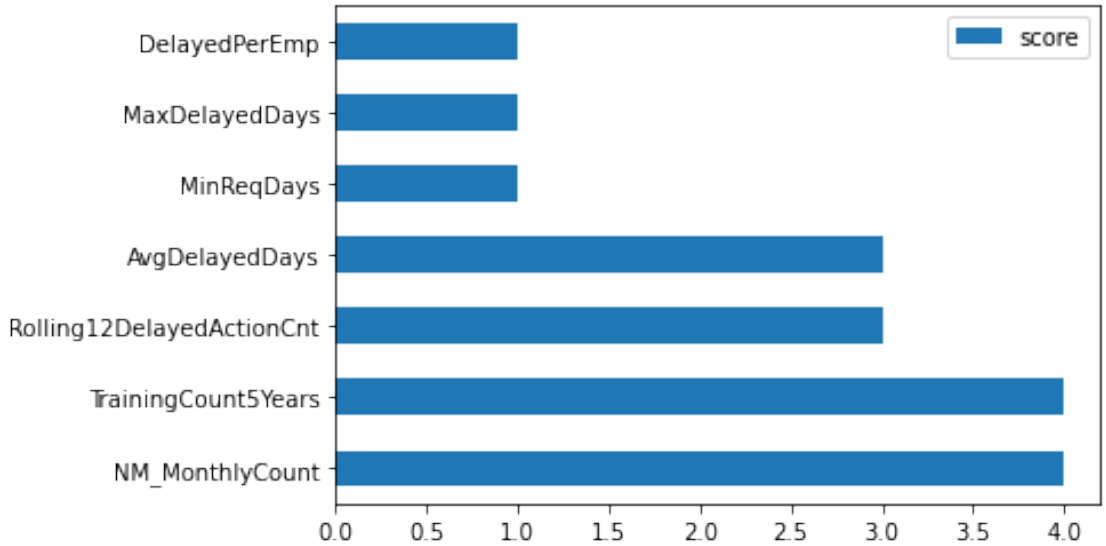
[53]:
```
feature_important_opt = random_search.best_estimator_.get_booster().
 ↪get_score(importance_type='gain')
keys = list(feature_important_opt.keys())
values = list(feature_important_opt.values())

data = pd.DataFrame(data=values, index=keys, columns=["score"]).sort_values(by =␣
 ↪"score", ascending=False)
data.plot(kind='barh')
```

[53]: <AxesSubplot:>



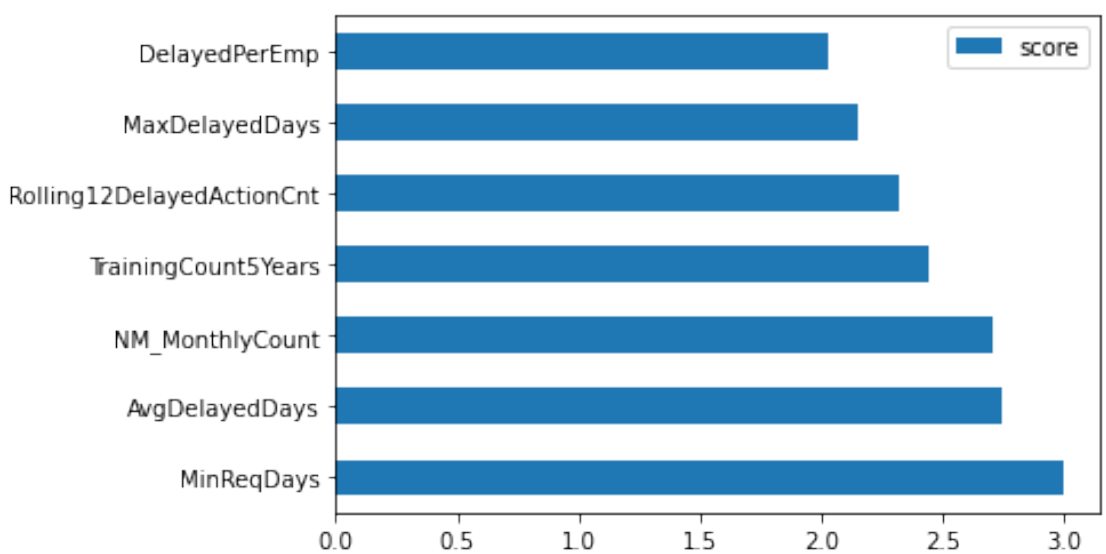
[54]:
```
feature_important_opt = random_search.best_estimator_.get_booster().
 ↪get_score(importance_type='weight')
keys = list(feature_important_opt.keys())
values = list(feature_important_opt.values())

data = pd.DataFrame(data=values, index=keys, columns=["score"]).sort_values(by =␣
 ↪"score", ascending=False)
data.plot(kind='barh')
```

[54]: <AxesSubplot:>



[55]:
```
feature_important_opt = random_search.best_estimator_.get_booster().
 ↪get_score(importance_type='cover')
keys = list(feature_important_opt.keys())
values = list(feature_important_opt.values())

data = pd.DataFrame(data=values, index=keys, columns=["score"]).sort_values(by =␣
 ↪"score", ascending=False)
data.plot(kind='barh')
```

[55]: <AxesSubplot:>



[56]:
```
feature_important_ = pd.DataFrame.from_dict(feature_important, orient='index',␣
↪columns=['feature_importance'])
feature_important_ = feature_important_.sort_values(by='feature_importance',␣
↪ascending=False)
rank_serial = range(1, len(feature_important_) +1)
#list(rank_serial)

feature_important_['XGBoost (Optimised)']= list(rank_serial) #rank
ranking_xbg_o = feature_important_.drop("feature_importance", axis=1)

change =pd.DataFrame( ranking_xbg_b["XGBoost (Default)"]-ranking_xbg_o['XGBoost␣
↪(Optimised)'], columns=["ranking change"])

pd.concat([ranking_xbg_o, ranking_xbg_b, change], axis=1)
```

[56]:
```
                           XGBoost (Optimised)  XGBoost (Default)  \
TrainingCount5Years                          1                  1
NM_MonthlyCount                              2                  2
MaxDelayedDays                               3                  3
Rolling12DelayedActionCnt                    4                  4
AvgDelayedDays                               5                  5
AvgActualDays                                6                  6
AvgReqDays                                   7                  7

                           ranking change
TrainingCount5Years                     0
NM_MonthlyCount                         0
MaxDelayedDays                          0
Rolling12DelayedActionCnt               0
AvgDelayedDays                          0
AvgActualDays                           0
AvgReqDays                              0
```

The ranking of feature importance keeps the same after the model was trained with optimised hyperparameters. In the optimised model, the feature of the top two ranking features become less dominate in constructing model as values of features importance become more evenly distributed than before.

[57]:
```
feature_important_ = pd.DataFrame.from_dict(feature_important, orient='index',␣
↪columns=['feature_importance'])
feature_important_ = feature_important_.sort_values(by='feature_importance',␣
↪ascending=False)
rank_serial = range(1, len(feature_important_) +1)
#list(rank_serial)

feature_important_['XGBoost (Optimised)']= list(rank_serial) #rank
```

```
ranking_ = feature_important_.drop("feature_importance", axis=1)

rank = pd.concat([rank, ranking_xbg_o], axis=1)
rank
```

[57]:

	GradientBoosting	XGBoost (Optimised)
TrainingCount5Years	1	1.0
NM_Rolling12NMCount	2	NaN
NM_MonthlyCount	3	2.0
MaxActualDays	4	NaN
MaxReqDays	5	NaN
Rolling12DelayedActionCnt	6	4.0
DelayedPerEmp	7	NaN
AvgReqDays	8	7.0
AvgActualDays	9	6.0
MaxDelayedDays	10	3.0
MinReqDays	11	NaN
AvgDelayedDays	12	5.0
AuditCount5Years	13	NaN
MinActualDays	14	NaN
MinDelayedDays	15	NaN

2.5 LightGBM

2.5.1 Import Libraries

[58]:
```
#!conda install -c conda-forge lightgbm
import lightgbm as lgb
from lightgbm import LGBMClassifier
```

2.5.2 import model

[59]:
```
model = LGBMClassifier()
```

2.5.3 cross-validation

[60]:
```
start = time()
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=50, random_state=0)
score = cross_val_score(model, X, y, scoring="accuracy", cv=cv, n_jobs=-1)

speed["LGBM"] = np.round(time() - start, 3)
accuracy["LGBM"] = np.nanmean(score).round(3)

print(
    f"Mean Accuracy: {accuracy['LGBM']}\nStd: {np.std(score):.3f}\nRun time:␣
 ↪{speed['LGBM']}s"
```

```
)
```

```
Mean Accuracy: 0.333
Std: 0.000
Run time: 1.622s
```

[61]:
```
print("Accuracy:")
{k: v for k, v in sorted(accuracy.items(), key=lambda i: i[1], reverse=True)}
```

```
Accuracy:
```

[61]:
```
{'GradientBoosting': 0.633,
 'XGB': 0.605,
 'HistGradientBoosting': 0.333,
 'LGBM': 0.333}
```

Notice that this accuracy is significantly lower than GradientBoosting and XGBoost. Therefore, it is not that surprising that the feature importance values equal to zeros, even after hyperparameter optimization (see below).

[62]:
```
#score
```

2.5.4 fit the model

[63]:
```
model.fit(X, y)
```

[63]:
```
LGBMClassifier()
```

2.5.5 Plot Feature Importance

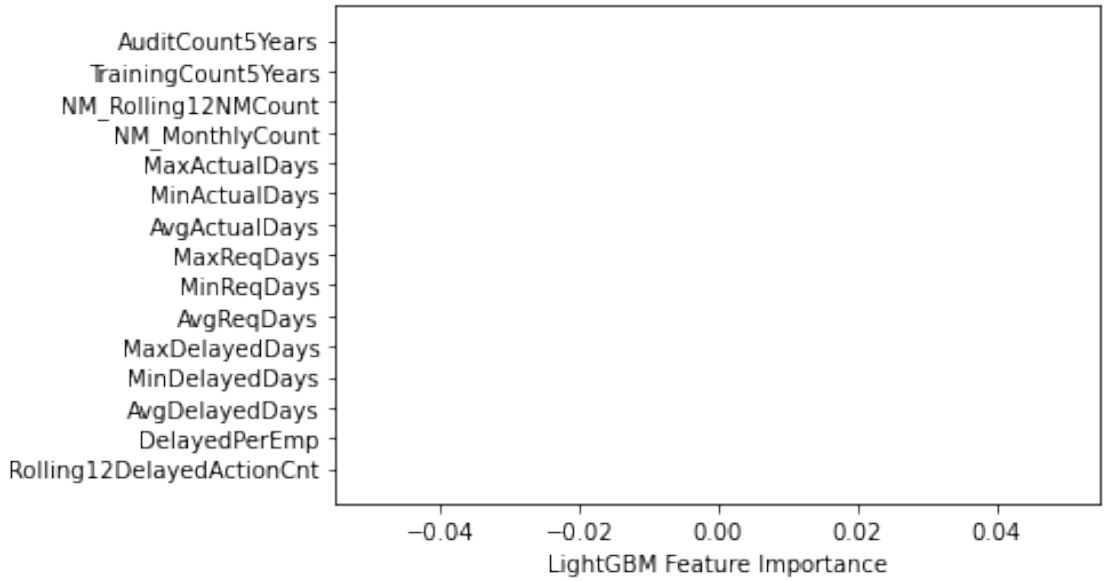
The default model fails to achieve feature importance when its accuracy is relatively low.

[64]:
```
print("Feature Importance: ",model.feature_importances_)
```

```
Feature Importance:  [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

[65]:
```
plt.barh(X.columns, model.feature_importances_)
plt.xlabel("LightGBM Feature Importance")
```

[65]:
```
Text(0.5, 0, 'LightGBM Feature Importance')
```



[66]:
```
#model.get_params()
```

2.5.6 Hyperparameter Optimaztion

[67]:
```
from scipy.stats import randint as sp_randint
from scipy.stats import uniform as sp_uniform
param_test ={'num_leaves': sp_randint(6, 50),
             'min_child_samples': sp_randint(100, 500),
             'min_child_weight': [1e-5, 1e-3, 1e-2, 1e-1, 1, 1e1, 1e2, 1e3, 1e4],
             'subsample': sp_uniform(loc=0.2, scale=0.8),
             'colsample_bytree': sp_uniform(loc=0.4, scale=0.6),
             'reg_alpha': [0, 1e-1, 1, 2, 5, 7, 10, 50, 100],
             'reg_lambda': [0, 1e-1, 1, 5, 10, 20, 50, 100]}

fit_params={"early_stopping_rounds":30,
            "eval_metric" : 'auc',
            "eval_set" : [(X_test,y_test)],
            'eval_names': ['valid'],
            #'callbacks': [lgb.
 ↪reset_parameter(learning_rate=learning_rate_010_decay_power_099)],
            'verbose': 100,
            'categorical_feature': 'auto'}
```

[68]:
```
#This parameter defines the number of HP points to be tested
n_HP_points_to_test = 100

import lightgbm as lgb
```

```
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV

#n_estimators is set to a "large value". The actual number of trees build will␣
↪depend on early stopping and 5000 define only the absolute maximum
clf = lgb.LGBMClassifier(max_depth=-1, random_state=314, silent=True,␣
↪metric='None', n_jobs=4, n_estimators=5000)
gs = RandomizedSearchCV(
    estimator=clf, param_distributions=param_test,
    n_iter=n_HP_points_to_test,
    scoring='roc_auc',
    cv=3,
    refit=True,
    random_state=0,
    verbose=False)
```

[69]:
```
gs.fit(X_train, y_train, **fit_params)
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

```
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[1]     valid's auc: 0.5
```

[69]:
```
RandomizedSearchCV(cv=3,
                   estimator=LGBMClassifier(metric='None', n_estimators=5000,
                                            n_jobs=4, random_state=314),
                   n_iter=100,
                   param_distributions={'colsample_bytree':
<scipy.stats._distn_infrastructure.rv_frozen object at 0x0000000007B77040>,
                                        'min_child_samples':
<scipy.stats._distn_infrastructure.rv_frozen object at 0x0000000007B7B8B0>,
                                        'min_child_weight': [1e-05, 0.001, 0.01,
                                                             0.1, 1, 10.0,
                                                             100.0, 1000.0,
                                                             10000.0],
                                        'num_leaves':
<scipy.stats._distn_infrastructure.rv_frozen object at 0x0000000007B7B9A0>,
                                        'reg_alpha': [0, 0.1, 1, 2, 5, 7, 10,
                                                      50, 100],
                                        'reg_lambda': [0, 0.1, 1, 5, 10, 20, 50,
                                                       100],
                                        'subsample':
<scipy.stats._distn_infrastructure.rv_frozen object at 0x0000000007B7BD30>},
                   random_state=0, scoring='roc_auc', verbose=False)
```

[70]:
```
print('Best score reached: {} with params: {} '.format(gs.best_score_, gs.
 ↪best_params_))
```

```
Best score reached: 0.5 with params: {'colsample_bytree': 0.7292881023563949,
'min_child_samples': 217, 'min_child_weight': 1e-05, 'num_leaves': 9,
'reg_alpha': 2, 'reg_lambda': 100, 'subsample': 0.5389238394711238}
```

2.5.7 Fit the final model

[71]:
```
clf_final = LGBMClassifier(**clf.get_params())
```

[72]:
```
#set optimal parameters
clf_final.set_params(**gs.best_params_)
```

[72]:
```
LGBMClassifier(colsample_bytree=0.7292881023563949, metric='None',
               min_child_samples=217, min_child_weight=1e-05, n_estimators=5000,
               n_jobs=4, num_leaves=9, random_state=314, reg_alpha=2,
               reg_lambda=100, subsample=0.5389238394711238)
```

[73]:
```
#fit on the whole dataset
clf_final.fit(X, y)
```

[73]:
```
LGBMClassifier(colsample_bytree=0.7292881023563949, metric='None',
               min_child_samples=217, min_child_weight=1e-05, n_estimators=5000,
               n_jobs=4, num_leaves=9, random_state=314, reg_alpha=2,
               reg_lambda=100, subsample=0.5389238394711238)
```

2.5.8 Feature Importance

The values of feature importance all equals to zeros even with optimal hyperparameters.

[74]:
```
clf_final.feature_importances_
```

[74]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

This suggests that the optimised model of thelightGBM approach also fails to achieve feature importance in this case.

2.6 Catboost

2.6.1 Import Libraries

[75]:
```
from catboost import CatBoostClassifier
```

2.6.2 import model

[76]:
```
model = CatBoostClassifier()
```

2.6.3 cross-validation

[77]:
```
start = time()
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=50, random_state=0)
score = cross_val_score(model, X, y, scoring="accuracy", cv=cv, n_jobs=4)

speed["CatBoost"] = np.round(time() - start, 3)
accuracy["CatBoost"] = np.nanmean(score).round(3)

print(
    f"Mean Accuracy: {accuracy['CatBoost']}\nStd: {np.std(score):.3f}\nRun time:␣
 ↪{speed['CatBoost']}s"
)
```

```
Mean Accuracy: 0.697
Std: 0.257
Run time: 130.234s
```

[78]:
```
#score
```

2.6.4 fit the model

[79]:
```
model.fit(X, y,verbose_eval=False)
```

[79]:
```
<catboost.core.CatBoostClassifier at 0x7d349d0>
```

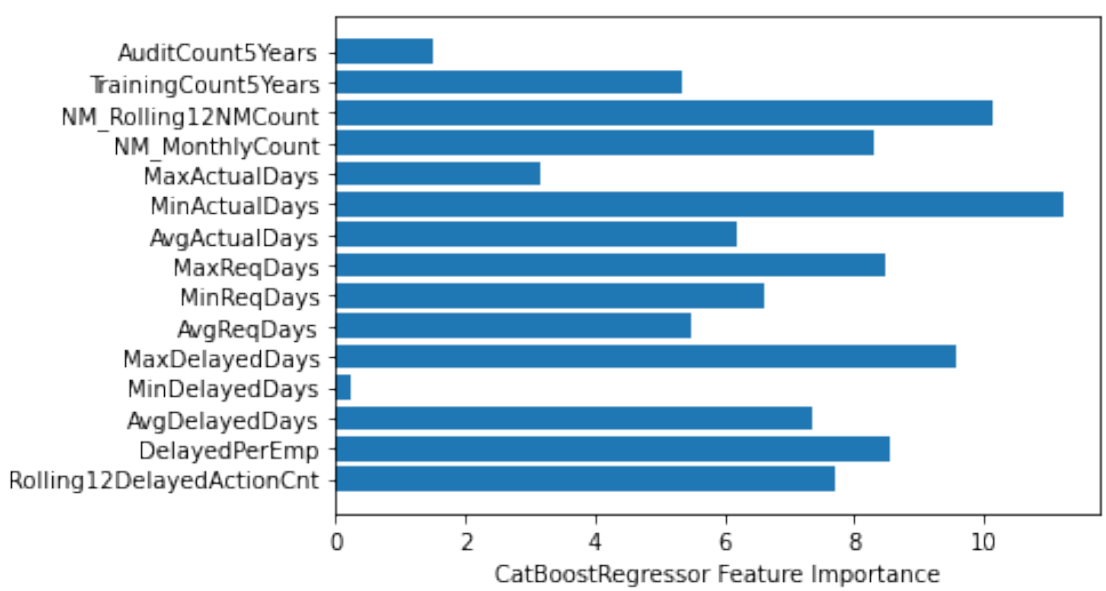
2.6.5 Plot Feature Importance

[80]:
```
print("Feature Importance: ",model.feature_importances_)
```

```
Feature Importance:  [ 7.71993885  8.56782952  7.37295708  0.23839668
9.58185151  5.48829867
  6.61653023  8.49250067  6.20588303 11.24807859  3.17545354  8.29574046
 10.14871373  5.3479403   1.49988714]
```

[81]:
```
plt.barh(model.feature_names_, model.feature_importances_)
plt.xlabel("CatBoostRegressor Feature Importance")
```

[81]:
```
Text(0.5, 0, 'CatBoostRegressor Feature Importance')
```



[ ]:
```

```

2.6.6 Hyperparameter Optimaztion

[82]:
```
parameters = {
    'depth': [3,4,5],
    'learning_rate' : [0.005,0.01,0.02]
    }
```

[83]:
```
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV
CBC = CatBoostClassifier()
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=10, random_state=0)
Grid_CBC = GridSearchCV(estimator=CBC, param_grid = parameters,cv = cv,␣
 ↪n_jobs=-1)
```

[84]:
```
Grid_CBC.fit(X_train, y_train,verbose_eval=False)
```

[84]:
```
GridSearchCV(cv=RepeatedStratifiedKFold(n_repeats=10, n_splits=5,
random_state=0),
             estimator=<catboost.core.CatBoostClassifier object at
0x0000000007E379D0>,
             n_jobs=-1,
             param_grid={'depth': [3, 4, 5],
                         'learning_rate': [0.005, 0.01, 0.02]})
```

[85]:
```
print('Best score reached: {} with params: {} '.format(Grid_CBC.best_score_,␣
 ↪Grid_CBC.best_params_))
```

```
Best score reached: 0.6566666666666667 with params: {'depth': 3,
'learning_rate': 0.01}
```

[86]:
```
Grid_CBC.best_estimator_.get_feature_importance()
```

[86]:
```
array([ 5.7386206 ,  9.41884586,  5.7650068 ,  0.0741454 ,  4.62164295,
        2.42841831,  5.06521003,  5.33670712,  5.54614495,  8.38002718,
        1.93224757, 36.32588783,  5.79597812,  2.90547419,  0.66564308])
```

[ ]:
```

```

2.6.7 Fit the final model

[87]:
```
Grid_CBC.best_params_
```

[87]:
```
{'depth': 3, 'learning_rate': 0.01}
```

[88]:
```
CBC_final = CatBoostClassifier(**CBC.get_params())
```

[89]:
```
#set optimal parameters
CBC_final.set_params(**Grid_CBC.best_params_)
```

[89]: <catboost.core.CatBoostClassifier at 0x7e27ac0>

[90]:
```
#fit on the whole dataset
CBC_final.fit(X, y,verbose_eval=False)
```

[90]: <catboost.core.CatBoostClassifier at 0x7e27ac0>

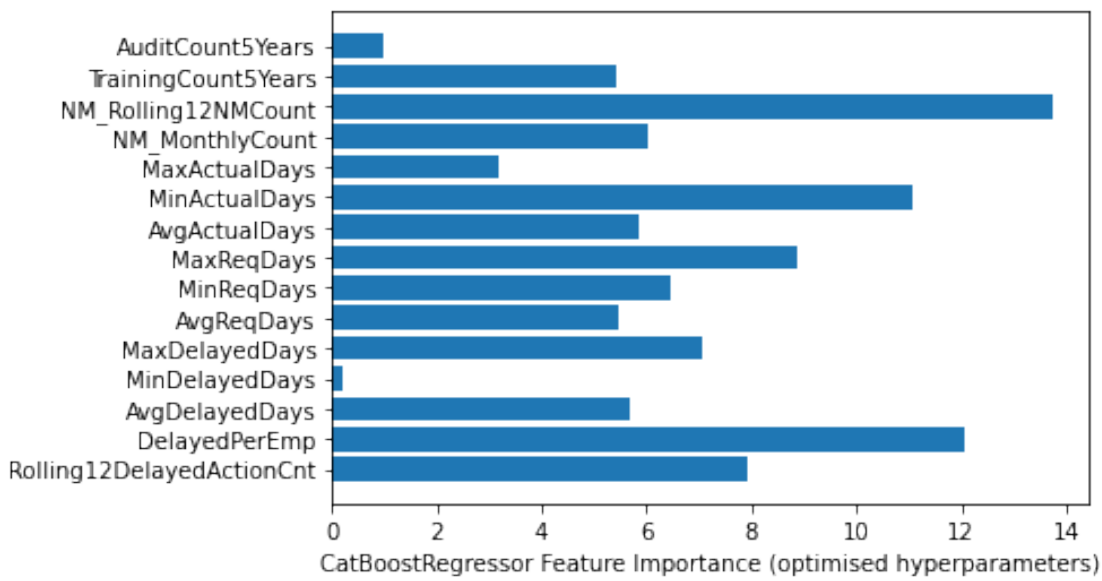
2.6.8 Plot Feature Importance

[91]:
```
print("Feature Importance: ",CBC_final.feature_importances_)
```

```
Feature Importance:  [ 7.93668536 12.05947977  5.67635647  0.19222108
7.05559532  5.47941133
  6.44164569  8.85567109  5.86076572 11.06205033  3.19338145  6.04011752
 13.76090728  5.41050929  0.97520231]
```

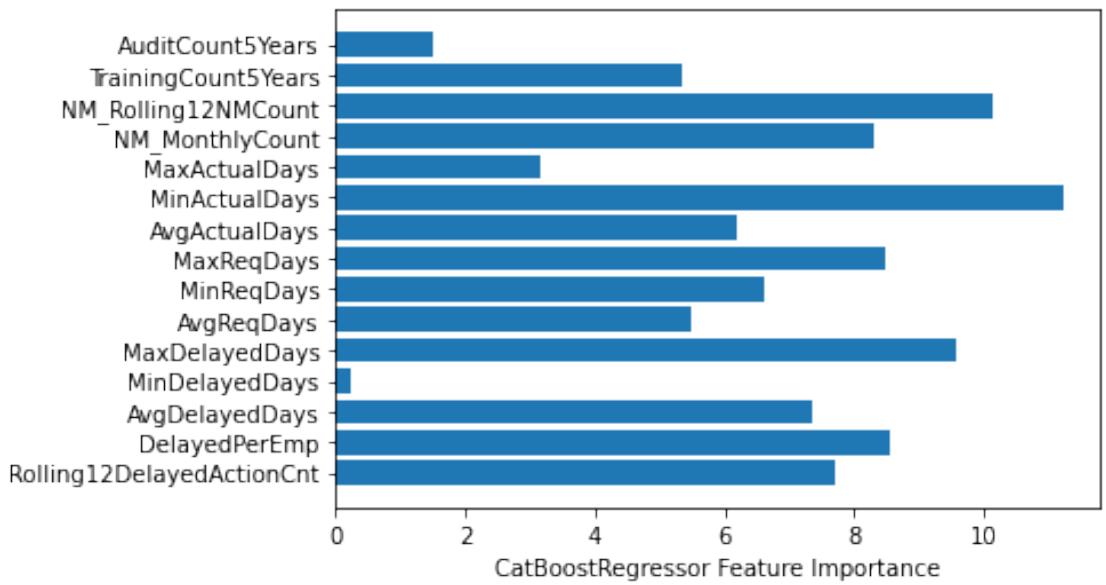
[92]:
```
plt.barh(CBC_final.feature_names_, CBC_final.feature_importances_)
plt.xlabel("CatBoostRegressor Feature Importance (optimised hyperparameters)")
```

[92]: Text(0.5, 0, 'CatBoostRegressor Feature Importance (optimised hyperparameters)')



[93]:
```
plt.barh(model.feature_names_, model.feature_importances_)
plt.xlabel("CatBoostRegressor Feature Importance")
```

[93]: Text(0.5, 0, 'CatBoostRegressor Feature Importance')



[94]:
```
sorted_idx = model.feature_importances_.argsort()
rank_1 = pd.DataFrame( (np.arange(1,len(model.
 ↪feature_importances_[sorted_idx])+1,1)),index=X.columns[sorted_idx][::-1],␣
 ↪columns=["CatBoost (Default)"])
sorted_idx = CBC_final.feature_importances_.argsort()
rank_2 = pd.DataFrame( (np.arange(1,len(CBC_final.
 ↪feature_importances_[sorted_idx])+1,1)),index=X.columns[sorted_idx][::-1],␣
 ↪columns=["CatBoost (Optimised)"])
change =pd.DataFrame( rank_1["CatBoost (Default)"]-rank_2["CatBoost␣
 ↪(Optimised)"], columns=["ranking change"])
ranking_ = pd.concat([rank_2, rank_1, change], axis=1)
ranking_
```

[94]:

	CatBoost (Optimised)	CatBoost (Default)	\
NM_Rolling12NMCount	1	2	
DelayedPerEmp	2	4	
MinActualDays	3	1	
MaxReqDays	4	5	
Rolling12DelayedActionCnt	5	7	
MaxDelayedDays	6	3	
MinReqDays	7	9	
NM_MonthlyCount	8	6	
AvgActualDays	9	10	
AvgDelayedDays	10	8	
AvgReqDays	11	11	
TrainingCount5Years	12	12	
MaxActualDays	13	13	

```
AuditCount5Years                              14                   14
MinDelayedDays                                15                   15

                            ranking change
NM_Rolling12NMCount                      1
DelayedPerEmp                            2
MinActualDays                           -2
MaxReqDays                               1
Rolling12DelayedActionCnt                2
MaxDelayedDays                          -3
MinReqDays                               2
NM_MonthlyCount                         -2
AvgActualDays                            1
AvgDelayedDays                          -2
AvgReqDays                               0
TrainingCount5Years                      0
MaxActualDays                            0
AuditCount5Years                         0
MinDelayedDays                           0
```

The optimised hyperparameters can affect the construction of the prediction model. The ranking of the feature importance slightly changed as optimised hyperparameters were introduced to the model. Risen from the 2nd, `NM_Rolling12NMCount` became the most important features. `Rolling12DelayedActionCnt`also became a more important feature than before. This case suggests that tuning hyperparameter may be necessary when Catboost approach deals with a complex dataset in future.

[95]:
```
rank = pd.concat([rank, rank_2], axis=1)
#rank
```

[96]:
```
#save the ranking file
rank.index.name = "Features"
rank.to_csv('result/Feature_importance_ranking_classifiers.csv')
```

2.7 Section Summary

This section investigates five implementations for a safety classification problem. Three approaches (e.g. GradientBoosting, XGB, and CatBoost) reach relatively high accuracies with feature importance. Their accuracies have a roughly inverted relationship with their time consumptions, suggesting a balance that worthies considering when implement.

In addition, the histogram-based approaches (e.g. Histogram-based Gradient Boosting and LightGBM) have inferior performance in this case. Because the dataset is relatively small in this study, these two approaches still worth a try when they face a complex dataset in the future.

[97]:
```
rank
```

[97]:

Features	GradientBoosting	XGBoost (Optimised)	CatBoost (Optimised)
TrainingCount5Years	1	1.0	12
NM_Rolling12NMCount	2	NaN	1
NM_MonthlyCount	3	2.0	8
MaxActualDays	4	NaN	13
MaxReqDays	5	NaN	4
Rolling12DelayedActionCnt	6	4.0	5
DelayedPerEmp	7	NaN	2
AvgReqDays	8	7.0	11
AvgActualDays	9	6.0	9
MaxDelayedDays	10	3.0	6
MinReqDays	11	NaN	7
AvgDelayedDays	12	5.0	10
AuditCount5Years	13	NaN	14
MinActualDays	14	NaN	3
MinDelayedDays	15	NaN	15

[98]:

```
print("Accuracy:")
{k: v for k, v in sorted(accuracy.items(), key=lambda i: i[1], reverse=True)}
```

```
Accuracy:
```

[98]:

```
{'CatBoost': 0.697,
 'GradientBoosting': 0.633,
 'XGB': 0.605,
 'HistGradientBoosting': 0.333,
 'LGBM': 0.333}
```

[99]:
```
print("Speed:")
{k: v for k, v in sorted(speed.items(), key=lambda i: i[1], reverse=False)}
```

```
Speed:
```

[99]:
```
{'LGBM': 1.622,
 'HistGradientBoosting': 2.144,
 'XGB': 4.189,
 'GradientBoosting': 7.093,
 'CatBoost': 130.234}
```

[ ]:
```

```

3 Section 2: Regression

The second section applies five implementations of the GBoost and the XGBoost algorithms for regression problem.

3.1 Data Preprocessing

3.1.1 Import Libraries

[100]:
```
import numpy as np
import pandas as pd
from time import time
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from scipy import stats
```

3.1.2 Import Dataset

[101]:
```
#import dataset
data = pd.read_csv('dataset/impact.csv')
# split the traing and testing datasets
X = data.drop('TRIFR',1)
y = data['TRIFR']
```

3.1.3 Skew

transform features with skew > 1

[102]:
```
import copy
X_skewed = copy.deepcopy(X)
for col in range(X_skewed.shape[1]):
```

```
    if stats.mstats.skew(X_skewed.iloc[:, col]).data >1 or stats.mstats.
↪skew(X_skewed.iloc[:, col]).data < -1 :
        if len(X_skewed[X_skewed.iloc[:, col]>0]) < X_skewed.shape[0]:
            # to shew, each value in the column should be positive number
            pass
        else:
            X_skewed.iloc[:, col] = np.log(X_skewed.iloc[:, col])
    else:
        pass
```

[103]:
```
stats.mstats.skew(X).data
```

[103]:
```
array([-0.35461034, -0.32561725,  2.19824437,  3.47439614,  0.05099458,
        0.03762476,  0.16789008, -0.92026225, -0.57649107, -0.407614  ,
       -0.60945159,  0.77786451, -0.64815153,  0.46823692,  1.79700391])
```

[104]:
```
stats.mstats.skew(X_skewed).data
```

[104]:
```
array([-0.35461034, -0.32561725,  1.57089682,  3.47439614,  0.05099458,
        0.03762476,  0.16789008, -0.92026225, -0.57649107, -0.407614  ,
       -0.60945159,  0.77786451, -0.64815153,  0.46823692,  1.79700391])
```

[105]:
```
X
```

[105]:
```
    Rolling12DelayedActionCnt  DelayedPerEmp  AvgDelayedDays  MinDelayedDays  \
0                         132           0.61              65               1
1                          92           0.42              78               1
2                          90           0.41              81               1
3                          89           0.41              82               1
4                         132           0.61              65               1
5                         131           0.60              65               1
6                         134           0.62              64               1
7                         134           0.62              62               1
8                         117           0.54              59               1
9                         118           0.54              91               1
10                         92           0.42              79               1
11                         81           0.38             139               2
12                         63           0.30              64               1
13                         51           0.24              64               1
14                         50           0.23              55               1

    MaxDelayedDays  AvgReqDays  MinReqDays  MaxReqDays  AvgActualDays  \
0              699         146        -699         619            212
1              528          78        -528         577            156
2              528          66        -528         577            148
3              528          58        -528         371            141
4              699         146        -699         619            211
```

```
5              699         153        -699         619             218
6              699         157        -699         619             222
7              699         161        -699         619             223
8              457         170        -204         577             230
9              528         106        -528         577             198
10             528          85        -528         577             164
11             952          67        -952         688             206
12             272          52        -216         345             116
13             272          25        -216         304              90
14             272          31        -216         304              87

    MinActualDays  MaxActualDays  NM_MonthlyCount  NM_Rolling12NMCount  \
0             -52            708                2                   12
1               0            581                1                   31
2               0            581                3                   33
3               0            428                4                   35
4             -52            708                1                   20
5             -52            708                2                   22
6             -52            708                2                   24
7             -52            708                4                   27
8             -52            708                2                   27
9               0            581                6                   31
10              0            581                5                   35
11              0            713                2                   29
12             -1            487                6                   38
13             -1            385                2                   40
14             -1            385                2                   38

    TrainingCount5Years  AuditCount5Years
0                     0                 0
1                     5                 9
2                     3                 0
3                     0                 0
4                     0                 0
5                     3                11
6                     1                 0
7                     3                 0
8                     2                 4
9                     3                12
10                    2                29
11                    0                 0
12                    2                11
13                    0                 0
14                    0                 0
```

[106]:
```
X_skewed
```

[106]:

	Rolling12DelayedActionCnt	DelayedPerEmp	AvgDelayedDays	MinDelayedDays	\
0	132	0.61	4.174387	0.000000	
1	92	0.42	4.356709	0.000000	
2	90	0.41	4.394449	0.000000	
3	89	0.41	4.406719	0.000000	
4	132	0.61	4.174387	0.000000	
5	131	0.60	4.174387	0.000000	
6	134	0.62	4.158883	0.000000	
7	134	0.62	4.127134	0.000000	
8	117	0.54	4.077537	0.000000	
9	118	0.54	4.510860	0.000000	
10	92	0.42	4.369448	0.000000	
11	81	0.38	4.934474	0.693147	
12	63	0.30	4.158883	0.000000	
13	51	0.24	4.158883	0.000000	
14	50	0.23	4.007333	0.000000	

	MaxDelayedDays	AvgReqDays	MinReqDays	MaxReqDays	AvgActualDays	\
0	699	146	-699	619	212	
1	528	78	-528	577	156	
2	528	66	-528	577	148	
3	528	58	-528	371	141	
4	699	146	-699	619	211	
5	699	153	-699	619	218	
6	699	157	-699	619	222	
7	699	161	-699	619	223	
8	457	170	-204	577	230	
9	528	106	-528	577	198	
10	528	85	-528	577	164	
11	952	67	-952	688	206	
12	272	52	-216	345	116	
13	272	25	-216	304	90	
14	272	31	-216	304	87	

	MinActualDays	MaxActualDays	NM_MonthlyCount	NM_Rolling12NMCount	\
0	-52	708	2	12	
1	0	581	1	31	
2	0	581	3	33	
3	0	428	4	35	
4	-52	708	1	20	
5	-52	708	2	22	
6	-52	708	2	24	
7	-52	708	4	27	
8	-52	708	2	27	
9	0	581	6	31	
10	0	581	5	35	
11	0	713	2	29	

```
12              -1             487                 6                     38
13              -1             385                 2                     40
14              -1             385                 2                     38

    TrainingCount5Years  AuditCount5Years
0                     0                 0
1                     5                 9
2                     3                 0
3                     0                 0
4                     0                 0
5                     3                11
6                     1                 0
7                     3                 0
8                     2                 4
9                     3                12
10                    2                29
11                    0                 0
12                    2                11
13                    0                 0
14                    0                 0
```

[107]:
```
X = X_skewed
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=0)
```

[108]:
```
#keep record for comparison
accuracy = {}
speed = {}
```

3.2 GBoost: Scikit-Learn Gradient Boosting Approach

3.2.1 Import Libraries

[109]:
```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RepeatedKFold
```

3.2.2 import model

[110]:
```
model = GradientBoostingRegressor()
```

3.2.3 cross-validation

[111]:
```
start = time()
cv = RepeatedKFold(n_splits=5, n_repeats=50, random_state=0)
score = cross_val_score(model, X, y, scoring="neg_mean_absolute_error", cv=cv,␣
 ↪n_jobs=-1)

speed["GradientBoosting"] = np.round(time() - start, 3)
accuracy["GradientBoosting"] = np.nanmean(score).round(3)

print(
    f"Mean Accuracy: {accuracy['GradientBoosting']}\nStd: {np.std(score):.
 ↪3f}\nRun time: {speed['GradientBoosting']}s"
)
```

```
Mean Accuracy: -1.459
Std: 1.753
Run time: 3.78s
```

[112]:
```
#score
```

3.2.4 fit the model

[113]:
```
# fit the model on the whole dataset
model.fit(X, y)
```

[113]:
```
GradientBoostingRegressor()
```

3.2.5 Plot Feature Importance

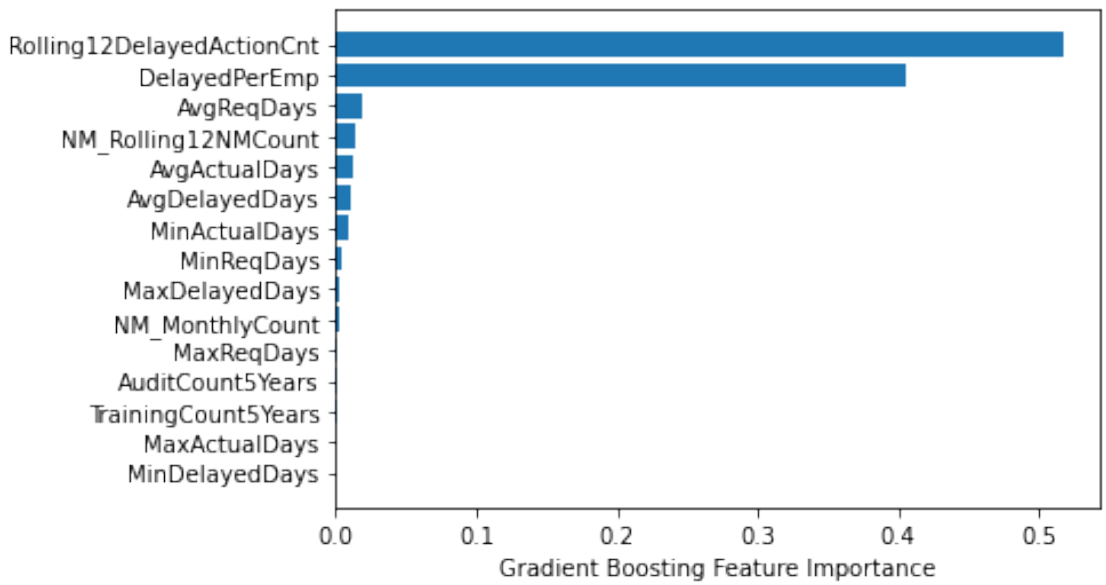
[114]:
```
print("The impurity-based feature importances :\n", model.feature_importances_)
```

```
The impurity-based feature importances :
 [5.17949996e-01 4.06008469e-01 1.13311716e-02 2.12186580e-06
 2.83203825e-03 1.92741795e-02 3.69388199e-03 6.60144266e-04
 1.20691332e-02 9.07013229e-03 4.30236166e-05 2.48508477e-03
 1.36589673e-02 2.71041455e-04 6.50614763e-04]
```

[115]:
```
#sorted_idx = model.feature_importances_.argsort()
#sorted_idx
```

[116]:
```
sorted_idx = model.feature_importances_.argsort()
plt.barh(X.columns[sorted_idx], model.feature_importances_[sorted_idx])
plt.xlabel("Gradient Boosting Feature Importance")
```

[116]:
```
Text(0.5, 0, 'Gradient Boosting Feature Importance')
```



[117]:
```
rank_c = pd.DataFrame( (np.arange(1,len(model.
↪feature_importances_[sorted_idx])+1,1)),index=X.columns[sorted_idx][::-1],␣
↪columns=["GradientBoosting"])
print("Feature Importance Ranking:")
rank_c
```

```
Feature Importance Ranking:
```

[117]:

	GradientBoosting
Rolling12DelayedActionCnt	1
DelayedPerEmp	2
AvgReqDays	3
NM_Rolling12NMCount	4
AvgActualDays	5
AvgDelayedDays	6
MinActualDays	7
MinReqDays	8
MaxDelayedDays	9
NM_MonthlyCount	10
MaxReqDays	11
AuditCount5Years	12
TrainingCount5Years	13
MaxActualDays	14
MinDelayedDays	15

[ ]:
```
```

3.3 Scikit-Learn Histogram-based Gradient Boosting Approach

3.3.1 Import Libraries

[118]:
```
from sklearn.experimental import enable_hist_gradient_boosting
from sklearn.ensemble import HistGradientBoostingRegressor
```

3.3.2 import model

[119]:
```
model = HistGradientBoostingRegressor()
```

3.3.3 cross-validation

[120]:
```
start = time()
cv = RepeatedKFold(n_splits=5, n_repeats=50, random_state=0)
score = cross_val_score(model, X, y, scoring="neg_mean_absolute_error", cv=cv,␣
 ↪n_jobs=-1)

speed["HistGradientBoosting"] = np.round(time() - start, 3)
accuracy["HistGradientBoosting"] = np.nanmean(score).round(3)

print(
    f"Mean Accuracy: {accuracy['HistGradientBoosting']}\nStd: {np.std(score):.
 ↪3f}\nRun time: {speed['HistGradientBoosting']}s"
)
```

```
Mean Accuracy: -5.122
Std: 1.674
Run time: 5.728s
```

[121]:
```
print("Accuracy:")
{k: v for k, v in sorted(accuracy.items(), key=lambda i: i[1], reverse=True)}
```

```
Accuracy:
```

[121]: {'GradientBoosting': -1.459, 'HistGradientBoosting': -5.122}

[122]:
```
print("Speed:")
{k: v for k, v in sorted(speed.items(), key=lambda i: i[1], reverse=False)}
```

```
Speed:
```

[122]: {'GradientBoosting': 3.78, 'HistGradientBoosting': 5.728}

[ ]:
```

```

[ ]:
```

```

3.3.4 fit the model

[123]:
```
# fit the model on the whole dataset
model.fit(X, y)
```

[123]:
```
HistGradientBoostingRegressor()
```

3.3.5 Feature Importance

As an experimental approach, feature importance is not provided by this approach (see API and attribute of the model below)).

[124]:
```
dir(model)
```

[124]:
```
['_VALID_LOSSES',
 '__abstractmethods__',
 '__class__',
 '__delattr__',
 '__dict__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__le__',
 '__lt__',
 '__module__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__setattr__',
 '__setstate__',
 '__sizeof__',
 '__slots__',
 '__str__',
 '__subclasshook__',
 '__weakref__',
 '_abc_impl',
 '_baseline_prediction',
 '_bin_data',
```

```
'_check_early_stopping_loss',
'_check_early_stopping_scorer',
'_check_n_features',
'_clear_state',
'_compute_partial_dependence_recursion',
'_encode_y',
'_estimator_type',
'_get_loss',
'_get_param_names',
'_get_small_trainset',
'_get_tags',
'_is_fitted',
'_more_tags',
'_predictors',
'_print_iteration_stats',
'_random_seed',
'_raw_predict',
'_repr_html_',
'_repr_html_inner',
'_repr_mimebundle_',
'_should_stop',
'_use_validation_data',
'_validate_data',
'_validate_parameters',
'bin_mapper_',
'do_early_stopping_',
'early_stopping',
'fit',
'get_params',
'l2_regularization',
'learning_rate',
'loss',
'loss_',
'max_bins',
'max_depth',
'max_iter',
'max_leaf_nodes',
'min_samples_leaf',
'monotonic_cst',
'n_features_',
'n_features_in_',
'n_iter_',
'n_iter_no_change',
'n_trees_per_iteration_',
'predict',
'random_state',
'score',
```

```
 'scorer_',
 'scoring',
 'set_params',
 'tol',
 'train_score_',
 'validation_fraction',
 'validation_score_',
 'verbose',
 'warm_start']
```

[ ]:

```

```

3.4 XGBoost

3.4.1 Import Libraries

[125]:

```
#! pip install xgboost
from xgboost import XGBRegressor, plot_importance
```

3.4.2 import model

[126]:

```
model = XGBRegressor()
```

3.4.3 cross-validation

[127]:

```
start = time()
cv = RepeatedKFold(n_splits=5, n_repeats=50, random_state=0)
score = cross_val_score(model, X, y, scoring="neg_mean_absolute_error", cv=cv,␣
 ↪n_jobs=-1)

speed["XGB"] = np.round(time() - start, 3)
accuracy["XGB"] = np.nanmean(score).round(3)

print(
    f"Mean Accuracy: {accuracy['XGB']}\nStd: {np.std(score):.3f}\nRun time:␣
 ↪{speed['XGB']}s"
)
```

```
Mean Accuracy: -1.581
Std: 1.746
Run time: 6.869s
```

[128]:

```
#score
```

[ ]:

```

```

3.4.4 fit the model

[129]:
```
# fit the model on the whole dataset
#model = XGBRegressor()
model.fit(X, y)
```

[129]:
```
XGBRegressor(base_score=0.5, booster='gbtree', colsample_bylevel=1,
             colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
             importance_type='gain', interaction_constraints='',
             learning_rate=0.300000012, max_delta_step=0, max_depth=6,
             min_child_weight=1, missing=nan, monotone_constraints='()',
             n_estimators=100, n_jobs=4, num_parallel_tree=1, random_state=0,
             reg_alpha=0, reg_lambda=1, scale_pos_weight=1, subsample=1,
             tree_method='exact', validate_parameters=1, verbosity=None)
```

3.4.5 Plot Feature Importance

As mensioned in the first section, the importance of this approach is calculated thought "weight", "gain" or "cover" method.

"weight" is the number of times a feature appears in a tree.

"gain" is the average gain of splits which use the feature.

"cover" is the average coverage of splits which use the feature where coverage is defined as the number of samples affected by the split.

[130]:
```
print("Feature Importance: ",model.feature_importances_)
```

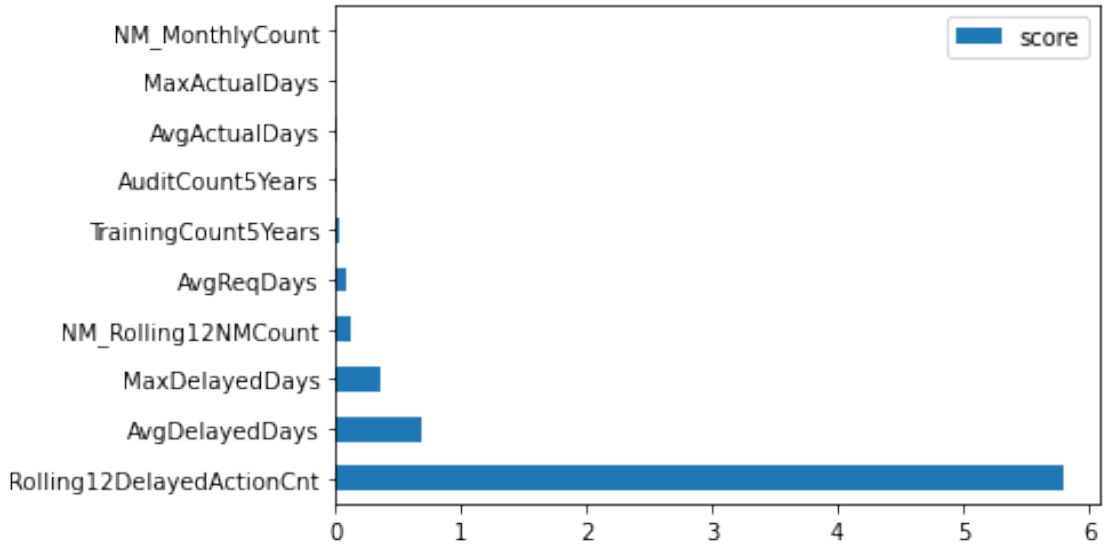
```
Feature Importance:  [8.1917024e-01 0.0000000e+00 9.7510934e-02 0.0000000e+00
5.0932989e-02
 1.2130307e-02 0.0000000e+00 0.0000000e+00 1.9593060e-04 0.0000000e+00
 3.3575142e-07 3.2842721e-07 1.6343715e-02 2.8297000e-03 8.8552712e-04]
```

[131]:
```
model.importance_type
```

[131]:
```
'gain'
```

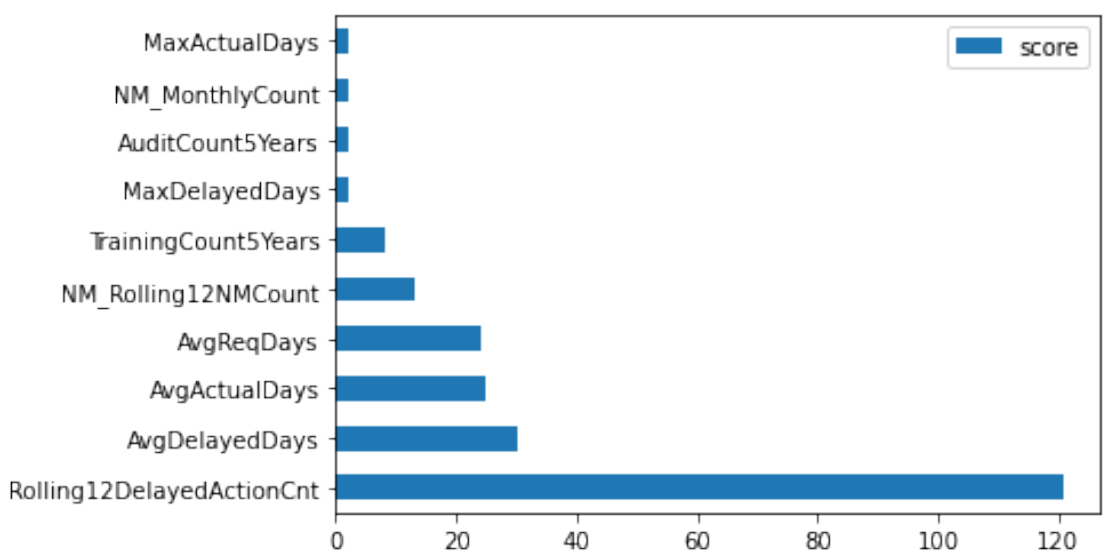
[132]:
```
feature_important = model.get_booster().get_score(importance_type='gain')
keys = list(feature_important.keys())
values = list(feature_important.values())

data = pd.DataFrame(data=values, index=keys, columns=["score"]).sort_values(by =␣
 ↪"score", ascending=False)
data.plot(kind='barh')
```

[132]:
```
<AxesSubplot:>
```



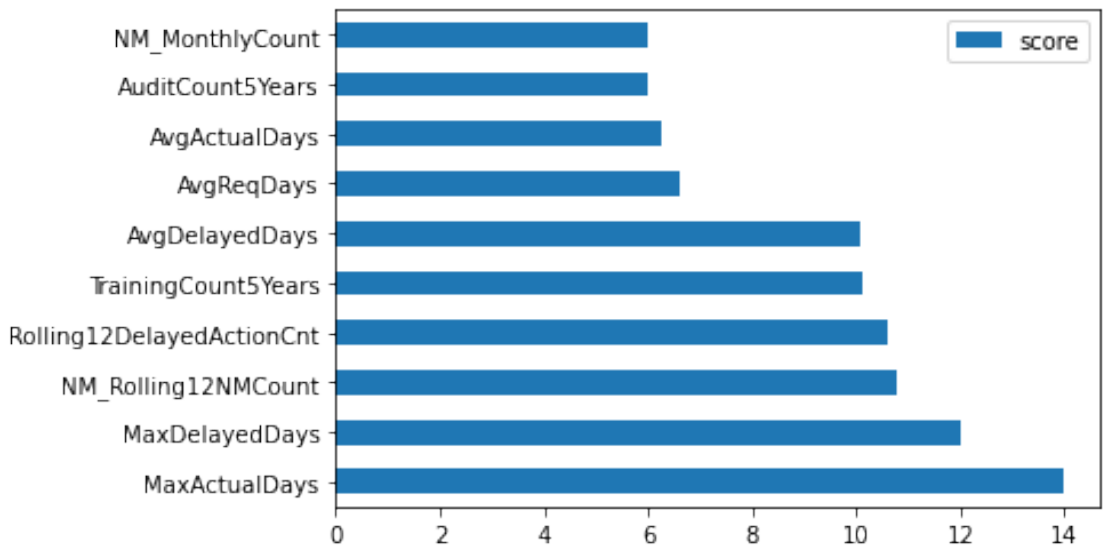
[133]:
```
feature_important = model.get_booster().get_score(importance_type='weight')
keys = list(feature_important.keys())
values = list(feature_important.values())

data = pd.DataFrame(data=values, index=keys, columns=["score"]).sort_values(by =␣
↪"score", ascending=False)
data.plot(kind='barh')
```

[133]: <AxesSubplot:>



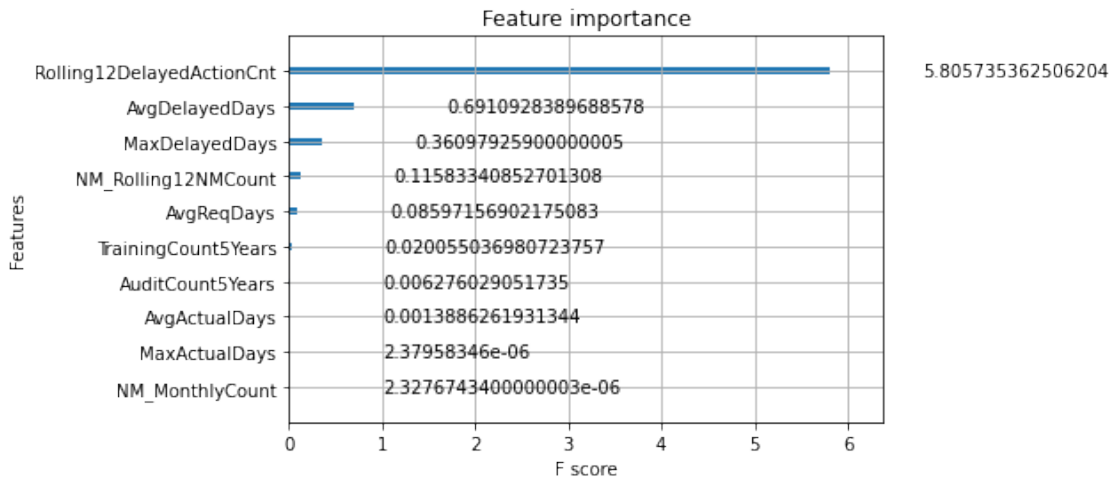
[134]:
```
feature_important = model.get_booster().get_score(importance_type='cover')
keys = list(feature_important.keys())
values = list(feature_important.values())

data = pd.DataFrame(data=values, index=keys, columns=["score"]).sort_values(by =␣
↪"score", ascending=False)
data.plot(kind='barh')
```

[134]: <AxesSubplot:>



Apply a build-in plot_importance() function as an alternative method to plot feature importance

[135]:
```
plot_importance(model, importance_type='gain')
plt.figure(figsize = (16, 12))
plt.show()
```




```
<Figure size 1152x864 with 0 Axes>
```

[ ]:

```
```

[136]:

```
feature_important = model.get_booster().get_score(importance_type='gain')
feature_important_ = pd.DataFrame.from_dict(feature_important, orient='index',␣
↪columns=['feature_importance'])
feature_important_ = feature_important_.sort_values(by='feature_importance',␣
↪ascending=False)
rank_serial = range(1, len(feature_important_) +1)
#list(rank_serial)

feature_important_['XGBoost (Default)']= list(rank_serial) #rank
ranking_xbg_c_b = feature_important_.drop("feature_importance", axis=1)
ranking_xbg_c_b
```

[136]:

	XGBoost (Default)
Rolling12DelayedActionCnt	1
AvgDelayedDays	2
MaxDelayedDays	3
NM_Rolling12NMCount	4
AvgReqDays	5
TrainingCount5Years	6
AuditCount5Years	7
AvgActualDays	8
MaxActualDays	9
NM_MonthlyCount	10

Parameter Optimization

[137]:
```
# A parameter grid for XGBoost
params = {
        'min_child_weight': [1, 5, 10],
        'gamma': [0.5, 1, 2],
        #'subsample': [0.6, 0.8, 1.0],
        #'colsample_bytree': [0.6, 0.8, 1.0],
        'max_depth': [3, 4, 5]
        }
```

[138]:
```
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV
cv = RepeatedKFold(n_splits=5, n_repeats=10, random_state=0)
model = XGBRegressor()
xgb_grid = GridSearchCV(model, params, scoring="neg_mean_absolute_error", cv =␣
 ↪cv, n_jobs = -1,verbose=False)
```

[139]:
```
xgb_grid.fit(X, y)
```

[139]:
```
GridSearchCV(cv=RepeatedKFold(n_repeats=10, n_splits=5, random_state=0),
             estimator=XGBRegressor(base_score=None, booster=None,
                                    colsample_bylevel=None,
                                    colsample_bynode=None,
                                    colsample_bytree=None, gamma=None,
                                    gpu_id=None, importance_type='gain',
                                    interaction_constraints=None,
                                    learning_rate=None, max_delta_step=None,
                                    max_depth=None, min_child_weight=None,
                                    missing=na...constraints=None,
                                    n_estimators=100, n_jobs=None,
                                    num_parallel_tree=None, random_state=None,
                                    reg_alpha=None, reg_lambda=None,
                                    scale_pos_weight=None, subsample=None,
                                    tree_method=None, validate_parameters=None,
                                    verbosity=None),
             n_jobs=-1,
             param_grid={'gamma': [0.5, 1, 2], 'max_depth': [3, 4, 5],
                         'min_child_weight': [1, 5, 10]},
             scoring='neg_mean_absolute_error', verbose=False)
```

[140]:
```
print(xgb_grid.best_score_)
print(xgb_grid.best_params_)
```

```
-1.5748042417450594
{'gamma': 0.5, 'max_depth': 3, 'min_child_weight': 1}
```

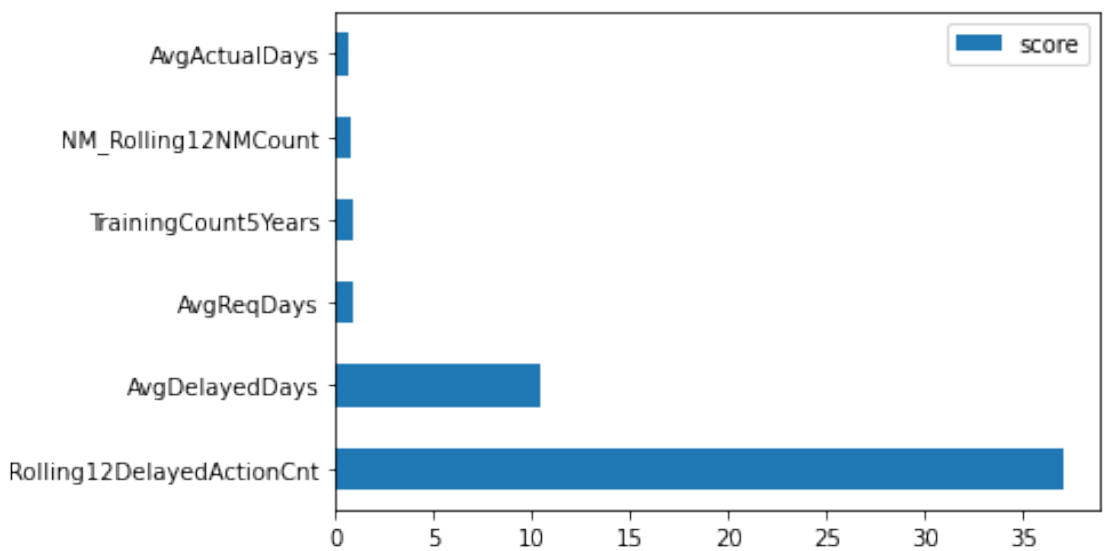
3.4.6 Feature Importance of The Best Estimator

[141]:
```
xgb_grid.best_estimator_.get_booster().get_score(importance_type='gain')
```

[141]:
```
{'Rolling12DelayedActionCnt': 37.073231759947376,
 'AvgDelayedDays': 10.37497046,
 'AvgReqDays': 0.9188615073333333,
 'NM_Rolling12NMCount': 0.749077976,
 'AvgActualDays': 0.62513113,
 'TrainingCount5Years': 0.85164392}
```

[142]:
```
feature_important = xgb_grid.best_estimator_.get_booster().
 ↪get_score(importance_type='gain')
keys = list(feature_important.keys())
values = list(feature_important.values())

data = pd.DataFrame(data=values, index=keys, columns=["score"]).sort_values(by =␣
 ↪"score", ascending=False)
data.plot(kind='barh')
```

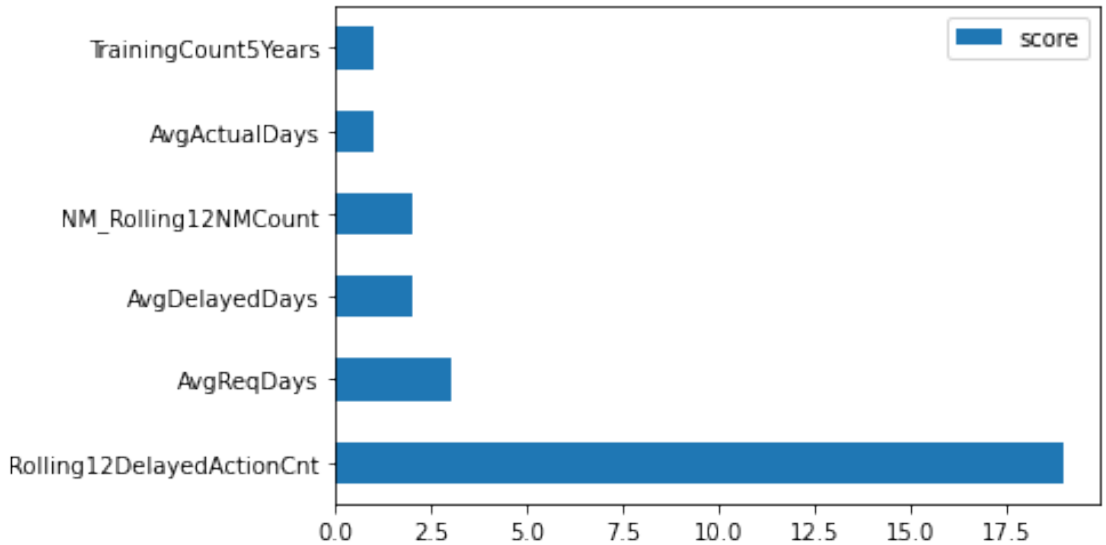
[142]: <AxesSubplot:>



[143]:
```
feature_important = xgb_grid.best_estimator_.get_booster().
 ↪get_score(importance_type='weight')
keys = list(feature_important.keys())
values = list(feature_important.values())
```

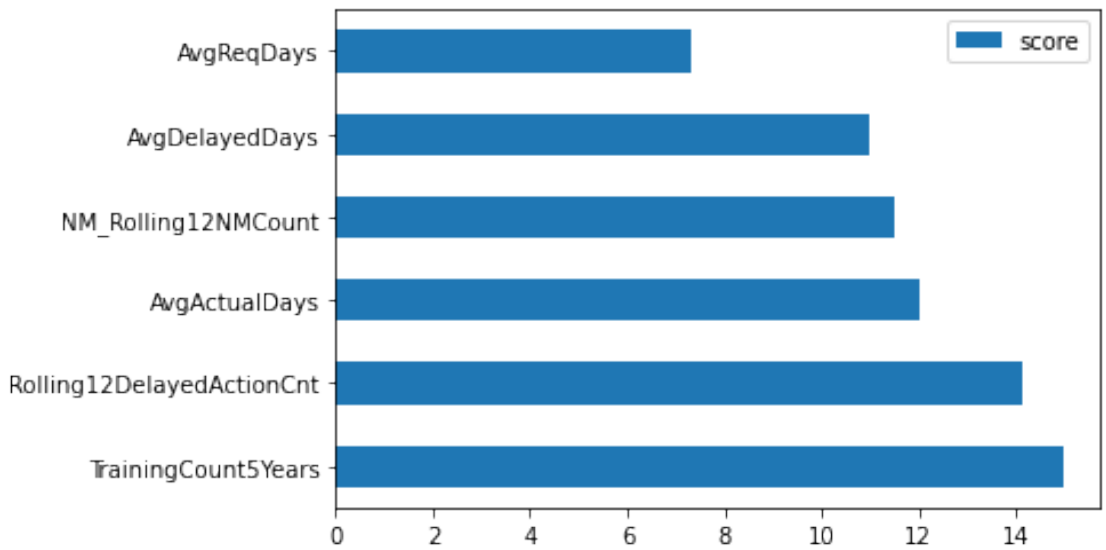
```
data = pd.DataFrame(data=values, index=keys, columns=["score"]).sort_values(by =␣
 ↪"score", ascending=False)
data.plot(kind='barh')
```

[143]: <AxesSubplot:>



[144]:
```
feature_important = xgb_grid.best_estimator_.get_booster().
 ↪get_score(importance_type='cover')
keys = list(feature_important.keys())
values = list(feature_important.values())

data = pd.DataFrame(data=values, index=keys, columns=["score"]).sort_values(by =␣
 ↪"score", ascending=False)
data.plot(kind='barh')
```

[144]: <AxesSubplot:>



[ ]:

```

```

[145]:

```
feature_important = xgb_grid.best_estimator_.get_booster().
 ↪get_score(importance_type='gain')
feature_important_ = pd.DataFrame.from_dict(feature_important, orient='index',␣
 ↪columns=['feature_importance'])
feature_important_ = feature_important_.sort_values(by='feature_importance',␣
 ↪ascending=False)
rank_serial = range(1, len(feature_important_) +1)

feature_important_['XGBoost (Optimised)']= list(rank_serial) #rank
ranking_xbg_c_o = feature_important_.drop("feature_importance", axis=1)
ranking_xbg_c_o
```

[145]:

	XGBoost (Optimised)
Rolling12DelayedActionCnt	1
AvgDelayedDays	2
AvgReqDays	3
TrainingCount5Years	4
NM_Rolling12NMCount	5
AvgActualDays	6

[146]:

```
ranking_xbg_c = pd.concat([ranking_xbg_c_o, ranking_xbg_c_b], axis=1)
ranking_xbg_c
```

[146]:

	XGBoost (Optimised)	XGBoost (Default)
Rolling12DelayedActionCnt	1.0	1
AvgDelayedDays	2.0	2

```
AvgReqDays                                3.0                  5
TrainingCount5Years                       4.0                  6
NM_Rolling12NMCount                       5.0                  4
AvgActualDays                             6.0                  8
MaxDelayedDays                            NaN                  3
AuditCount5Years                          NaN                  7
MaxActualDays                             NaN                  9
NM_MonthlyCount                           NaN                 10
```

With the optimised hyperparameters, the number of important features decreases. Compared to the GradientBoosting approach, this suggests that the XGBoost has learned a more concise model with roughly equal accuracy.

[147]:
```
rank_c = pd.concat([rank_c, ranking_xbg_c_o], axis=1)
rank_c
```

[147]:
```
                           GradientBoosting  XGBoost (Optimised)
Rolling12DelayedActionCnt                 1                  1.0
DelayedPerEmp                             2                  NaN
AvgReqDays                                3                  3.0
NM_Rolling12NMCount                       4                  5.0
AvgActualDays                             5                  6.0
AvgDelayedDays                            6                  2.0
MinActualDays                             7                  NaN
MinReqDays                                8                  NaN
MaxDelayedDays                            9                  NaN
NM_MonthlyCount                          10                  NaN
MaxReqDays                               11                  NaN
AuditCount5Years                         12                  NaN
TrainingCount5Years                      13                  4.0
MaxActualDays                            14                  NaN
MinDelayedDays                           15                  NaN
```

[148]:
```
print("Accuracy:")
{k: v for k, v in sorted(accuracy.items(), key=lambda i: i[1], reverse=True)}
```

```
Accuracy:
```

[148]:
```
{'GradientBoosting': -1.459, 'XGB': -1.581, 'HistGradientBoosting': -5.122}
```

3.5 LightGBM

3.5.1 Import Libraries

[149]:
```
#!conda install -c conda-forge lightgbm
from lightgbm import LGBMRegressor
```

3.5.2 import model

[150]:
```
model = LGBMRegressor()
```

3.5.3 cross-validation

[151]:
```
start = time()
cv = RepeatedKFold(n_splits=5, n_repeats=50, random_state=0)
score = cross_val_score(model, X, y, scoring="neg_mean_absolute_error", cv=cv,␣
 ↪n_jobs=-1)

speed["LGBM"] = np.round(time() - start, 3)
accuracy["LGBM"] = np.nanmean(score).round(3)

print(
    f"Mean Accuracy: {accuracy['LGBM']}\nStd: {np.std(score):.3f}\nRun time:␣
 ↪{speed['LGBM']}s"
)
```

```
Mean Accuracy: -5.122
Std: 1.674
Run time: 1.583s
```

[152]:
```
#score
```

3.5.4 fit the model

[153]:
```
# fit the model on the whole dataset
#model = LGBMRegressor(importance_type='gain')
model.fit(X, y)
```

[153]:
```
LGBMRegressor()
```

3.5.5 Plot Feature Importance

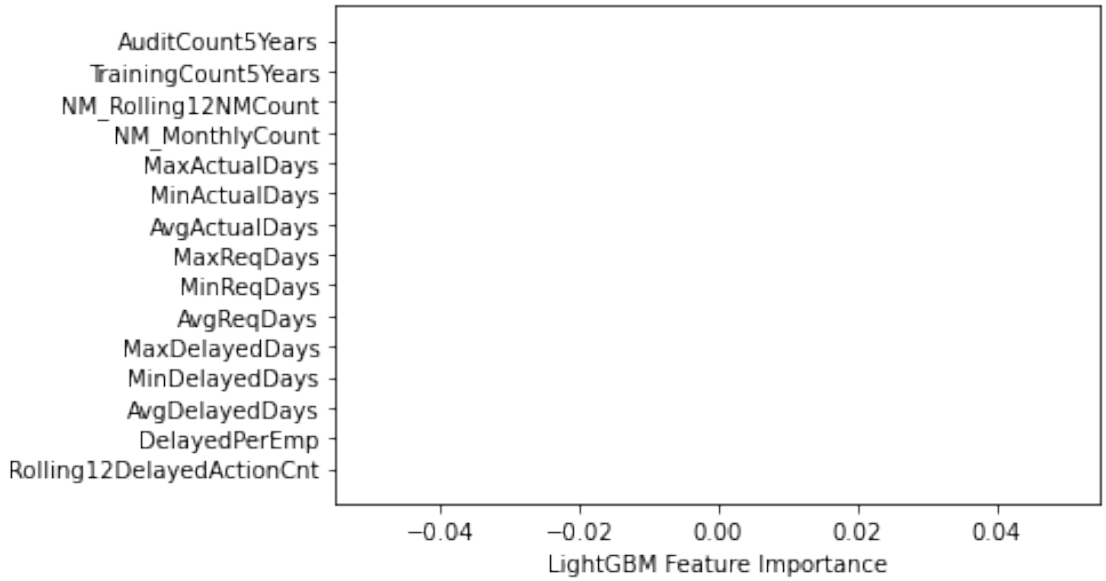
[154]:
```
print("Feature Importance: ",model.feature_importances_)
```

```
Feature Importance:  [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

[155]:
```
#model.booster_.feature_importance(importance_type='gain')
```

[156]:
```
plt.barh(X.columns, model.feature_importances_)
plt.xlabel("LightGBM Feature Importance")
```

[156]:
```
Text(0.5, 0, 'LightGBM Feature Importance')
```



3.5.6 Hyperparameter Optimaztion

[157]:
```
from scipy.stats import randint as sp_randint
from scipy.stats import uniform as sp_uniform
param_test ={
    'num_leaves': sp_randint(6, 50),
    'min_child_samples': sp_randint(100, 500),
    'min_child_weight': [1e-5, 1e-3, 1e-2, 1e-1, 1, 1e1, 1e2, 1e3, 1e4],
    'subsample': sp_uniform(loc=0.2, scale=0.8),
    'colsample_bytree': sp_uniform(loc=0.4, scale=0.6),
    'reg_alpha': [0, 1e-1, 1, 2, 5, 7, 10, 50, 100],
    'reg_lambda': [0, 1e-1, 1, 5, 10, 20, 50, 100]
}
```

[ ]:
```

```

[158]:
```
cv = RepeatedKFold(n_splits=5, n_repeats=10, random_state=0)
model = LGBMRegressor()
lgb_grid = GridSearchCV(model, params, scoring="neg_mean_absolute_error", cv =␣
 ↪cv, n_jobs = -1,verbose=False)
```

[159]:
```
lgb_grid.fit(X, y)
```

[159]:
```
GridSearchCV(cv=RepeatedKFold(n_repeats=10, n_splits=5, random_state=0),
             estimator=LGBMRegressor(), n_jobs=-1,
             param_grid={'gamma': [0.5, 1, 2], 'max_depth': [3, 4, 5],
                         'min_child_weight': [1, 5, 10]},
```

```
                    scoring='neg_mean_absolute_error', verbose=False)
```

[160]:
```
print(lgb_grid.best_score_)
print(lgb_grid.best_params_)
```

```
-5.123304369121951
{'gamma': 0.5, 'max_depth': 3, 'min_child_weight': 1}
```

3.5.7 Feature Importance of The Best Estimator

[161]:
```
lgb_grid.best_estimator_.feature_importances_
```

[161]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

This suggests that the lightGBM fails to achieve feature importance in this case.

[ ]:
```

```

[ ]:
```

```

3.6 Catboost

3.6.1 Import Libraries

[162]:
```
#conda install -c conda-forge lightgbm
from catboost import CatBoostRegressor
```

3.6.2 import model

[163]:
```
model = CatBoostRegressor()
```

3.6.3 cross-validation

[164]:
```
start = time()
cv = RepeatedKFold(n_splits=5, n_repeats=50, random_state=0)
score = cross_val_score(model, X, y, scoring="neg_mean_absolute_error", cv=cv,␣
 ↪n_jobs=-1)

speed["CatBoost"] = np.round(time() - start, 3)
accuracy["CatBoost"] = np.nanmean(score).round(3)

print(
    f"Mean Accuracy: {accuracy['CatBoost']}\nStd: {np.std(score):.3f}\nRun time:␣
 ↪{speed['CatBoost']}s"
)
```

```
Mean Accuracy: -2.119
Std: 1.677
Run time: 132.857s
```

[165]:
```
#score
```

[166]:
```
print("Accuracy:")
{k: v for k, v in sorted(accuracy.items(), key=lambda i: i[1], reverse=True)}
```

```
Accuracy:
```

[166]:
```
{'GradientBoosting': -1.459,
 'XGB': -1.581,
 'CatBoost': -2.119,
 'HistGradientBoosting': -5.122,
 'LGBM': -5.122}
```

[167]:
```
print("Speed:")
{k: v for k, v in sorted(speed.items(), key=lambda i: i[1], reverse=False)}
```

```
Speed:
```

[167]:
```
{'LGBM': 1.583,
 'GradientBoosting': 3.78,
 'HistGradientBoosting': 5.728,
 'XGB': 6.869,
 'CatBoost': 132.857}
```

3.6.4 fit the model

[168]:
```
# fit the model on the whole dataset
#model = CatBoostRegressor()
model.fit(X, y,verbose_eval=False)
```

[168]:
```
<catboost.core.CatBoostRegressor at 0xd3868e0>
```

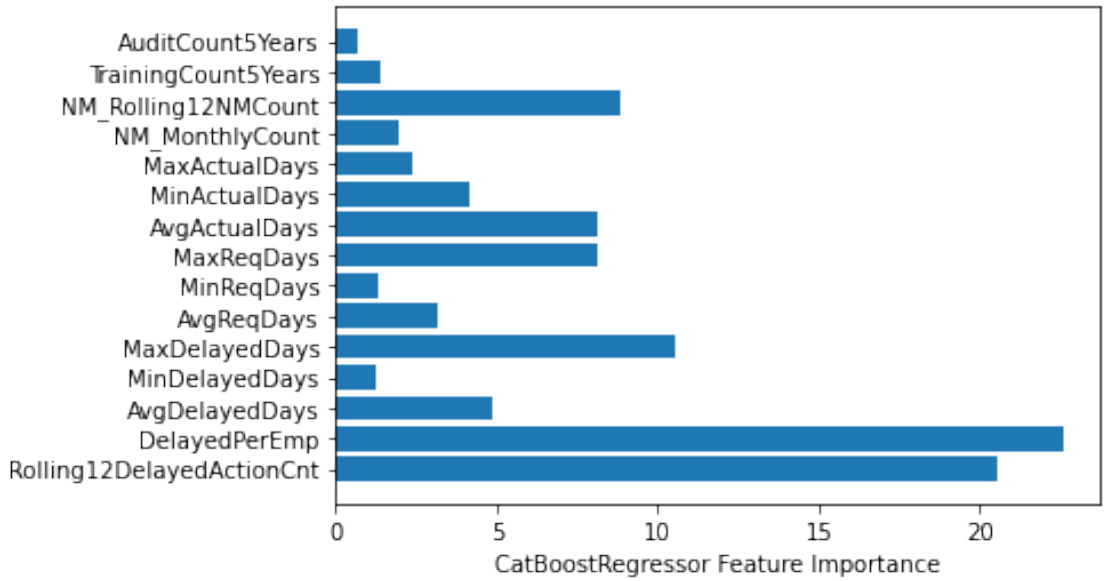
3.6.5 Plot Feature Importance

[169]:
```
print("Feature Importance: ",model.feature_importances_)
```

```
Feature Importance:  [20.5678865  22.6333273   4.83569783  1.27829703
10.54951566  3.17974758
  1.31560452  8.12781023  8.12000687  4.14687287  2.39215217  1.95552907
  8.84347322  1.36616674  0.68791242]
```

[170]:
```
plt.barh(model.feature_names_, model.feature_importances_)
plt.xlabel("CatBoostRegressor Feature Importance")
```

[170]: Text(0.5, 0, 'CatBoostRegressor Feature Importance')



[171]:
```
sorted_idx = model.feature_importances_.argsort()
rank_cat_c_b = pd.DataFrame( (np.arange(1,len(model.
 ↪feature_importances_[sorted_idx])+1,1)),index=X.columns[sorted_idx][::-1],␣
 ↪columns=["CatBoost (Default)"])
rank_cat_c_b
```

[171]:

	CatBoost (Default)
DelayedPerEmp	1
Rolling12DelayedActionCnt	2
MaxDelayedDays	3
NM_Rolling12NMCount	4
MaxReqDays	5
AvgActualDays	6
AvgDelayedDays	7
MinActualDays	8
AvgReqDays	9
MaxActualDays	10
NM_MonthlyCount	11
TrainingCount5Years	12
MinReqDays	13
MinDelayedDays	14
AuditCount5Years	15

[ ]:
```
```

[172]:
```
#dir(model)
```

[173]:
```
model.get_all_params()
```

[173]:
```
{'nan_mode': 'Min',
 'eval_metric': 'RMSE',
 'iterations': 1000,
 'sampling_frequency': 'PerTree',
 'leaf_estimation_method': 'Newton',
 'grow_policy': 'SymmetricTree',
 'penalties_coefficient': 1,
 'boosting_type': 'Plain',
 'model_shrink_mode': 'Constant',
 'feature_border_type': 'GreedyLogSum',
 'bayesian_matrix_reg': 0.10000000149011612,
 'l2_leaf_reg': 3,
 'random_strength': 1,
 'rsm': 1,
 'boost_from_average': True,
 'model_size_reg': 0.5,
 'subsample': 1,
 'use_best_model': False,
 'random_seed': 0,
 'depth': 6,
 'posterior_sampling': False,
 'border_count': 254,
 'classes_count': 0,
 'auto_class_weights': 'None',
 'sparse_features_conflict_fraction': 0,
 'leaf_estimation_backtracking': 'AnyImprovement',
 'best_model_min_trees': 1,
 'model_shrink_rate': 0,
 'min_data_in_leaf': 1,
 'loss_function': 'RMSE',
 'learning_rate': 0.01848199963569641,
 'score_function': 'Cosine',
 'task_type': 'CPU',
 'leaf_estimation_iterations': 1,
 'bootstrap_type': 'MVS',
 'max_leaves': 64}
```

[174]:
```
model.learning_rate_
```

[174]:
```
0.01848199963569641
```

3.6.6 Hyperparameter Optimaztion

[175]:
```
parameters = {'depth'         : [6,8,10],
          'learning_rate' : [0.01, 0.02, 0.1],
          'iterations'    : [100, 500, 1000]
         }
model = CatBoostRegressor()
cat_grid = GridSearchCV(estimator=model, param_grid = parameters,␣
 ↪scoring="neg_mean_absolute_error", cv = 5, n_jobs = -1,verbose=False)
```

[176]:
```
cat_grid.fit(X, y)
```

```
0:      learn: 5.5898992        total: 579us    remaining: 579ms
1:      learn: 5.5710423        total: 1.18ms   remaining: 589ms
2:      learn: 5.5490394        total: 1.52ms   remaining: 505ms
3:      learn: 5.5241129        total: 1.83ms   remaining: 457ms
4:      learn: 5.4972387        total: 2.3ms    remaining: 457ms
5:      learn: 5.4757961        total: 2.76ms   remaining: 457ms
6:      learn: 5.4570062        total: 3.21ms   remaining: 456ms
7:      learn: 5.4377315        total: 3.81ms   remaining: 472ms
8:      learn: 5.4173607        total: 3.97ms   remaining: 437ms
9:      learn: 5.3973785        total: 4.54ms   remaining: 449ms
10:     learn: 5.3725703        total: 4.93ms   remaining: 443ms
11:     learn: 5.3524792        total: 5.33ms   remaining: 439ms
12:     learn: 5.3278565        total: 5.65ms   remaining: 429ms
13:     learn: 5.3083612        total: 6.04ms   remaining: 425ms
14:     learn: 5.2904360        total: 6.37ms   remaining: 419ms
15:     learn: 5.2707947        total: 6.77ms   remaining: 416ms
16:     learn: 5.2477165        total: 7.02ms   remaining: 406ms
17:     learn: 5.2277349        total: 7.5ms    remaining: 409ms
18:     learn: 5.2029494        total: 7.82ms   remaining: 404ms
19:     learn: 5.1821680        total: 8.37ms   remaining: 410ms
20:     learn: 5.1568555        total: 8.67ms   remaining: 404ms
21:     learn: 5.1357995        total: 9.16ms   remaining: 407ms
22:     learn: 5.1076592        total: 9.58ms   remaining: 407ms
23:     learn: 5.0913362        total: 10.1ms   remaining: 411ms
24:     learn: 5.0704167        total: 10.4ms   remaining: 405ms
25:     learn: 5.0523885        total: 11ms     remaining: 412ms
26:     learn: 5.0303222        total: 11.5ms   remaining: 414ms
27:     learn: 5.0096548        total: 11.8ms   remaining: 410ms
28:     learn: 4.9903844        total: 12.3ms   remaining: 412ms
29:     learn: 4.9733692        total: 12.8ms   remaining: 414ms
30:     learn: 4.9569788        total: 13.9ms   remaining: 435ms
31:     learn: 4.9348556        total: 14.2ms   remaining: 430ms
32:     learn: 4.9109835        total: 14.4ms   remaining: 423ms
33:     learn: 4.8930295        total: 14.8ms   remaining: 421ms
34:     learn: 4.8730780        total: 15.1ms   remaining: 416ms
35:     learn: 4.8555035        total: 15.3ms   remaining: 409ms
```

```
36:     learn: 4.8349491        total: 15.7ms   remaining: 408ms
37:     learn: 4.8114558        total: 16ms     remaining: 405ms
38:     learn: 4.7923332        total: 16.3ms   remaining: 402ms
39:     learn: 4.7693532        total: 16.5ms   remaining: 397ms
40:     learn: 4.7521685        total: 16.9ms   remaining: 395ms
41:     learn: 4.7334408        total: 17.2ms   remaining: 393ms
42:     learn: 4.7194591        total: 17.7ms   remaining: 394ms
43:     learn: 4.6988576        total: 18ms     remaining: 391ms
44:     learn: 4.6822694        total: 18.4ms   remaining: 389ms
45:     learn: 4.6674643        total: 18.8ms   remaining: 391ms
46:     learn: 4.6451937        total: 19.1ms   remaining: 388ms
47:     learn: 4.6244190        total: 19.9ms   remaining: 395ms
48:     learn: 4.6024117        total: 20.1ms   remaining: 391ms
49:     learn: 4.5856866        total: 20.5ms   remaining: 390ms
50:     learn: 4.5687256        total: 21ms     remaining: 391ms
51:     learn: 4.5536715        total: 21.3ms   remaining: 388ms
52:     learn: 4.5321004        total: 21.5ms   remaining: 384ms
53:     learn: 4.5167572        total: 21.9ms   remaining: 384ms
54:     learn: 4.4990007        total: 22.4ms   remaining: 385ms
55:     learn: 4.4824915        total: 23.1ms   remaining: 389ms
56:     learn: 4.4663155        total: 23.4ms   remaining: 387ms
57:     learn: 4.4411337        total: 23.7ms   remaining: 384ms
58:     learn: 4.4245829        total: 24.1ms   remaining: 385ms
59:     learn: 4.4094390        total: 24.5ms   remaining: 384ms
60:     learn: 4.3916028        total: 25.1ms   remaining: 386ms
61:     learn: 4.3712324        total: 25.6ms   remaining: 387ms
62:     learn: 4.3523185        total: 26ms     remaining: 386ms
63:     learn: 4.3387065        total: 26.3ms   remaining: 385ms
64:     learn: 4.3184783        total: 26.7ms   remaining: 385ms
65:     learn: 4.3069439        total: 27.2ms   remaining: 386ms
66:     learn: 4.2926961        total: 27.8ms   remaining: 387ms
67:     learn: 4.2738927        total: 28.1ms   remaining: 385ms
68:     learn: 4.2589319        total: 28.4ms   remaining: 383ms
69:     learn: 4.2452035        total: 28.9ms   remaining: 384ms
70:     learn: 4.2306306        total: 29.4ms   remaining: 384ms
71:     learn: 4.2157398        total: 29.8ms   remaining: 384ms
72:     learn: 4.2020003        total: 30.4ms   remaining: 386ms
73:     learn: 4.1885199        total: 30.8ms   remaining: 385ms
74:     learn: 4.1686491        total: 31.1ms   remaining: 384ms
75:     learn: 4.1492344        total: 31.4ms   remaining: 381ms
76:     learn: 4.1350424        total: 31.9ms   remaining: 383ms
77:     learn: 4.1233478        total: 32.5ms   remaining: 384ms
78:     learn: 4.1097073        total: 32.8ms   remaining: 382ms
79:     learn: 4.0964138        total: 33.2ms   remaining: 382ms
80:     learn: 4.0799246        total: 33.6ms   remaining: 382ms
81:     learn: 4.0649152        total: 34ms     remaining: 380ms
82:     learn: 4.0543205        total: 34.2ms   remaining: 378ms
83:     learn: 4.0410275        total: 34.7ms   remaining: 379ms
```

```
84:     learn: 4.0215331        total: 35ms     remaining: 377ms
85:     learn: 4.0043633        total: 35.3ms   remaining: 375ms
86:     learn: 3.9907182        total: 35.8ms   remaining: 375ms
87:     learn: 3.9792814        total: 36.3ms   remaining: 376ms
88:     learn: 3.9566523        total: 36.6ms   remaining: 374ms
89:     learn: 3.9438376        total: 37.1ms   remaining: 375ms
90:     learn: 3.9262949        total: 37.4ms   remaining: 374ms
91:     learn: 3.9089795        total: 37.7ms   remaining: 372ms
92:     learn: 3.8903649        total: 38ms     remaining: 370ms
93:     learn: 3.8770181        total: 38.5ms   remaining: 371ms
94:     learn: 3.8646640        total: 38.7ms   remaining: 369ms
95:     learn: 3.8476771        total: 39.1ms   remaining: 369ms
96:     learn: 3.8311678        total: 39.5ms   remaining: 367ms
97:     learn: 3.8175876        total: 39.9ms   remaining: 367ms
98:     learn: 3.8043698        total: 40.4ms   remaining: 367ms
99:     learn: 3.7863309        total: 40.8ms   remaining: 367ms
100:    learn: 3.7704769        total: 41.2ms   remaining: 367ms
101:    learn: 3.7533727        total: 41.6ms   remaining: 367ms
102:    learn: 3.7362356        total: 41.9ms   remaining: 365ms
103:    learn: 3.7197287        total: 42.2ms   remaining: 364ms
104:    learn: 3.7021500        total: 42.4ms   remaining: 362ms
105:    learn: 3.6905951        total: 42.9ms   remaining: 362ms
106:    learn: 3.6750785        total: 43.2ms   remaining: 360ms
107:    learn: 3.6583900        total: 43.4ms   remaining: 358ms
108:    learn: 3.6452396        total: 43.8ms   remaining: 358ms
109:    learn: 3.6332343        total: 44.3ms   remaining: 358ms
110:    learn: 3.6205253        total: 44.6ms   remaining: 357ms
111:    learn: 3.6011685        total: 44.9ms   remaining: 356ms
112:    learn: 3.5843373        total: 45.2ms   remaining: 354ms
113:    learn: 3.5690705        total: 45.4ms   remaining: 353ms
114:    learn: 3.5545021        total: 45.9ms   remaining: 353ms
115:    learn: 3.5428480        total: 46.2ms   remaining: 352ms
116:    learn: 3.5300199        total: 46.6ms   remaining: 351ms
117:    learn: 3.5131871        total: 46.8ms   remaining: 350ms
118:    learn: 3.4994253        total: 47.2ms   remaining: 350ms
119:    learn: 3.4827698        total: 47.5ms   remaining: 348ms
120:    learn: 3.4713171        total: 47.9ms   remaining: 348ms
121:    learn: 3.4565767        total: 48.2ms   remaining: 347ms
122:    learn: 3.4456887        total: 48.7ms   remaining: 347ms
123:    learn: 3.4333638        total: 49.3ms   remaining: 348ms
124:    learn: 3.4183552        total: 49.7ms   remaining: 348ms
125:    learn: 3.4087986        total: 50.3ms   remaining: 349ms
126:    learn: 3.3955903        total: 51ms     remaining: 351ms
127:    learn: 3.3825852        total: 51.6ms   remaining: 352ms
128:    learn: 3.3724796        total: 52.1ms   remaining: 352ms
129:    learn: 3.3603412        total: 52.4ms   remaining: 351ms
130:    learn: 3.3458343        total: 52.9ms   remaining: 351ms
131:    learn: 3.3336980        total: 53.3ms   remaining: 351ms
```

```
132:    learn: 3.3228507        total: 53.8ms   remaining: 351ms
133:    learn: 3.3071589        total: 54.1ms   remaining: 349ms
134:    learn: 3.2887102        total: 54.4ms   remaining: 349ms
135:    learn: 3.2748846        total: 54.8ms   remaining: 348ms
136:    learn: 3.2595560        total: 55ms     remaining: 347ms
137:    learn: 3.2515824        total: 55.5ms   remaining: 347ms
138:    learn: 3.2392660        total: 56ms     remaining: 347ms
139:    learn: 3.2304119        total: 56.5ms   remaining: 347ms
140:    learn: 3.2164283        total: 56.9ms   remaining: 347ms
141:    learn: 3.2014300        total: 57.3ms   remaining: 346ms
142:    learn: 3.1907416        total: 57.8ms   remaining: 346ms
143:    learn: 3.1809249        total: 58.3ms   remaining: 346ms
144:    learn: 3.1674101        total: 58.8ms   remaining: 347ms
145:    learn: 3.1569979        total: 59.4ms   remaining: 347ms
146:    learn: 3.1466213        total: 59.9ms   remaining: 348ms
147:    learn: 3.1363523        total: 60.6ms   remaining: 349ms
148:    learn: 3.1233528        total: 61ms     remaining: 349ms
149:    learn: 3.1138109        total: 61.2ms   remaining: 347ms
150:    learn: 3.1036809        total: 61.9ms   remaining: 348ms
151:    learn: 3.0907227        total: 62.3ms   remaining: 347ms
152:    learn: 3.0806809        total: 62.7ms   remaining: 347ms
153:    learn: 3.0668799        total: 63ms     remaining: 346ms
154:    learn: 3.0530321        total: 63.3ms   remaining: 345ms
155:    learn: 3.0399651        total: 63.7ms   remaining: 345ms
156:    learn: 3.0263988        total: 64ms     remaining: 344ms
157:    learn: 3.0107305        total: 64.2ms   remaining: 342ms
158:    learn: 3.0016768        total: 64.4ms   remaining: 341ms
159:    learn: 2.9912054        total: 64.9ms   remaining: 341ms
160:    learn: 2.9775246        total: 65.1ms   remaining: 339ms
161:    learn: 2.9646031        total: 65.4ms   remaining: 338ms
162:    learn: 2.9544374        total: 65.7ms   remaining: 338ms
163:    learn: 2.9417192        total: 66.1ms   remaining: 337ms
164:    learn: 2.9316970        total: 66.6ms   remaining: 337ms
165:    learn: 2.9220637        total: 67.1ms   remaining: 337ms
166:    learn: 2.9134127        total: 67.3ms   remaining: 336ms
167:    learn: 2.9054665        total: 67.9ms   remaining: 336ms
168:    learn: 2.8960453        total: 68.4ms   remaining: 336ms
169:    learn: 2.8798195        total: 68.6ms   remaining: 335ms
170:    learn: 2.8712677        total: 69.2ms   remaining: 335ms
171:    learn: 2.8592033        total: 69.4ms   remaining: 334ms
172:    learn: 2.8462351        total: 69.7ms   remaining: 333ms
173:    learn: 2.8381588        total: 69.9ms   remaining: 332ms
174:    learn: 2.8285874        total: 70.3ms   remaining: 331ms
175:    learn: 2.8165388        total: 70.6ms   remaining: 331ms
176:    learn: 2.8072329        total: 71.1ms   remaining: 330ms
177:    learn: 2.7978243        total: 71.6ms   remaining: 330ms
178:    learn: 2.7886802        total: 71.9ms   remaining: 330ms
179:    learn: 2.7774094        total: 72.4ms   remaining: 330ms
```

```
180:    learn: 2.7684857        total: 72.9ms   remaining: 330ms
181:    learn: 2.7570979        total: 73.2ms   remaining: 329ms
182:    learn: 2.7491434        total: 73.4ms   remaining: 328ms
183:    learn: 2.7398532        total: 73.9ms   remaining: 328ms
184:    learn: 2.7306012        total: 74.3ms   remaining: 327ms
185:    learn: 2.7197068        total: 74.6ms   remaining: 326ms
186:    learn: 2.7099415        total: 74.9ms   remaining: 326ms
187:    learn: 2.6987664        total: 75.3ms   remaining: 325ms
188:    learn: 2.6887743        total: 75.6ms   remaining: 324ms
189:    learn: 2.6767454        total: 76ms     remaining: 324ms
190:    learn: 2.6657796        total: 76.3ms   remaining: 323ms
191:    learn: 2.6538504        total: 76.6ms   remaining: 322ms
192:    learn: 2.6457594        total: 76.9ms   remaining: 321ms
193:    learn: 2.6370510        total: 77.2ms   remaining: 321ms
194:    learn: 2.6271150        total: 77.6ms   remaining: 320ms
195:    learn: 2.6174840        total: 77.9ms   remaining: 320ms
196:    learn: 2.6059588        total: 78.5ms   remaining: 320ms
197:    learn: 2.5951349        total: 78.9ms   remaining: 320ms
198:    learn: 2.5843443        total: 79.2ms   remaining: 319ms
199:    learn: 2.5709375        total: 79.7ms   remaining: 319ms
200:    learn: 2.5608376        total: 80.2ms   remaining: 319ms
201:    learn: 2.5523656        total: 80.7ms   remaining: 319ms
202:    learn: 2.5434620        total: 81ms     remaining: 318ms
203:    learn: 2.5354665        total: 81.5ms   remaining: 318ms
204:    learn: 2.5251309        total: 81.8ms   remaining: 317ms
205:    learn: 2.5172914        total: 82.3ms   remaining: 317ms
206:    learn: 2.5033823        total: 82.6ms   remaining: 316ms
207:    learn: 2.4949051        total: 83ms     remaining: 316ms
208:    learn: 2.4865000        total: 83.5ms   remaining: 316ms
209:    learn: 2.4782134        total: 83.8ms   remaining: 315ms
210:    learn: 2.4674671        total: 84.2ms   remaining: 315ms
211:    learn: 2.4600271        total: 84.6ms   remaining: 315ms
212:    learn: 2.4519955        total: 84.9ms   remaining: 314ms
213:    learn: 2.4418046        total: 85.4ms   remaining: 314ms
214:    learn: 2.4340130        total: 85.9ms   remaining: 314ms
215:    learn: 2.4265640        total: 86.4ms   remaining: 313ms
216:    learn: 2.4178994        total: 86.8ms   remaining: 313ms
217:    learn: 2.4105212        total: 87.3ms   remaining: 313ms
218:    learn: 2.3996975        total: 87.5ms   remaining: 312ms
219:    learn: 2.3916251        total: 87.9ms   remaining: 312ms
220:    learn: 2.3819941        total: 88.3ms   remaining: 311ms
221:    learn: 2.3721519        total: 88.6ms   remaining: 310ms
222:    learn: 2.3630497        total: 88.9ms   remaining: 310ms
223:    learn: 2.3557006        total: 89.4ms   remaining: 310ms
224:    learn: 2.3461591        total: 89.7ms   remaining: 309ms
225:    learn: 2.3380982        total: 90.4ms   remaining: 310ms
226:    learn: 2.3306470        total: 90.7ms   remaining: 309ms
227:    learn: 2.3204694        total: 91ms     remaining: 308ms
```

```
228:    learn: 2.3110933        total: 91.3ms   remaining: 308ms
229:    learn: 2.3008071        total: 91.6ms   remaining: 307ms
230:    learn: 2.2930916        total: 92ms     remaining: 306ms
231:    learn: 2.2859138        total: 92.3ms   remaining: 306ms
232:    learn: 2.2783831        total: 92.7ms   remaining: 305ms
233:    learn: 2.2711599        total: 93.2ms   remaining: 305ms
234:    learn: 2.2649456        total: 93.6ms   remaining: 305ms
235:    learn: 2.2579750        total: 94ms     remaining: 304ms
236:    learn: 2.2514582        total: 94.5ms   remaining: 304ms
237:    learn: 2.2441701        total: 94.8ms   remaining: 304ms
238:    learn: 2.2371117        total: 95.3ms   remaining: 303ms
239:    learn: 2.2303122        total: 95.8ms   remaining: 303ms
240:    learn: 2.2217020        total: 96.3ms   remaining: 303ms
241:    learn: 2.2129006        total: 96.8ms   remaining: 303ms
242:    learn: 2.2050089        total: 97.1ms   remaining: 303ms
243:    learn: 2.1952810        total: 97.5ms   remaining: 302ms
244:    learn: 2.1884738        total: 98ms     remaining: 302ms
245:    learn: 2.1802425        total: 98.5ms   remaining: 302ms
246:    learn: 2.1728918        total: 99ms     remaining: 302ms
247:    learn: 2.1658130        total: 99.3ms   remaining: 301ms
248:    learn: 2.1565070        total: 99.6ms   remaining: 301ms
249:    learn: 2.1488975        total: 100ms    remaining: 300ms
250:    learn: 2.1420378        total: 101ms    remaining: 300ms
251:    learn: 2.1361835        total: 102ms    remaining: 302ms
252:    learn: 2.1270015        total: 102ms    remaining: 301ms
253:    learn: 2.1178974        total: 102ms    remaining: 301ms
254:    learn: 2.1096963        total: 103ms    remaining: 300ms
255:    learn: 2.1011287        total: 103ms    remaining: 300ms
256:    learn: 2.0924967        total: 103ms    remaining: 299ms
257:    learn: 2.0836775        total: 104ms    remaining: 298ms
258:    learn: 2.0747920        total: 104ms    remaining: 298ms
259:    learn: 2.0682519        total: 104ms    remaining: 297ms
260:    learn: 2.0594757        total: 105ms    remaining: 297ms
261:    learn: 2.0508643        total: 105ms    remaining: 296ms
262:    learn: 2.0440051        total: 105ms    remaining: 296ms
263:    learn: 2.0367817        total: 106ms    remaining: 296ms
264:    learn: 2.0316597        total: 106ms    remaining: 295ms
265:    learn: 2.0251096        total: 107ms    remaining: 294ms
266:    learn: 2.0185340        total: 107ms    remaining: 294ms
267:    learn: 2.0105783        total: 108ms    remaining: 294ms
268:    learn: 2.0042626        total: 108ms    remaining: 294ms
269:    learn: 1.9967163        total: 109ms    remaining: 294ms
270:    learn: 1.9896083        total: 109ms    remaining: 294ms
271:    learn: 1.9797196        total: 110ms    remaining: 293ms
272:    learn: 1.9726174        total: 110ms    remaining: 293ms
273:    learn: 1.9676679        total: 111ms    remaining: 293ms
274:    learn: 1.9614771        total: 111ms    remaining: 294ms
275:    learn: 1.9512933        total: 112ms    remaining: 294ms
```

```
276:    learn: 1.9430884        total: 112ms    remaining: 293ms
277:    learn: 1.9363453        total: 113ms    remaining: 293ms
278:    learn: 1.9269352        total: 113ms    remaining: 293ms
279:    learn: 1.9192489        total: 114ms    remaining: 292ms
280:    learn: 1.9110132        total: 114ms    remaining: 291ms
281:    learn: 1.9047068        total: 114ms    remaining: 291ms
282:    learn: 1.8997277        total: 115ms    remaining: 291ms
283:    learn: 1.8930007        total: 115ms    remaining: 291ms
284:    learn: 1.8850996        total: 116ms    remaining: 290ms
285:    learn: 1.8777864        total: 116ms    remaining: 290ms
286:    learn: 1.8717229        total: 117ms    remaining: 290ms
287:    learn: 1.8660211        total: 117ms    remaining: 290ms
288:    learn: 1.8597110        total: 118ms    remaining: 291ms
289:    learn: 1.8517632        total: 119ms    remaining: 291ms
290:    learn: 1.8464053        total: 119ms    remaining: 291ms
291:    learn: 1.8392372        total: 120ms    remaining: 290ms
292:    learn: 1.8321777        total: 120ms    remaining: 289ms
293:    learn: 1.8252592        total: 120ms    remaining: 289ms
294:    learn: 1.8191803        total: 121ms    remaining: 288ms
295:    learn: 1.8133581        total: 121ms    remaining: 288ms
296:    learn: 1.8080458        total: 122ms    remaining: 288ms
297:    learn: 1.8020479        total: 122ms    remaining: 288ms
298:    learn: 1.7963815        total: 123ms    remaining: 287ms
299:    learn: 1.7910182        total: 123ms    remaining: 287ms
300:    learn: 1.7820887        total: 123ms    remaining: 286ms
301:    learn: 1.7752677        total: 123ms    remaining: 285ms
302:    learn: 1.7679923        total: 124ms    remaining: 285ms
303:    learn: 1.7622162        total: 124ms    remaining: 285ms
304:    learn: 1.7550050        total: 125ms    remaining: 284ms
305:    learn: 1.7486756        total: 125ms    remaining: 284ms
306:    learn: 1.7395131        total: 125ms    remaining: 283ms
307:    learn: 1.7310573        total: 126ms    remaining: 283ms
308:    learn: 1.7240041        total: 126ms    remaining: 282ms
309:    learn: 1.7188767        total: 126ms    remaining: 282ms
310:    learn: 1.7133592        total: 127ms    remaining: 281ms
311:    learn: 1.7088111        total: 127ms    remaining: 281ms
312:    learn: 1.7024269        total: 128ms    remaining: 280ms
313:    learn: 1.6960479        total: 128ms    remaining: 280ms
314:    learn: 1.6889801        total: 128ms    remaining: 279ms
315:    learn: 1.6839543        total: 129ms    remaining: 279ms
316:    learn: 1.6788993        total: 129ms    remaining: 279ms
317:    learn: 1.6738918        total: 130ms    remaining: 278ms
318:    learn: 1.6686991        total: 130ms    remaining: 278ms
319:    learn: 1.6619484        total: 131ms    remaining: 278ms
320:    learn: 1.6557450        total: 131ms    remaining: 278ms
321:    learn: 1.6511750        total: 132ms    remaining: 278ms
322:    learn: 1.6452091        total: 132ms    remaining: 277ms
323:    learn: 1.6401369        total: 133ms    remaining: 277ms
```

```
324:    learn: 1.6350706        total: 133ms    remaining: 276ms
325:    learn: 1.6284771        total: 133ms    remaining: 276ms
326:    learn: 1.6225980        total: 134ms    remaining: 275ms
327:    learn: 1.6179481        total: 134ms    remaining: 275ms
328:    learn: 1.6132199        total: 135ms    remaining: 275ms
329:    learn: 1.6083388        total: 135ms    remaining: 275ms
330:    learn: 1.6025506        total: 136ms    remaining: 274ms
331:    learn: 1.5968550        total: 136ms    remaining: 274ms
332:    learn: 1.5921337        total: 137ms    remaining: 274ms
333:    learn: 1.5857684        total: 137ms    remaining: 273ms
334:    learn: 1.5808679        total: 137ms    remaining: 273ms
335:    learn: 1.5760832        total: 138ms    remaining: 272ms
336:    learn: 1.5702580        total: 138ms    remaining: 271ms
337:    learn: 1.5658110        total: 139ms    remaining: 271ms
338:    learn: 1.5595831        total: 139ms    remaining: 271ms
339:    learn: 1.5533906        total: 139ms    remaining: 270ms
340:    learn: 1.5473810        total: 139ms    remaining: 269ms
341:    learn: 1.5430497        total: 140ms    remaining: 269ms
342:    learn: 1.5380818        total: 140ms    remaining: 269ms
343:    learn: 1.5336790        total: 141ms    remaining: 269ms
344:    learn: 1.5261804        total: 141ms    remaining: 268ms
345:    learn: 1.5214607        total: 142ms    remaining: 268ms
346:    learn: 1.5155991        total: 142ms    remaining: 267ms
347:    learn: 1.5110892        total: 143ms    remaining: 268ms
348:    learn: 1.5066887        total: 143ms    remaining: 267ms
349:    learn: 1.5007339        total: 144ms    remaining: 267ms
350:    learn: 1.4930425        total: 144ms    remaining: 266ms
351:    learn: 1.4881039        total: 145ms    remaining: 266ms
352:    learn: 1.4836741        total: 145ms    remaining: 266ms
353:    learn: 1.4765695        total: 145ms    remaining: 265ms
354:    learn: 1.4690167        total: 146ms    remaining: 265ms
355:    learn: 1.4631164        total: 146ms    remaining: 264ms
356:    learn: 1.4573306        total: 146ms    remaining: 264ms
357:    learn: 1.4520568        total: 147ms    remaining: 264ms
358:    learn: 1.4469932        total: 147ms    remaining: 263ms
359:    learn: 1.4416904        total: 148ms    remaining: 263ms
360:    learn: 1.4361129        total: 148ms    remaining: 262ms
361:    learn: 1.4305124        total: 149ms    remaining: 262ms
362:    learn: 1.4267355        total: 149ms    remaining: 262ms
363:    learn: 1.4226142        total: 150ms    remaining: 262ms
364:    learn: 1.4187046        total: 150ms    remaining: 261ms
365:    learn: 1.4132383        total: 151ms    remaining: 261ms
366:    learn: 1.4093314        total: 151ms    remaining: 260ms
367:    learn: 1.4026739        total: 151ms    remaining: 260ms
368:    learn: 1.3980446        total: 152ms    remaining: 259ms
369:    learn: 1.3940280        total: 152ms    remaining: 259ms
370:    learn: 1.3870453        total: 153ms    remaining: 259ms
371:    learn: 1.3816497        total: 153ms    remaining: 258ms
```

```
372:    learn: 1.3778849        total: 153ms    remaining: 258ms
373:    learn: 1.3741301        total: 154ms    remaining: 258ms
374:    learn: 1.3701505        total: 155ms    remaining: 258ms
375:    learn: 1.3635239        total: 155ms    remaining: 258ms
376:    learn: 1.3600723        total: 156ms    remaining: 258ms
377:    learn: 1.3552020        total: 157ms    remaining: 258ms
378:    learn: 1.3500604        total: 157ms    remaining: 258ms
379:    learn: 1.3464311        total: 158ms    remaining: 258ms
380:    learn: 1.3412862        total: 159ms    remaining: 259ms
381:    learn: 1.3375659        total: 160ms    remaining: 259ms
382:    learn: 1.3330514        total: 161ms    remaining: 259ms
383:    learn: 1.3293259        total: 161ms    remaining: 259ms
384:    learn: 1.3254583        total: 162ms    remaining: 259ms
385:    learn: 1.3218663        total: 163ms    remaining: 259ms
386:    learn: 1.3169002        total: 164ms    remaining: 259ms
387:    learn: 1.3134586        total: 164ms    remaining: 259ms
388:    learn: 1.3093363        total: 165ms    remaining: 259ms
389:    learn: 1.3047254        total: 165ms    remaining: 258ms
390:    learn: 1.3010638        total: 166ms    remaining: 259ms
391:    learn: 1.2975981        total: 167ms    remaining: 259ms
392:    learn: 1.2915631        total: 167ms    remaining: 259ms
393:    learn: 1.2877726        total: 168ms    remaining: 258ms
394:    learn: 1.2839907        total: 168ms    remaining: 258ms
395:    learn: 1.2791473        total: 169ms    remaining: 258ms
396:    learn: 1.2756322        total: 169ms    remaining: 257ms
397:    learn: 1.2693478        total: 170ms    remaining: 257ms
398:    learn: 1.2658940        total: 170ms    remaining: 257ms
399:    learn: 1.2621998        total: 171ms    remaining: 256ms
400:    learn: 1.2574808        total: 172ms    remaining: 257ms
401:    learn: 1.2528418        total: 172ms    remaining: 257ms
402:    learn: 1.2470230        total: 173ms    remaining: 256ms
403:    learn: 1.2408914        total: 173ms    remaining: 256ms
404:    learn: 1.2348296        total: 174ms    remaining: 255ms
405:    learn: 1.2309060        total: 174ms    remaining: 255ms
406:    learn: 1.2249985        total: 174ms    remaining: 254ms
407:    learn: 1.2210733        total: 175ms    remaining: 254ms
408:    learn: 1.2176658        total: 175ms    remaining: 254ms
409:    learn: 1.2144583        total: 176ms    remaining: 253ms
410:    learn: 1.2102268        total: 177ms    remaining: 253ms
411:    learn: 1.2068042        total: 177ms    remaining: 253ms
412:    learn: 1.2038462        total: 178ms    remaining: 253ms
413:    learn: 1.2006416        total: 178ms    remaining: 252ms
414:    learn: 1.1972749        total: 179ms    remaining: 252ms
415:    learn: 1.1940149        total: 179ms    remaining: 252ms
416:    learn: 1.1901834        total: 180ms    remaining: 251ms
417:    learn: 1.1868174        total: 180ms    remaining: 251ms
418:    learn: 1.1834871        total: 181ms    remaining: 251ms
419:    learn: 1.1799690        total: 182ms    remaining: 252ms
```

```
420:    learn: 1.1756615        total: 183ms    remaining: 251ms
421:    learn: 1.1726040        total: 183ms    remaining: 251ms
422:    learn: 1.1683314        total: 184ms    remaining: 250ms
423:    learn: 1.1628822        total: 184ms    remaining: 250ms
424:    learn: 1.1573184        total: 184ms    remaining: 249ms
425:    learn: 1.1530995        total: 185ms    remaining: 249ms
426:    learn: 1.1499331        total: 185ms    remaining: 248ms
427:    learn: 1.1467573        total: 186ms    remaining: 248ms
428:    learn: 1.1411796        total: 186ms    remaining: 247ms
429:    learn: 1.1379275        total: 186ms    remaining: 247ms
430:    learn: 1.1348342        total: 187ms    remaining: 247ms
431:    learn: 1.1313842        total: 188ms    remaining: 247ms
432:    learn: 1.1283889        total: 188ms    remaining: 246ms
433:    learn: 1.1248080        total: 189ms    remaining: 246ms
434:    learn: 1.1193496        total: 189ms    remaining: 246ms
435:    learn: 1.1160828        total: 190ms    remaining: 245ms
436:    learn: 1.1131621        total: 190ms    remaining: 245ms
437:    learn: 1.1080263        total: 191ms    remaining: 245ms
438:    learn: 1.1038149        total: 191ms    remaining: 244ms
439:    learn: 1.1000831        total: 192ms    remaining: 244ms
440:    learn: 1.0972550        total: 193ms    remaining: 244ms
441:    learn: 1.0921185        total: 193ms    remaining: 244ms
442:    learn: 1.0889622        total: 194ms    remaining: 244ms
443:    learn: 1.0861840        total: 194ms    remaining: 243ms
444:    learn: 1.0832060        total: 195ms    remaining: 243ms
445:    learn: 1.0793262        total: 196ms    remaining: 243ms
446:    learn: 1.0742398        total: 196ms    remaining: 243ms
447:    learn: 1.0704664        total: 197ms    remaining: 242ms
448:    learn: 1.0670894        total: 197ms    remaining: 242ms
449:    learn: 1.0620873        total: 198ms    remaining: 242ms
450:    learn: 1.0592505        total: 198ms    remaining: 242ms
451:    learn: 1.0543745        total: 199ms    remaining: 241ms
452:    learn: 1.0493643        total: 200ms    remaining: 241ms
453:    learn: 1.0443060        total: 200ms    remaining: 241ms
454:    learn: 1.0404870        total: 201ms    remaining: 240ms
455:    learn: 1.0376215        total: 201ms    remaining: 240ms
456:    learn: 1.0339713        total: 202ms    remaining: 240ms
457:    learn: 1.0306714        total: 202ms    remaining: 239ms
458:    learn: 1.0279239        total: 203ms    remaining: 239ms
459:    learn: 1.0249675        total: 203ms    remaining: 239ms
460:    learn: 1.0214295        total: 204ms    remaining: 238ms
461:    learn: 1.0167700        total: 204ms    remaining: 238ms
462:    learn: 1.0141624        total: 205ms    remaining: 238ms
463:    learn: 1.0102731        total: 206ms    remaining: 237ms
464:    learn: 1.0056345        total: 206ms    remaining: 237ms
465:    learn: 1.0030820        total: 207ms    remaining: 237ms
466:    learn: 1.0001832        total: 207ms    remaining: 237ms
467:    learn: 0.9954299        total: 208ms    remaining: 236ms
```

```
468:    learn: 0.9927606        total: 208ms    remaining: 236ms
469:    learn: 0.9890535        total: 209ms    remaining: 236ms
470:    learn: 0.9855380        total: 209ms    remaining: 235ms
471:    learn: 0.9829159        total: 210ms    remaining: 235ms
472:    learn: 0.9794536        total: 210ms    remaining: 234ms
473:    learn: 0.9768109        total: 211ms    remaining: 234ms
474:    learn: 0.9735659        total: 212ms    remaining: 234ms
475:    learn: 0.9689950        total: 212ms    remaining: 234ms
476:    learn: 0.9654009        total: 213ms    remaining: 234ms
477:    learn: 0.9628877        total: 214ms    remaining: 233ms
478:    learn: 0.9604080        total: 214ms    remaining: 233ms
479:    learn: 0.9558774        total: 215ms    remaining: 233ms
480:    learn: 0.9533354        total: 215ms    remaining: 232ms
481:    learn: 0.9496746        total: 216ms    remaining: 232ms
482:    learn: 0.9464339        total: 216ms    remaining: 232ms
483:    learn: 0.9420227        total: 217ms    remaining: 231ms
484:    learn: 0.9386190        total: 217ms    remaining: 231ms
485:    learn: 0.9358596        total: 218ms    remaining: 230ms
486:    learn: 0.9322749        total: 218ms    remaining: 230ms
487:    learn: 0.9279159        total: 219ms    remaining: 230ms
488:    learn: 0.9246000        total: 219ms    remaining: 229ms
489:    learn: 0.9210594        total: 220ms    remaining: 229ms
490:    learn: 0.9178914        total: 220ms    remaining: 228ms
491:    learn: 0.9135851        total: 221ms    remaining: 228ms
492:    learn: 0.9105664        total: 221ms    remaining: 228ms
493:    learn: 0.9073118        total: 222ms    remaining: 227ms
494:    learn: 0.9030153        total: 224ms    remaining: 229ms
495:    learn: 0.9007592        total: 225ms    remaining: 229ms
496:    learn: 0.8966104        total: 226ms    remaining: 228ms
497:    learn: 0.8937243        total: 226ms    remaining: 228ms
498:    learn: 0.8907998        total: 227ms    remaining: 228ms
499:    learn: 0.8865561        total: 227ms    remaining: 227ms
500:    learn: 0.8831837        total: 228ms    remaining: 227ms
501:    learn: 0.8791280        total: 228ms    remaining: 226ms
502:    learn: 0.8769160        total: 229ms    remaining: 226ms
503:    learn: 0.8730040        total: 230ms    remaining: 226ms
504:    learn: 0.8708649        total: 230ms    remaining: 226ms
505:    learn: 0.8687313        total: 231ms    remaining: 226ms
506:    learn: 0.8665736        total: 232ms    remaining: 226ms
507:    learn: 0.8643156        total: 233ms    remaining: 225ms
508:    learn: 0.8621629        total: 234ms    remaining: 225ms
509:    learn: 0.8600837        total: 234ms    remaining: 225ms
510:    learn: 0.8573710        total: 235ms    remaining: 225ms
511:    learn: 0.8543863        total: 236ms    remaining: 225ms
512:    learn: 0.8504814        total: 236ms    remaining: 224ms
513:    learn: 0.8483997        total: 237ms    remaining: 224ms
514:    learn: 0.8444483        total: 237ms    remaining: 224ms
515:    learn: 0.8421726        total: 238ms    remaining: 223ms
```

```
516:    learn: 0.8381754        total: 239ms    remaining: 223ms
517:    learn: 0.8354639        total: 239ms    remaining: 223ms
518:    learn: 0.8333278        total: 240ms    remaining: 222ms
519:    learn: 0.8304102        total: 241ms    remaining: 222ms
520:    learn: 0.8266123        total: 241ms    remaining: 222ms
521:    learn: 0.8227690        total: 242ms    remaining: 221ms
522:    learn: 0.8189844        total: 242ms    remaining: 221ms
523:    learn: 0.8170160        total: 243ms    remaining: 221ms
524:    learn: 0.8148191        total: 244ms    remaining: 220ms
525:    learn: 0.8126519        total: 244ms    remaining: 220ms
526:    learn: 0.8097262        total: 245ms    remaining: 220ms
527:    learn: 0.8065838        total: 246ms    remaining: 220ms
528:    learn: 0.8045052        total: 246ms    remaining: 219ms
529:    learn: 0.8008290        total: 247ms    remaining: 219ms
530:    learn: 0.7979320        total: 248ms    remaining: 219ms
531:    learn: 0.7942070        total: 248ms    remaining: 218ms
532:    learn: 0.7913741        total: 249ms    remaining: 218ms
533:    learn: 0.7892908        total: 250ms    remaining: 218ms
534:    learn: 0.7856382        total: 251ms    remaining: 218ms
535:    learn: 0.7835456        total: 252ms    remaining: 218ms
536:    learn: 0.7807616        total: 252ms    remaining: 218ms
537:    learn: 0.7779666        total: 253ms    remaining: 217ms
538:    learn: 0.7752179        total: 255ms    remaining: 218ms
539:    learn: 0.7725251        total: 255ms    remaining: 217ms
540:    learn: 0.7690315        total: 256ms    remaining: 217ms
541:    learn: 0.7671391        total: 256ms    remaining: 217ms
542:    learn: 0.7652650        total: 257ms    remaining: 216ms
543:    learn: 0.7625533        total: 257ms    remaining: 216ms
544:    learn: 0.7598213        total: 258ms    remaining: 215ms
545:    learn: 0.7570638        total: 258ms    remaining: 215ms
546:    learn: 0.7543261        total: 259ms    remaining: 214ms
547:    learn: 0.7516451        total: 260ms    remaining: 214ms
548:    learn: 0.7496584        total: 260ms    remaining: 214ms
549:    learn: 0.7468521        total: 261ms    remaining: 213ms
550:    learn: 0.7449531        total: 261ms    remaining: 213ms
551:    learn: 0.7431218        total: 262ms    remaining: 212ms
552:    learn: 0.7396283        total: 262ms    remaining: 212ms
553:    learn: 0.7368115        total: 263ms    remaining: 212ms
554:    learn: 0.7348935        total: 263ms    remaining: 211ms
555:    learn: 0.7320723        total: 264ms    remaining: 211ms
556:    learn: 0.7297403        total: 265ms    remaining: 210ms
557:    learn: 0.7263647        total: 265ms    remaining: 210ms
558:    learn: 0.7245955        total: 266ms    remaining: 210ms
559:    learn: 0.7212558        total: 266ms    remaining: 209ms
560:    learn: 0.7186822        total: 267ms    remaining: 209ms
561:    learn: 0.7161909        total: 267ms    remaining: 208ms
562:    learn: 0.7129275        total: 268ms    remaining: 208ms
563:    learn: 0.7103657        total: 268ms    remaining: 207ms
```

```
564:    learn: 0.7078779        total: 269ms    remaining: 207ms
565:    learn: 0.7053342        total: 269ms    remaining: 207ms
566:    learn: 0.7027620        total: 270ms    remaining: 207ms
567:    learn: 0.7010323        total: 271ms    remaining: 206ms
568:    learn: 0.6993182        total: 272ms    remaining: 206ms
569:    learn: 0.6976443        total: 272ms    remaining: 206ms
570:    learn: 0.6959080        total: 273ms    remaining: 205ms
571:    learn: 0.6927695        total: 273ms    remaining: 205ms
572:    learn: 0.6895530        total: 274ms    remaining: 204ms
573:    learn: 0.6871102        total: 274ms    remaining: 204ms
574:    learn: 0.6853623        total: 275ms    remaining: 203ms
575:    learn: 0.6836766        total: 276ms    remaining: 203ms
576:    learn: 0.6812385        total: 276ms    remaining: 203ms
577:    learn: 0.6795057        total: 277ms    remaining: 202ms
578:    learn: 0.6763886        total: 278ms    remaining: 202ms
579:    learn: 0.6747282        total: 278ms    remaining: 201ms
580:    learn: 0.6723357        total: 279ms    remaining: 201ms
581:    learn: 0.6706623        total: 279ms    remaining: 200ms
582:    learn: 0.6689550        total: 280ms    remaining: 200ms
583:    learn: 0.6672424        total: 280ms    remaining: 199ms
584:    learn: 0.6655451        total: 280ms    remaining: 199ms
585:    learn: 0.6637866        total: 281ms    remaining: 198ms
586:    learn: 0.6614446        total: 281ms    remaining: 198ms
587:    learn: 0.6596962        total: 282ms    remaining: 197ms
588:    learn: 0.6579642        total: 282ms    remaining: 197ms
589:    learn: 0.6563580        total: 283ms    remaining: 196ms
590:    learn: 0.6533346        total: 283ms    remaining: 196ms
591:    learn: 0.6510813        total: 284ms    remaining: 196ms
592:    learn: 0.6487811        total: 284ms    remaining: 195ms
593:    learn: 0.6471173        total: 285ms    remaining: 195ms
594:    learn: 0.6454728        total: 285ms    remaining: 194ms
595:    learn: 0.6425030        total: 286ms    remaining: 194ms
596:    learn: 0.6402398        total: 286ms    remaining: 193ms
597:    learn: 0.6372941        total: 287ms    remaining: 193ms
598:    learn: 0.6356832        total: 287ms    remaining: 192ms
599:    learn: 0.6340732        total: 288ms    remaining: 192ms
600:    learn: 0.6316746        total: 288ms    remaining: 191ms
601:    learn: 0.6287565        total: 289ms    remaining: 191ms
602:    learn: 0.6271585        total: 289ms    remaining: 190ms
603:    learn: 0.6255454        total: 289ms    remaining: 190ms
604:    learn: 0.6226723        total: 290ms    remaining: 189ms
605:    learn: 0.6210954        total: 291ms    remaining: 189ms
606:    learn: 0.6194888        total: 291ms    remaining: 189ms
607:    learn: 0.6173502        total: 292ms    remaining: 188ms
608:    learn: 0.6145219        total: 292ms    remaining: 188ms
609:    learn: 0.6129482        total: 293ms    remaining: 187ms
610:    learn: 0.6101166        total: 293ms    remaining: 187ms
611:    learn: 0.6086282        total: 294ms    remaining: 186ms
```

```
612:    learn: 0.6071361        total: 294ms    remaining: 186ms
613:    learn: 0.6055740        total: 295ms    remaining: 185ms
614:    learn: 0.6034121        total: 295ms    remaining: 185ms
615:    learn: 0.6019320        total: 296ms    remaining: 184ms
616:    learn: 0.6003457        total: 296ms    remaining: 184ms
617:    learn: 0.5982071        total: 297ms    remaining: 183ms
618:    learn: 0.5961045        total: 297ms    remaining: 183ms
619:    learn: 0.5945777        total: 297ms    remaining: 182ms
620:    learn: 0.5930681        total: 298ms    remaining: 182ms
621:    learn: 0.5909804        total: 299ms    remaining: 182ms
622:    learn: 0.5891414        total: 299ms    remaining: 181ms
623:    learn: 0.5876316        total: 300ms    remaining: 181ms
624:    learn: 0.5860792        total: 300ms    remaining: 180ms
625:    learn: 0.5845313        total: 301ms    remaining: 180ms
626:    learn: 0.5824541        total: 301ms    remaining: 179ms
627:    learn: 0.5809626        total: 302ms    remaining: 179ms
628:    learn: 0.5794932        total: 302ms    remaining: 178ms
629:    learn: 0.5780058        total: 302ms    remaining: 178ms
630:    learn: 0.5765953        total: 303ms    remaining: 177ms
631:    learn: 0.5751190        total: 303ms    remaining: 177ms
632:    learn: 0.5736598        total: 304ms    remaining: 176ms
633:    learn: 0.5722008        total: 304ms    remaining: 176ms
634:    learn: 0.5701929        total: 305ms    remaining: 175ms
635:    learn: 0.5681893        total: 305ms    remaining: 175ms
636:    learn: 0.5661763        total: 306ms    remaining: 174ms
637:    learn: 0.5641738        total: 306ms    remaining: 174ms
638:    learn: 0.5627222        total: 307ms    remaining: 173ms
639:    learn: 0.5607476        total: 307ms    remaining: 173ms
640:    learn: 0.5587390        total: 307ms    remaining: 172ms
641:    learn: 0.5573052        total: 308ms    remaining: 172ms
642:    learn: 0.5559669        total: 308ms    remaining: 171ms
643:    learn: 0.5545568        total: 309ms    remaining: 171ms
644:    learn: 0.5531347        total: 309ms    remaining: 170ms
645:    learn: 0.5516844        total: 310ms    remaining: 170ms
646:    learn: 0.5502695        total: 310ms    remaining: 169ms
647:    learn: 0.5483212        total: 311ms    remaining: 169ms
648:    learn: 0.5468861        total: 311ms    remaining: 168ms
649:    learn: 0.5454550        total: 312ms    remaining: 168ms
650:    learn: 0.5440759        total: 312ms    remaining: 167ms
651:    learn: 0.5426756        total: 313ms    remaining: 167ms
652:    learn: 0.5407153        total: 313ms    remaining: 166ms
653:    learn: 0.5393462        total: 314ms    remaining: 166ms
654:    learn: 0.5368802        total: 314ms    remaining: 165ms
655:    learn: 0.5349891        total: 314ms    remaining: 165ms
656:    learn: 0.5330996        total: 315ms    remaining: 164ms
657:    learn: 0.5317503        total: 315ms    remaining: 164ms
658:    learn: 0.5304066        total: 316ms    remaining: 163ms
659:    learn: 0.5291108        total: 316ms    remaining: 163ms
```

```
660:    learn: 0.5277739        total: 317ms    remaining: 162ms
661:    learn: 0.5258697        total: 317ms    remaining: 162ms
662:    learn: 0.5233712        total: 317ms    remaining: 161ms
663:    learn: 0.5220489        total: 318ms    remaining: 161ms
664:    learn: 0.5207299        total: 318ms    remaining: 160ms
665:    learn: 0.5193975        total: 319ms    remaining: 160ms
666:    learn: 0.5175802        total: 319ms    remaining: 159ms
667:    learn: 0.5162688        total: 320ms    remaining: 159ms
668:    learn: 0.5149626        total: 320ms    remaining: 158ms
669:    learn: 0.5136636        total: 321ms    remaining: 158ms
670:    learn: 0.5123655        total: 321ms    remaining: 157ms
671:    learn: 0.5110677        total: 322ms    remaining: 157ms
672:    learn: 0.5092804        total: 322ms    remaining: 157ms
673:    learn: 0.5079951        total: 323ms    remaining: 156ms
674:    learn: 0.5067116        total: 323ms    remaining: 156ms
675:    learn: 0.5049294        total: 324ms    remaining: 155ms
676:    learn: 0.5036414        total: 324ms    remaining: 155ms
677:    learn: 0.5023141        total: 325ms    remaining: 154ms
678:    learn: 0.5005487        total: 325ms    remaining: 154ms
679:    learn: 0.4992587        total: 326ms    remaining: 153ms
680:    learn: 0.4977029        total: 326ms    remaining: 153ms
681:    learn: 0.4964428        total: 327ms    remaining: 152ms
682:    learn: 0.4951695        total: 327ms    remaining: 152ms
683:    learn: 0.4939139        total: 327ms    remaining: 151ms
684:    learn: 0.4921908        total: 328ms    remaining: 151ms
685:    learn: 0.4909450        total: 328ms    remaining: 150ms
686:    learn: 0.4897036        total: 329ms    remaining: 150ms
687:    learn: 0.4884674        total: 329ms    remaining: 149ms
688:    learn: 0.4872323        total: 330ms    remaining: 149ms
689:    learn: 0.4859842        total: 330ms    remaining: 148ms
690:    learn: 0.4847428        total: 331ms    remaining: 148ms
691:    learn: 0.4830421        total: 331ms    remaining: 147ms
692:    learn: 0.4818032        total: 332ms    remaining: 147ms
693:    learn: 0.4805433        total: 332ms    remaining: 146ms
694:    learn: 0.4792750        total: 332ms    remaining: 146ms
695:    learn: 0.4780132        total: 333ms    remaining: 145ms
696:    learn: 0.4768065        total: 334ms    remaining: 145ms
697:    learn: 0.4756033        total: 334ms    remaining: 145ms
698:    learn: 0.4744038        total: 335ms    remaining: 144ms
699:    learn: 0.4732036        total: 335ms    remaining: 144ms
700:    learn: 0.4719919        total: 336ms    remaining: 143ms
701:    learn: 0.4707982        total: 336ms    remaining: 143ms
702:    learn: 0.4691474        total: 336ms    remaining: 142ms
703:    learn: 0.4679611        total: 337ms    remaining: 142ms
704:    learn: 0.4667652        total: 337ms    remaining: 141ms
705:    learn: 0.4655850        total: 338ms    remaining: 141ms
706:    learn: 0.4644107        total: 338ms    remaining: 140ms
707:    learn: 0.4632194        total: 339ms    remaining: 140ms
```

```
708:    learn: 0.4620827        total: 339ms    remaining: 139ms
709:    learn: 0.4609144        total: 340ms    remaining: 139ms
710:    learn: 0.4593103        total: 340ms    remaining: 138ms
711:    learn: 0.4581394        total: 340ms    remaining: 138ms
712:    learn: 0.4569815        total: 341ms    remaining: 137ms
713:    learn: 0.4558118        total: 341ms    remaining: 137ms
714:    learn: 0.4546599        total: 342ms    remaining: 136ms
715:    learn: 0.4535110        total: 342ms    remaining: 136ms
716:    learn: 0.4523526        total: 343ms    remaining: 135ms
717:    learn: 0.4512092        total: 343ms    remaining: 135ms
718:    learn: 0.4495885        total: 344ms    remaining: 134ms
719:    learn: 0.4484526        total: 344ms    remaining: 134ms
720:    learn: 0.4473197        total: 345ms    remaining: 133ms
721:    learn: 0.4461921        total: 345ms    remaining: 133ms
722:    learn: 0.4445919        total: 346ms    remaining: 133ms
723:    learn: 0.4434597        total: 346ms    remaining: 132ms
724:    learn: 0.4423232        total: 347ms    remaining: 132ms
725:    learn: 0.4407740        total: 347ms    remaining: 131ms
726:    learn: 0.4396610        total: 348ms    remaining: 131ms
727:    learn: 0.4385505        total: 349ms    remaining: 130ms
728:    learn: 0.4370153        total: 349ms    remaining: 130ms
729:    learn: 0.4359135        total: 350ms    remaining: 129ms
730:    learn: 0.4343863        total: 350ms    remaining: 129ms
731:    learn: 0.4323942        total: 351ms    remaining: 128ms
732:    learn: 0.4312893        total: 351ms    remaining: 128ms
733:    learn: 0.4301853        total: 352ms    remaining: 127ms
734:    learn: 0.4290988        total: 352ms    remaining: 127ms
735:    learn: 0.4279823        total: 353ms    remaining: 126ms
736:    learn: 0.4269016        total: 353ms    remaining: 126ms
737:    learn: 0.4254090        total: 354ms    remaining: 126ms
738:    learn: 0.4239337        total: 355ms    remaining: 125ms
739:    learn: 0.4228638        total: 355ms    remaining: 125ms
740:    learn: 0.4217962        total: 356ms    remaining: 124ms
741:    learn: 0.4207313        total: 356ms    remaining: 124ms
742:    learn: 0.4196544        total: 357ms    remaining: 123ms
743:    learn: 0.4185809        total: 357ms    remaining: 123ms
744:    learn: 0.4171300        total: 358ms    remaining: 123ms
745:    learn: 0.4160457        total: 358ms    remaining: 122ms
746:    learn: 0.4150271        total: 359ms    remaining: 122ms
747:    learn: 0.4139800        total: 360ms    remaining: 121ms
748:    learn: 0.4129369        total: 360ms    remaining: 121ms
749:    learn: 0.4118948        total: 361ms    remaining: 120ms
750:    learn: 0.4108554        total: 361ms    remaining: 120ms
751:    learn: 0.4097879        total: 362ms    remaining: 119ms
752:    learn: 0.4087539        total: 362ms    remaining: 119ms
753:    learn: 0.4077226        total: 363ms    remaining: 118ms
754:    learn: 0.4066944        total: 363ms    remaining: 118ms
755:    learn: 0.4056685        total: 364ms    remaining: 117ms
```

```
756:    learn: 0.4042635        total: 364ms    remaining: 117ms
757:    learn: 0.4032455        total: 365ms    remaining: 116ms
758:    learn: 0.4018496        total: 365ms    remaining: 116ms
759:    learn: 0.4008365        total: 366ms    remaining: 115ms
760:    learn: 0.3998255        total: 366ms    remaining: 115ms
761:    learn: 0.3988172        total: 367ms    remaining: 115ms
762:    learn: 0.3974374        total: 367ms    remaining: 114ms
763:    learn: 0.3964351        total: 368ms    remaining: 114ms
764:    learn: 0.3954360        total: 368ms    remaining: 113ms
765:    learn: 0.3944390        total: 369ms    remaining: 113ms
766:    learn: 0.3934445        total: 369ms    remaining: 112ms
767:    learn: 0.3924531        total: 370ms    remaining: 112ms
768:    learn: 0.3914643        total: 370ms    remaining: 111ms
769:    learn: 0.3904775        total: 371ms    remaining: 111ms
770:    learn: 0.3894933        total: 371ms    remaining: 110ms
771:    learn: 0.3885117        total: 372ms    remaining: 110ms
772:    learn: 0.3875325        total: 373ms    remaining: 109ms
773:    learn: 0.3865559        total: 373ms    remaining: 109ms
774:    learn: 0.3855819        total: 374ms    remaining: 109ms
775:    learn: 0.3846103        total: 375ms    remaining: 108ms
776:    learn: 0.3836421        total: 375ms    remaining: 108ms
777:    learn: 0.3826761        total: 376ms    remaining: 107ms
778:    learn: 0.3817125        total: 376ms    remaining: 107ms
779:    learn: 0.3807512        total: 377ms    remaining: 106ms
780:    learn: 0.3797921        total: 377ms    remaining: 106ms
781:    learn: 0.3788360        total: 377ms    remaining: 105ms
782:    learn: 0.3778819        total: 378ms    remaining: 105ms
783:    learn: 0.3765650        total: 379ms    remaining: 104ms
784:    learn: 0.3756166        total: 379ms    remaining: 104ms
785:    learn: 0.3746707        total: 380ms    remaining: 103ms
786:    learn: 0.3737272        total: 380ms    remaining: 103ms
787:    learn: 0.3727862        total: 381ms    remaining: 102ms
788:    learn: 0.3714877        total: 381ms    remaining: 102ms
789:    learn: 0.3705527        total: 382ms    remaining: 101ms
790:    learn: 0.3696198        total: 382ms    remaining: 101ms
791:    learn: 0.3686893        total: 382ms    remaining: 100ms
792:    learn: 0.3677612        total: 383ms    remaining: 100ms
793:    learn: 0.3664809        total: 384ms    remaining: 99.5ms
794:    learn: 0.3652056        total: 384ms    remaining: 99ms
795:    learn: 0.3642868        total: 385ms    remaining: 98.5ms
796:    learn: 0.3633700        total: 385ms    remaining: 98.1ms
797:    learn: 0.3624461        total: 386ms    remaining: 97.6ms
798:    learn: 0.3615339        total: 386ms    remaining: 97.1ms
799:    learn: 0.3602765        total: 386ms    remaining: 96.6ms
800:    learn: 0.3590241        total: 387ms    remaining: 96.1ms
801:    learn: 0.3581208        total: 387ms    remaining: 95.6ms
802:    learn: 0.3572197        total: 388ms    remaining: 95.1ms
803:    learn: 0.3563209        total: 388ms    remaining: 94.7ms
```

```
804:    learn: 0.3554244        total: 389ms    remaining: 94.2ms
805:    learn: 0.3545302        total: 390ms    remaining: 93.8ms
806:    learn: 0.3536383        total: 390ms    remaining: 93.3ms
807:    learn: 0.3527488        total: 391ms    remaining: 92.8ms
808:    learn: 0.3518615        total: 391ms    remaining: 92.3ms
809:    learn: 0.3509765        total: 391ms    remaining: 91.8ms
810:    learn: 0.3500816        total: 392ms    remaining: 91.3ms
811:    learn: 0.3492016        total: 392ms    remaining: 90.8ms
812:    learn: 0.3483235        total: 393ms    remaining: 90.4ms
813:    learn: 0.3474481        total: 393ms    remaining: 89.9ms
814:    learn: 0.3465745        total: 394ms    remaining: 89.4ms
815:    learn: 0.3457032        total: 394ms    remaining: 88.9ms
816:    learn: 0.3448341        total: 395ms    remaining: 88.5ms
817:    learn: 0.3439672        total: 396ms    remaining: 88.1ms
818:    learn: 0.3431030        total: 396ms    remaining: 87.6ms
819:    learn: 0.3422406        total: 397ms    remaining: 87.1ms
820:    learn: 0.3413809        total: 397ms    remaining: 86.6ms
821:    learn: 0.3405234        total: 398ms    remaining: 86.1ms
822:    learn: 0.3396677        total: 398ms    remaining: 85.6ms
823:    learn: 0.3388141        total: 398ms    remaining: 85.1ms
824:    learn: 0.3379633        total: 399ms    remaining: 84.6ms
825:    learn: 0.3371141        total: 399ms    remaining: 84.1ms
826:    learn: 0.3362671        total: 400ms    remaining: 83.7ms
827:    learn: 0.3354224        total: 401ms    remaining: 83.2ms
828:    learn: 0.3345797        total: 401ms    remaining: 82.8ms
829:    learn: 0.3337393        total: 402ms    remaining: 82.3ms
830:    learn: 0.3329015        total: 402ms    remaining: 81.8ms
831:    learn: 0.3320654        total: 403ms    remaining: 81.3ms
832:    learn: 0.3312314        total: 403ms    remaining: 80.8ms
833:    learn: 0.3303996        total: 404ms    remaining: 80.3ms
834:    learn: 0.3295699        total: 404ms    remaining: 79.8ms
835:    learn: 0.3287424        total: 405ms    remaining: 79.4ms
836:    learn: 0.3279170        total: 405ms    remaining: 78.9ms
837:    learn: 0.3270942        total: 406ms    remaining: 78.4ms
838:    learn: 0.3262730        total: 406ms    remaining: 77.9ms
839:    learn: 0.3254540        total: 407ms    remaining: 77.4ms
840:    learn: 0.3246375        total: 407ms    remaining: 76.9ms
841:    learn: 0.3238227        total: 407ms    remaining: 76.5ms
842:    learn: 0.3230099        total: 408ms    remaining: 76ms
843:    learn: 0.3221993        total: 408ms    remaining: 75.5ms
844:    learn: 0.3213907        total: 409ms    remaining: 75ms
845:    learn: 0.3205842        total: 409ms    remaining: 74.5ms
846:    learn: 0.3197798        total: 410ms    remaining: 74ms
847:    learn: 0.3189775        total: 410ms    remaining: 73.5ms
848:    learn: 0.3178668        total: 411ms    remaining: 73ms
849:    learn: 0.3170693        total: 411ms    remaining: 72.5ms
850:    learn: 0.3162739        total: 412ms    remaining: 72.1ms
851:    learn: 0.3154805        total: 412ms    remaining: 71.6ms
```

```
852:    learn: 0.3146891        total: 413ms    remaining: 71.1ms
853:    learn: 0.3138998        total: 413ms    remaining: 70.6ms
854:    learn: 0.3131126        total: 414ms    remaining: 70.2ms
855:    learn: 0.3123273        total: 414ms    remaining: 69.7ms
856:    learn: 0.3115441        total: 415ms    remaining: 69.2ms
857:    learn: 0.3107629        total: 415ms    remaining: 68.7ms
858:    learn: 0.3099837        total: 416ms    remaining: 68.2ms
859:    learn: 0.3092065        total: 416ms    remaining: 67.7ms
860:    learn: 0.3084313        total: 417ms    remaining: 67.2ms
861:    learn: 0.3076581        total: 417ms    remaining: 66.8ms
862:    learn: 0.3068869        total: 417ms    remaining: 66.3ms
863:    learn: 0.3061177        total: 418ms    remaining: 65.8ms
864:    learn: 0.3053504        total: 418ms    remaining: 65.3ms
865:    learn: 0.3045851        total: 419ms    remaining: 64.8ms
866:    learn: 0.3038218        total: 419ms    remaining: 64.3ms
867:    learn: 0.3030605        total: 420ms    remaining: 63.8ms
868:    learn: 0.3023011        total: 420ms    remaining: 63.3ms
869:    learn: 0.3015436        total: 421ms    remaining: 62.9ms
870:    learn: 0.3007881        total: 421ms    remaining: 62.4ms
871:    learn: 0.3000346        total: 422ms    remaining: 61.9ms
872:    learn: 0.2992830        total: 422ms    remaining: 61.4ms
873:    learn: 0.2985333        total: 422ms    remaining: 60.9ms
874:    learn: 0.2977856        total: 423ms    remaining: 60.4ms
875:    learn: 0.2970397        total: 423ms    remaining: 59.9ms
876:    learn: 0.2962958        total: 424ms    remaining: 59.5ms
877:    learn: 0.2955538        total: 425ms    remaining: 59ms
878:    learn: 0.2948140        total: 425ms    remaining: 58.5ms
879:    learn: 0.2940758        total: 426ms    remaining: 58.1ms
880:    learn: 0.2933395        total: 426ms    remaining: 57.6ms
881:    learn: 0.2926055        total: 427ms    remaining: 57.1ms
882:    learn: 0.2918729        total: 427ms    remaining: 56.6ms
883:    learn: 0.2911423        total: 428ms    remaining: 56.1ms
884:    learn: 0.2904139        total: 428ms    remaining: 55.6ms
885:    learn: 0.2896870        total: 429ms    remaining: 55.1ms
886:    learn: 0.2889619        total: 429ms    remaining: 54.7ms
887:    learn: 0.2882391        total: 429ms    remaining: 54.2ms
888:    learn: 0.2875178        total: 430ms    remaining: 53.7ms
889:    learn: 0.2867983        total: 430ms    remaining: 53.2ms
890:    learn: 0.2860811        total: 431ms    remaining: 52.7ms
891:    learn: 0.2853653        total: 431ms    remaining: 52.2ms
892:    learn: 0.2846514        total: 432ms    remaining: 51.7ms
893:    learn: 0.2839393        total: 432ms    remaining: 51.3ms
894:    learn: 0.2832290        total: 433ms    remaining: 50.8ms
895:    learn: 0.2825206        total: 433ms    remaining: 50.3ms
896:    learn: 0.2818139        total: 434ms    remaining: 49.8ms
897:    learn: 0.2811091        total: 434ms    remaining: 49.3ms
898:    learn: 0.2804064        total: 435ms    remaining: 48.9ms
899:    learn: 0.2797052        total: 436ms    remaining: 48.4ms
```

```
900:    learn: 0.2790058        total: 436ms    remaining: 47.9ms
901:    learn: 0.2783082        total: 437ms    remaining: 47.5ms
902:    learn: 0.2776124        total: 437ms    remaining: 47ms
903:    learn: 0.2769184        total: 438ms    remaining: 46.5ms
904:    learn: 0.2762265        total: 438ms    remaining: 46ms
905:    learn: 0.2755363        total: 439ms    remaining: 45.5ms
906:    learn: 0.2748476        total: 439ms    remaining: 45.1ms
907:    learn: 0.2741607        total: 440ms    remaining: 44.6ms
908:    learn: 0.2734755        total: 440ms    remaining: 44.1ms
909:    learn: 0.2727921        total: 441ms    remaining: 43.6ms
910:    learn: 0.2721105        total: 442ms    remaining: 43.1ms
911:    learn: 0.2714305        total: 443ms    remaining: 42.7ms
912:    learn: 0.2707524        total: 443ms    remaining: 42.2ms
913:    learn: 0.2700760        total: 444ms    remaining: 41.7ms
914:    learn: 0.2694013        total: 444ms    remaining: 41.3ms
915:    learn: 0.2687286        total: 445ms    remaining: 40.8ms
916:    learn: 0.2680574        total: 445ms    remaining: 40.3ms
917:    learn: 0.2673879        total: 445ms    remaining: 39.8ms
918:    learn: 0.2667201        total: 446ms    remaining: 39.3ms
919:    learn: 0.2660543        total: 446ms    remaining: 38.8ms
920:    learn: 0.2653900        total: 447ms    remaining: 38.3ms
921:    learn: 0.2647273        total: 447ms    remaining: 37.9ms
922:    learn: 0.2640663        total: 448ms    remaining: 37.4ms
923:    learn: 0.2634071        total: 449ms    remaining: 36.9ms
924:    learn: 0.2627495        total: 449ms    remaining: 36.4ms
925:    learn: 0.2620939        total: 450ms    remaining: 35.9ms
926:    learn: 0.2614397        total: 450ms    remaining: 35.4ms
927:    learn: 0.2607872        total: 451ms    remaining: 35ms
928:    learn: 0.2601366        total: 451ms    remaining: 34.5ms
929:    learn: 0.2594874        total: 452ms    remaining: 34ms
930:    learn: 0.2588402        total: 452ms    remaining: 33.5ms
931:    learn: 0.2581943        total: 453ms    remaining: 33ms
932:    learn: 0.2575501        total: 453ms    remaining: 32.5ms
933:    learn: 0.2569076        total: 454ms    remaining: 32.1ms
934:    learn: 0.2562667        total: 454ms    remaining: 31.6ms
935:    learn: 0.2556274        total: 455ms    remaining: 31.1ms
936:    learn: 0.2549898        total: 455ms    remaining: 30.6ms
937:    learn: 0.2543538        total: 455ms    remaining: 30.1ms
938:    learn: 0.2537194        total: 456ms    remaining: 29.6ms
939:    learn: 0.2530867        total: 456ms    remaining: 29.1ms
940:    learn: 0.2524556        total: 457ms    remaining: 28.6ms
941:    learn: 0.2518261        total: 458ms    remaining: 28.2ms
942:    learn: 0.2511985        total: 458ms    remaining: 27.7ms
943:    learn: 0.2505725        total: 459ms    remaining: 27.2ms
944:    learn: 0.2499479        total: 459ms    remaining: 26.7ms
945:    learn: 0.2493248        total: 460ms    remaining: 26.3ms
946:    learn: 0.2487034        total: 460ms    remaining: 25.8ms
947:    learn: 0.2480835        total: 461ms    remaining: 25.3ms
```

```
948:    learn: 0.2474652        total: 461ms    remaining: 24.8ms
949:    learn: 0.2468485        total: 462ms    remaining: 24.3ms
950:    learn: 0.2462334        total: 463ms    remaining: 23.8ms
951:    learn: 0.2456199        total: 463ms    remaining: 23.3ms
952:    learn: 0.2450080        total: 464ms    remaining: 22.9ms
953:    learn: 0.2443976        total: 465ms    remaining: 22.4ms
954:    learn: 0.2437888        total: 465ms    remaining: 21.9ms
955:    learn: 0.2431815        total: 466ms    remaining: 21.4ms
956:    learn: 0.2425760        total: 467ms    remaining: 21ms
957:    learn: 0.2419719        total: 467ms    remaining: 20.5ms
958:    learn: 0.2413693        total: 467ms    remaining: 20ms
959:    learn: 0.2407682        total: 468ms    remaining: 19.5ms
960:    learn: 0.2401687        total: 468ms    remaining: 19ms
961:    learn: 0.2395707        total: 469ms    remaining: 18.5ms
962:    learn: 0.2389743        total: 469ms    remaining: 18ms
963:    learn: 0.2383794        total: 470ms    remaining: 17.6ms
964:    learn: 0.2377860        total: 471ms    remaining: 17.1ms
965:    learn: 0.2371941        total: 471ms    remaining: 16.6ms
966:    learn: 0.2366038        total: 472ms    remaining: 16.1ms
967:    learn: 0.2360150        total: 472ms    remaining: 15.6ms
968:    learn: 0.2354276        total: 473ms    remaining: 15.1ms
969:    learn: 0.2348418        total: 473ms    remaining: 14.6ms
970:    learn: 0.2342575        total: 474ms    remaining: 14.2ms
971:    learn: 0.2336747        total: 475ms    remaining: 13.7ms
972:    learn: 0.2330934        total: 476ms    remaining: 13.2ms
973:    learn: 0.2325135        total: 476ms    remaining: 12.7ms
974:    learn: 0.2319352        total: 477ms    remaining: 12.2ms
975:    learn: 0.2313583        total: 477ms    remaining: 11.7ms
976:    learn: 0.2307829        total: 478ms    remaining: 11.3ms
977:    learn: 0.2302090        total: 479ms    remaining: 10.8ms
978:    learn: 0.2296366        total: 480ms    remaining: 10.3ms
979:    learn: 0.2290656        total: 481ms    remaining: 9.81ms
980:    learn: 0.2284961        total: 481ms    remaining: 9.32ms
981:    learn: 0.2279281        total: 482ms    remaining: 8.83ms
982:    learn: 0.2273615        total: 482ms    remaining: 8.34ms
983:    learn: 0.2267963        total: 483ms    remaining: 7.85ms
984:    learn: 0.2262327        total: 483ms    remaining: 7.36ms
985:    learn: 0.2256704        total: 484ms    remaining: 6.87ms
986:    learn: 0.2251096        total: 485ms    remaining: 6.38ms
987:    learn: 0.2245505        total: 485ms    remaining: 5.89ms
988:    learn: 0.2239925        total: 486ms    remaining: 5.4ms
989:    learn: 0.2234362        total: 486ms    remaining: 4.91ms
990:    learn: 0.2228812        total: 487ms    remaining: 4.42ms
991:    learn: 0.2223275        total: 487ms    remaining: 3.93ms
992:    learn: 0.2217753        total: 488ms    remaining: 3.44ms
993:    learn: 0.2212247        total: 488ms    remaining: 2.95ms
994:    learn: 0.2206753        total: 489ms    remaining: 2.46ms
995:    learn: 0.2201273        total: 490ms    remaining: 1.97ms
```

```
996:    learn: 0.2195807        total: 490ms    remaining: 1.48ms
997:    learn: 0.2190355        total: 491ms    remaining: 983us
998:    learn: 0.2184917        total: 492ms    remaining: 492us
999:    learn: 0.2179493        total: 492ms    remaining: 0us
```

[176]:
```
GridSearchCV(cv=5,
             estimator=<catboost.core.CatBoostRegressor object at
0x000000000D445520>,
             n_jobs=-1,
             param_grid={'depth': [6, 8, 10], 'iterations': [100, 500, 1000],
                         'learning_rate': [0.01, 0.02, 0.1]},
             scoring='neg_mean_absolute_error', verbose=False)
```

[177]:
```
print(cat_grid.best_score_)
print(cat_grid.best_params_)
```

```
-4.194920093013392
{'depth': 6, 'iterations': 1000, 'learning_rate': 0.01}
```

[178]:
```
#cat_grid.best_estimator_.get_feature_importance()
```

[179]:
```
#plt.barh(cat_grid.best_estimator_.feature_names_, cat_grid.best_estimator_.
 ↪get_feature_importance())
#plt.xlabel("CatBoostRegressor Feature Importance")
```

[ ]:
```

```

[ ]:
```

```

3.6.7 Fit the final model

[180]:
```
cat_grid.best_params_
```

[180]:
```
{'depth': 6, 'iterations': 1000, 'learning_rate': 0.01}
```

[181]:
```
cbc_final = CatBoostRegressor(**model.get_params())
```

[182]:
```
#set optimal parameters
cbc_final.set_params(**cat_grid.best_params_)
```

[182]:
```
<catboost.core.CatBoostRegressor at 0xd399820>
```

[183]:
```
#fit on the whole dataset
cbc_final.fit(X, y,verbose_eval=False)
```

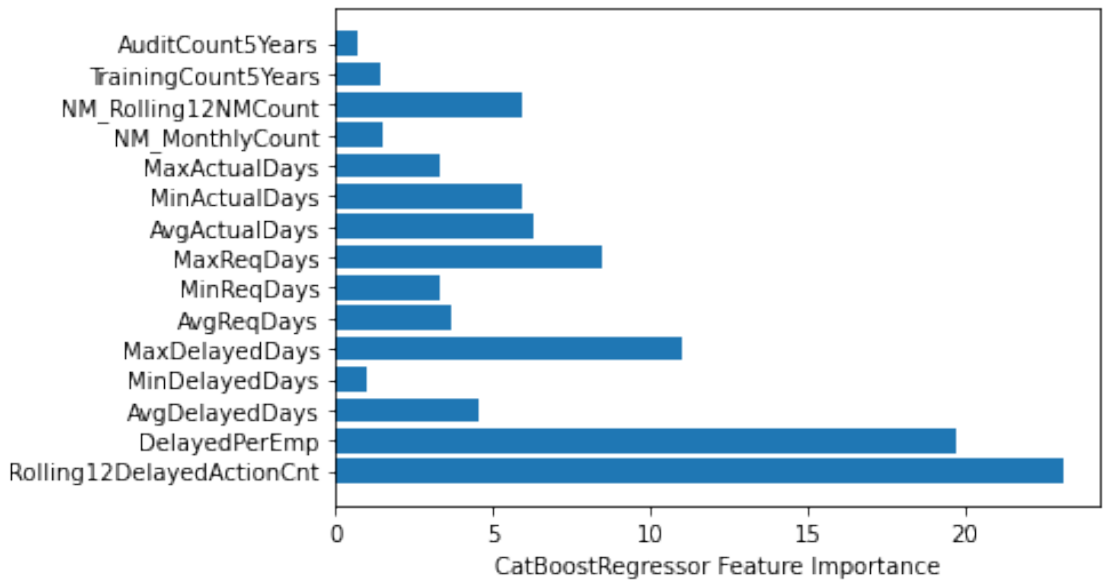
[183]:
```
<catboost.core.CatBoostRegressor at 0xd399820>
```

[184]:
```
cbc_final.feature_importances_
```

[184]:
```
array([23.14162881, 19.73935288,  4.53083662,  0.99795544, 10.97748201,
        3.68741119,  3.2794091 ,  8.47629551,  6.31178563,  5.89638962,
        3.33193311,  1.52601635,  5.93765284,  1.4549922 ,  0.71085868])
```

[185]:
```
plt.barh(cbc_final.feature_names_, cbc_final.feature_importances_)
plt.xlabel("CatBoostRegressor Feature Importance")
```

[185]:
```
Text(0.5, 0, 'CatBoostRegressor Feature Importance')
```



[186]:
```
sorted_idx = cbc_final.feature_importances_.argsort()
rank_cat_c_o = pd.DataFrame( (np.arange(1,len(cbc_final.
 ↪feature_importances_[sorted_idx])+1,1)),index=X.columns[sorted_idx][::-1],␣
 ↪columns=["CatBoost (Optimised)"])
change =pd.DataFrame( rank_cat_c_b["CatBoost (Default)"]-rank_cat_c_o["CatBoost␣
 ↪(Optimised)"], columns=["ranking change"])
ranking_ = pd.concat([rank_cat_c_o, rank_cat_c_b, change], axis=1)
ranking_
```

[186]:

	CatBoost (Optimised)	CatBoost (Default)	\
Rolling12DelayedActionCnt	1	2	
DelayedPerEmp	2	1	
MaxDelayedDays	3	3	
MaxReqDays	4	5	
AvgActualDays	5	6	
NM_Rolling12NMCount	6	4	

```
MinActualDays                                  7                    8
AvgDelayedDays                                 8                    7
AvgReqDays                                     9                    9
MaxActualDays                                 10                   10
MinReqDays                                    11                   13
NM_MonthlyCount                               12                   11
TrainingCount5Years                           13                   12
MinDelayedDays                                14                   14
AuditCount5Years                              15                   15

                            ranking change
Rolling12DelayedActionCnt                1
DelayedPerEmp                           -1
MaxDelayedDays                           0
MaxReqDays                               1
AvgActualDays                            1
NM_Rolling12NMCount                     -2
MinActualDays                            1
AvgDelayedDays                          -1
AvgReqDays                               0
MaxActualDays                            0
MinReqDays                               2
NM_MonthlyCount                         -1
TrainingCount5Years                     -1
MinDelayedDays                           0
AuditCount5Years                         0
```

Similarly to the CatBoost implementation in the classification problem, the hyperparameters slightly affect the ranking of the feature importance within the given hyperparameter searching space. Compared to the number of features selected by the XGboost, CatBoost selects much more feature than the XGBoost. In terms of values of feature importance, primary features selected by the CatBoost are not as dominate as the XGboost does to other features, suggesting its different construction of the model from the XGBoost's.

[187]:
```
rank_c = pd.concat([rank_c, rank_cat_c_o], axis=1)
```

[188]:
```
rank_c
```

[188]:
```
                            GradientBoosting  XGBoost (Optimised)  \
Rolling12DelayedActionCnt                  1                  1.0
DelayedPerEmp                              2                  NaN
AvgReqDays                                 3                  3.0
NM_Rolling12NMCount                        4                  5.0
AvgActualDays                              5                  6.0
AvgDelayedDays                             6                  2.0
MinActualDays                              7                  NaN
MinReqDays                                 8                  NaN
```

```
MaxDelayedDays                          9                   NaN
NM_MonthlyCount                        10                   NaN
MaxReqDays                             11                   NaN
AuditCount5Years                       12                   NaN
TrainingCount5Years                    13                   4.0
MaxActualDays                          14                   NaN
MinDelayedDays                         15                   NaN

                           CatBoost (Optimised)
Rolling12DelayedActionCnt                     1
DelayedPerEmp                                 2
AvgReqDays                                    9
NM_Rolling12NMCount                           6
AvgActualDays                                 5
AvgDelayedDays                                8
MinActualDays                                 7
MinReqDays                                   11
MaxDelayedDays                                3
NM_MonthlyCount                              12
MaxReqDays                                    4
AuditCount5Years                             15
TrainingCount5Years                          13
MaxActualDays                                10
MinDelayedDays                               14
```

[189]:
```
#save the ranking file
rank_c.index.name = "Features"
rank_c.to_csv('result/Feature_importance_ranking_regressor.csv')
```

3.7 Section Summary

TThis section investigates implementations for a safety regression problem. Five implementations included are GradientBoosting,Histogram-based Gradient Boosting, XGBoost, LightGBM and CatBoost. These implementations reach a decent accuracy when they trained models by a safety dataset. Three implementations, GradientBoosting, XGB, and CatBoost, also produce feature importances with the models produced. The rankings of feature importances suggest that the `Rolling12DelayedActionCnt`is the most important feature for the construction of the models. Although the definitions of feature importance differ among this investigation and the previous one, their ranking results are relatively consistent, suggesting feature importance can be considered as a critical factor for safety prediction.

This section also conducts hyperparameter-tuning for XGBoost algorithm implementations: XGBoost, LightGBM and CatBoost. Models with tuned hyperparameters have better performances than their default models. As hyperparameters impact the construction of a model, the hyperparameter-tuning process is preferred for a better performance of the model when it faces a complex dataset.

[190]:
```
rank_c
```

[190]:
```
                            GradientBoosting  XGBoost (Optimised)  \
Features
Rolling12DelayedActionCnt                  1                  1.0
DelayedPerEmp                              2                  NaN
AvgReqDays                                 3                  3.0
NM_Rolling12NMCount                        4                  5.0
AvgActualDays                              5                  6.0
AvgDelayedDays                             6                  2.0
MinActualDays                              7                  NaN
MinReqDays                                 8                  NaN
MaxDelayedDays                             9                  NaN
NM_MonthlyCount                           10                  NaN
MaxReqDays                                11                  NaN
AuditCount5Years                          12                  NaN
TrainingCount5Years                       13                  4.0
MaxActualDays                             14                  NaN
MinDelayedDays                            15                  NaN

                            CatBoost (Optimised)
Features
Rolling12DelayedActionCnt                      1
DelayedPerEmp                                  2
AvgReqDays                                     9
NM_Rolling12NMCount                            6
AvgActualDays                                  5
AvgDelayedDays                                 8
MinActualDays                                  7
MinReqDays                                    11
MaxDelayedDays                                 3
NM_MonthlyCount                               12
MaxReqDays                                     4
AuditCount5Years                              15
TrainingCount5Years                           13
MaxActualDays                                 10
MinDelayedDays                                14
```

[191]:
```
print("Accuracy:")
{k: v for k, v in sorted(accuracy.items(), key=lambda i: i[1], reverse=True)}
```

```
Accuracy:
```

[191]:
```
{'GradientBoosting': -1.459,
 'XGB': -1.581,
 'CatBoost': -2.119,
 'HistGradientBoosting': -5.122,
```

```
 'LGBM': -5.122}
```

[192]:
```python
print("Speed:")
{k: v for k, v in sorted(speed.items(), key=lambda i: i[1], reverse=False)}
```

```
Speed:
```

[192]:
```
{'LGBM': 1.583,
 'GradientBoosting': 3.78,
 'HistGradientBoosting': 5.728,
 'XGB': 6.869,
 'CatBoost': 132.857}
```

[ ]:
```python

```